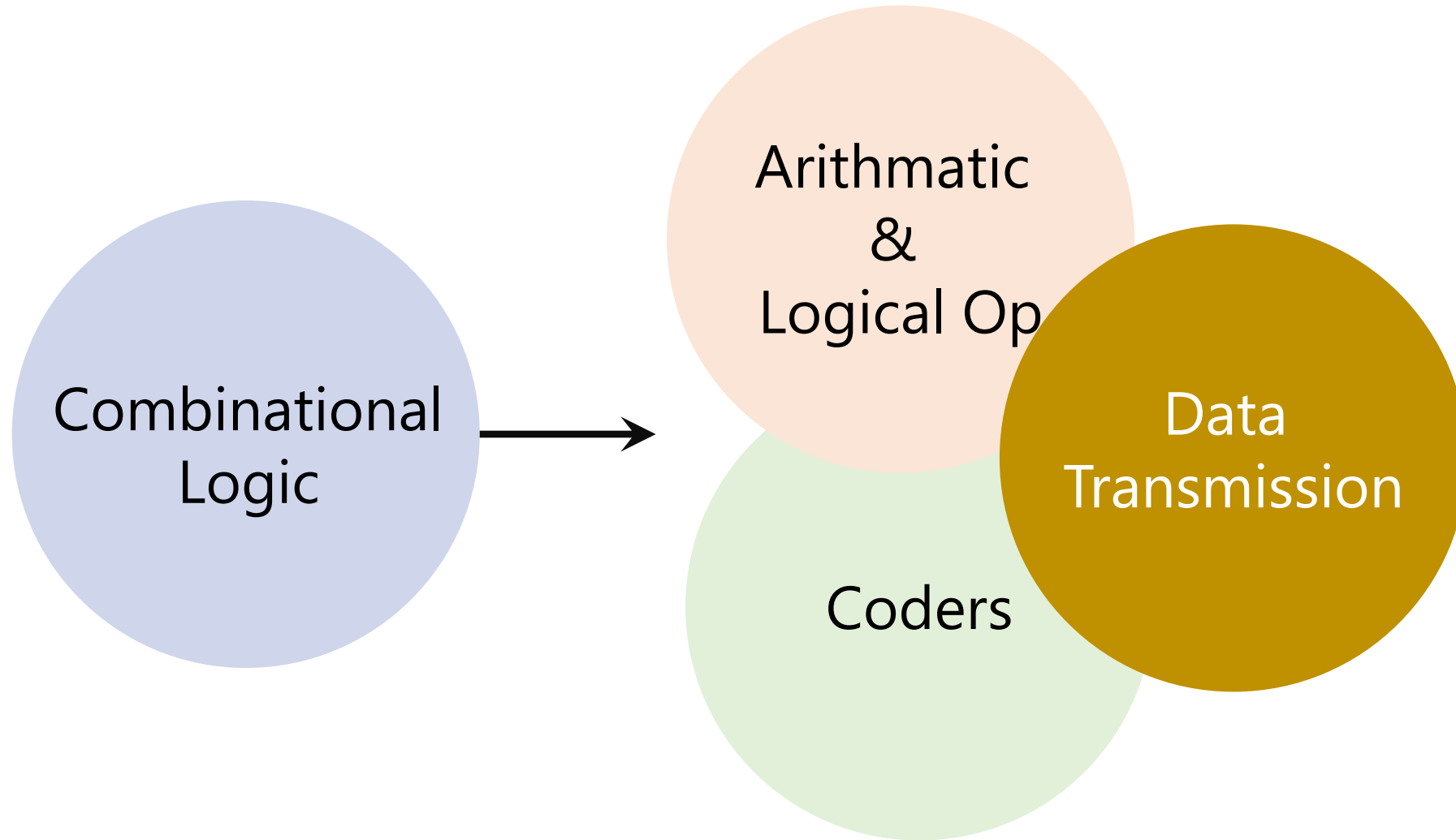




THE INTERNATIONAL Calculator Collector

Spring 1993

Issue No. 1



The Cat Tech circa 1967

Photo Courtesy Texas Instruments

The Beginning

If you're past your mid-30s, you probably remember your first simple hand-held calculator costing over \$50 (in early 1970's dollars). Depending how much older you are, your first could have been upwards to \$400. And we're just talking the basic four functions here — addition, subtraction, multiplication, and division. Percentage and memory features were extra (if they were even available at that point in time).

Company Profile:



Who can forget the "Bowmar Brain" series of calculators from the early '70s?

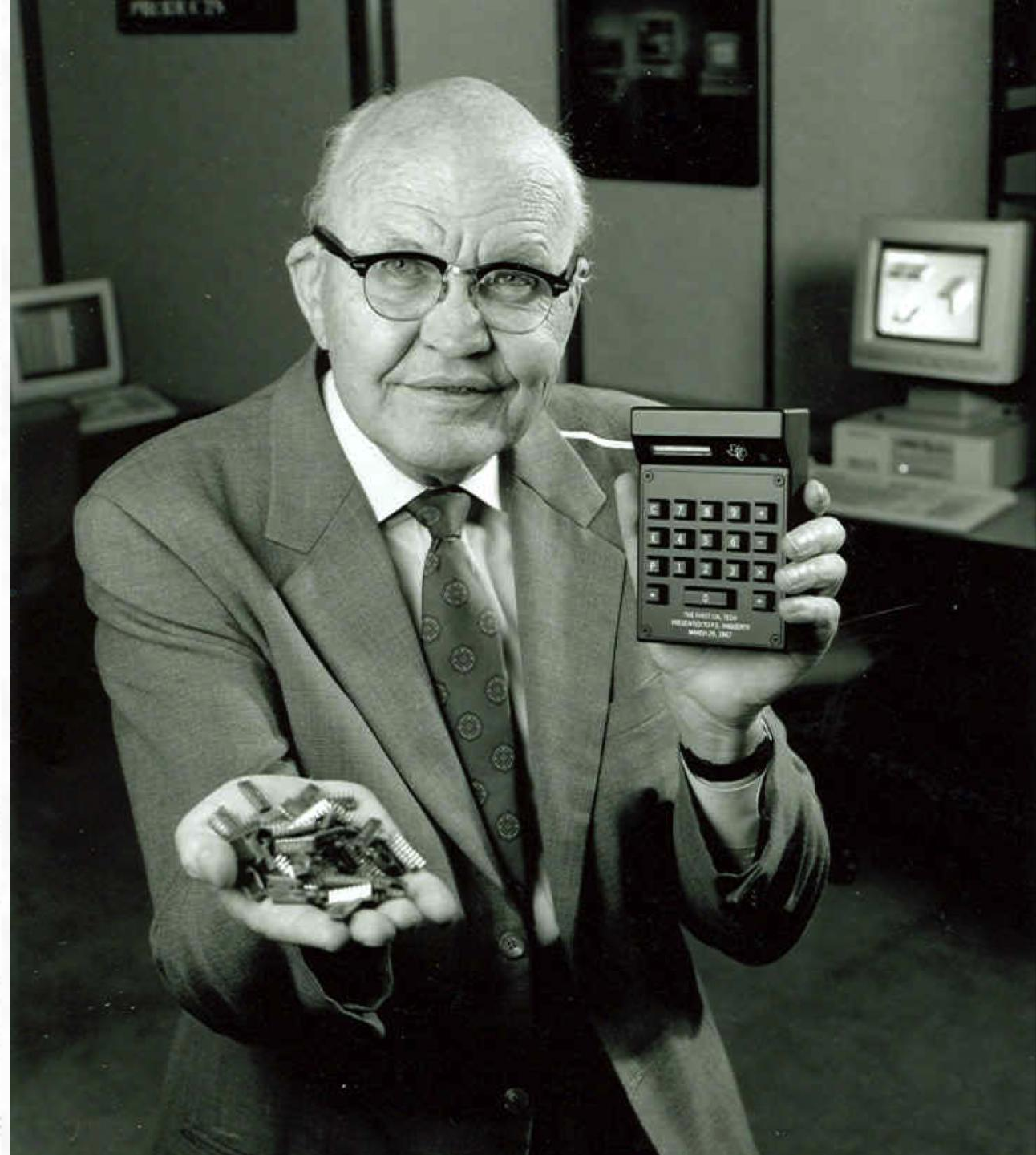
Bowmar was the first American company that made and sold their own line of portable electronic machines.

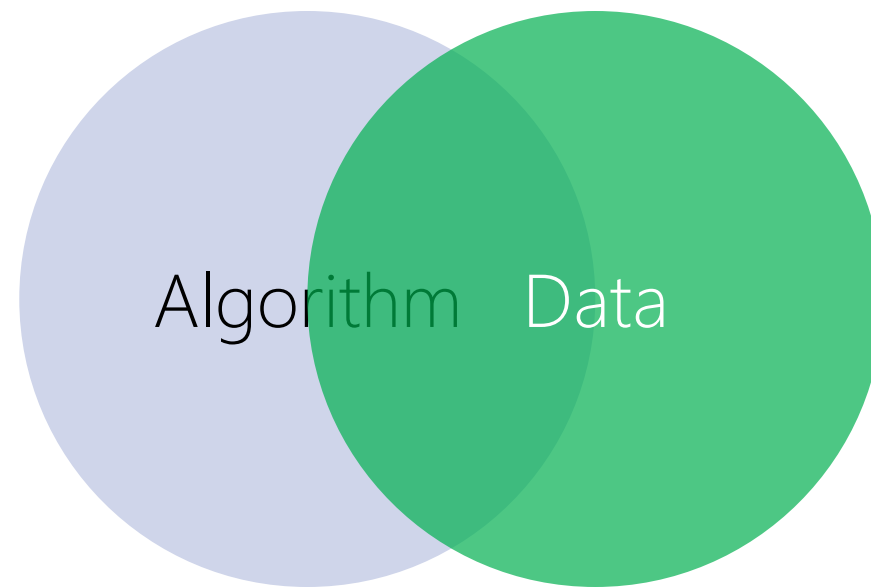
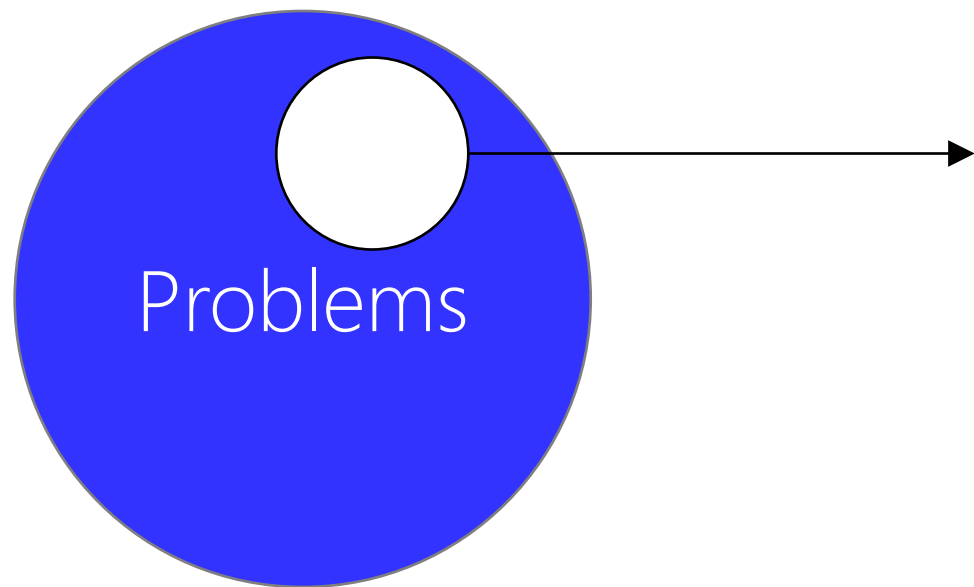
The story starts around 1970 when Bowmar, then a manufacturer of Light Emitting Diodes (LEDs), tried to sell their numeric display product to Japanese manufacturers for use in their electronic products.

Bowmar wasn't too successful. The Japanese were using a fluorescent style display that was cheaper and had a few design features the manufacturers liked better.

So, president Ed White, a consummate entrepreneur, and his staff came up with an even better idea — make the whole electronic calculator themselves.

Up to now, most of the so-called "portable" calculators





Design a Computer System

John von Neumann

([/vɒn 'nɔɪmən/](#))

1903 –1957

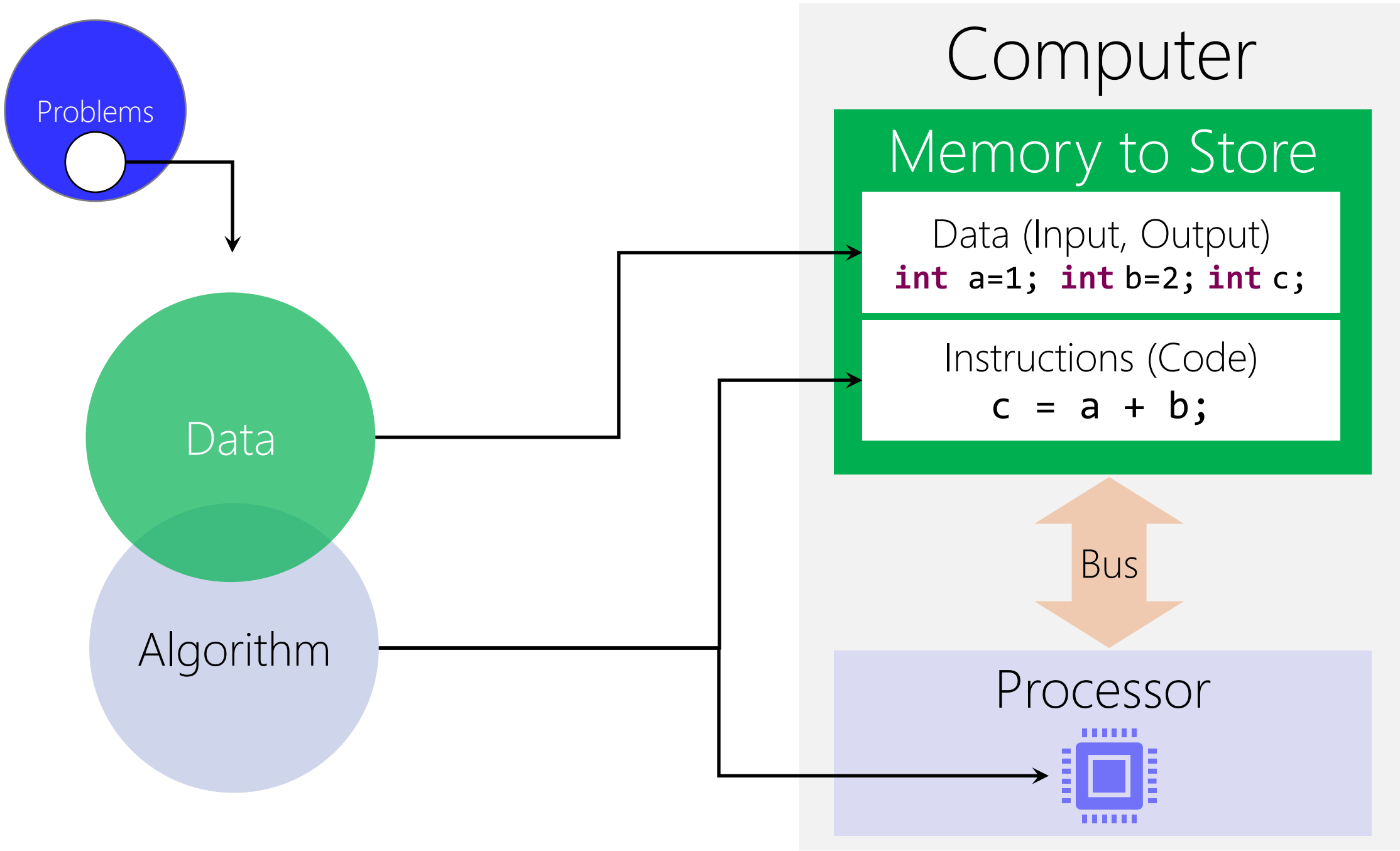
Mathematician, Physicist, Computer Scientist, Engineer

Polymath

He integrated pure and applied sciences. He made major contributions to many fields, including:

- Mathematics
- Physics
- Economics (game theory)
- Computing
- Statistics

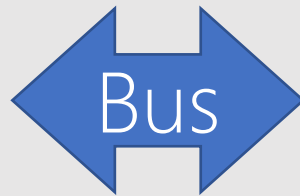




Computer System

Input/Output
Devices

```
scanf("%d", &a);  
scanf("%d", &b);  
printf("%d", c);
```



Computer

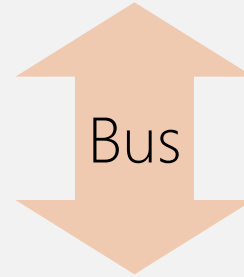
Memory to Store

Data (Input, Output)

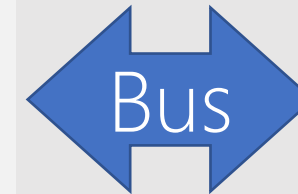
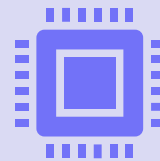
```
int a=1; int b=2; int c;
```

Instructions (Code)

```
c = a + b;
```

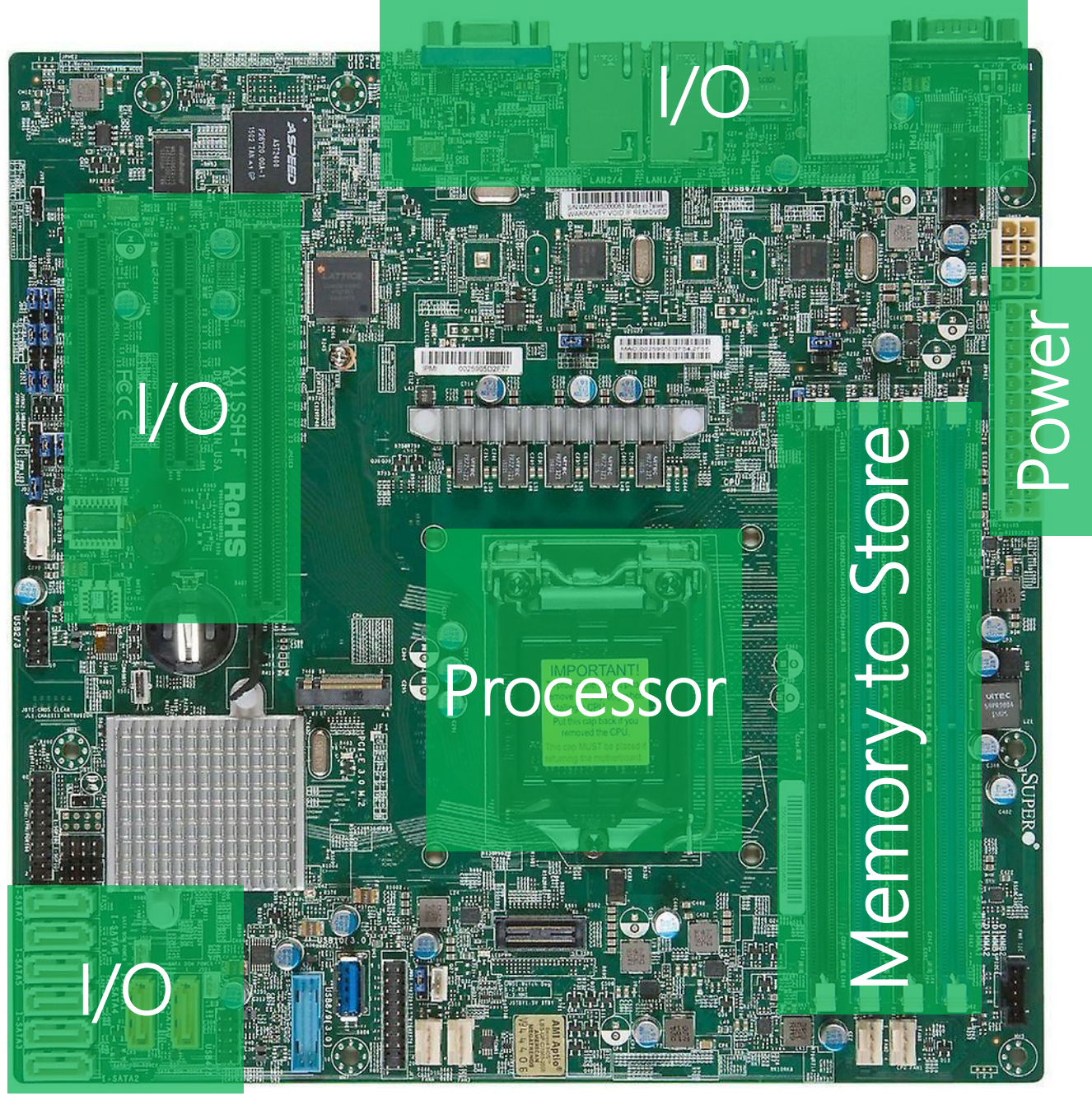


Processor



Permanent
Storage

```
fprintf()  
fscanf()  
fread()  
fwrite()  
fseek()
```



I/O

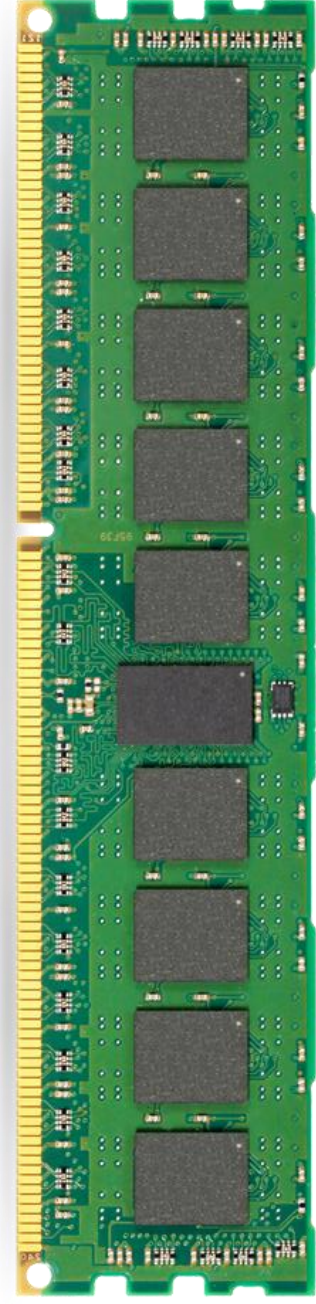
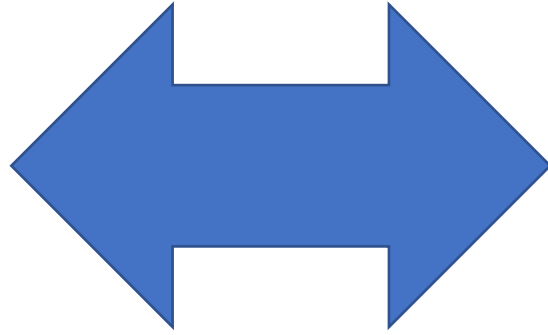
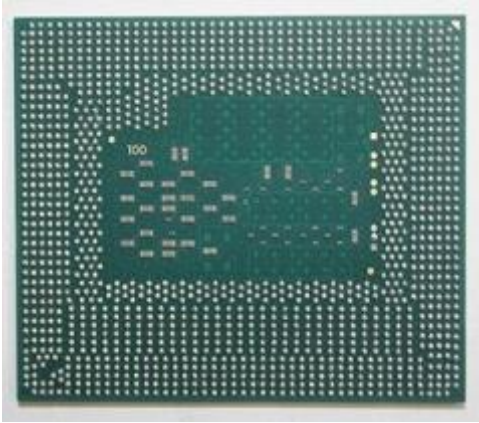
I/O

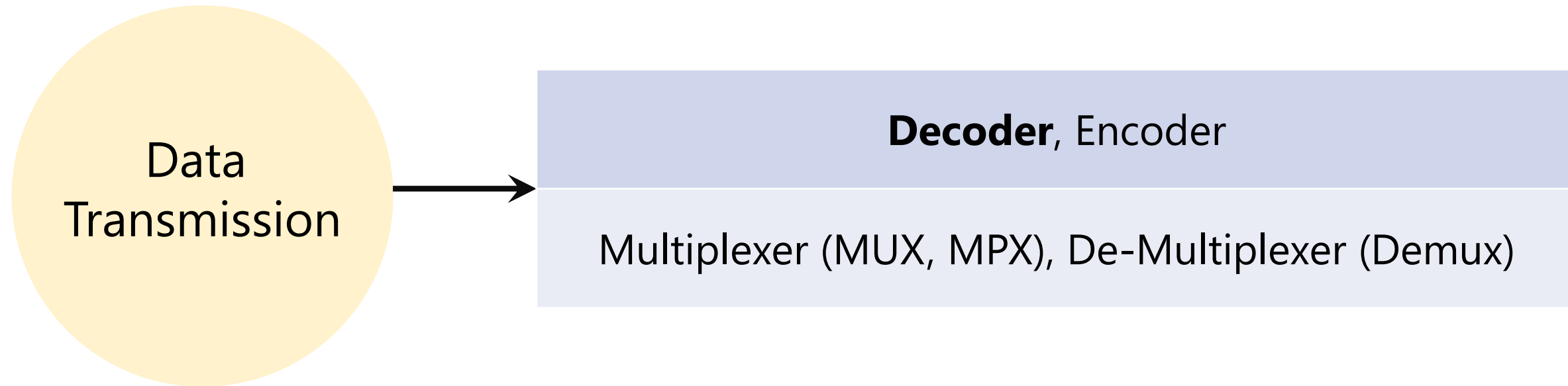
Processor

Memory to Store

Power

I/O





Decoder

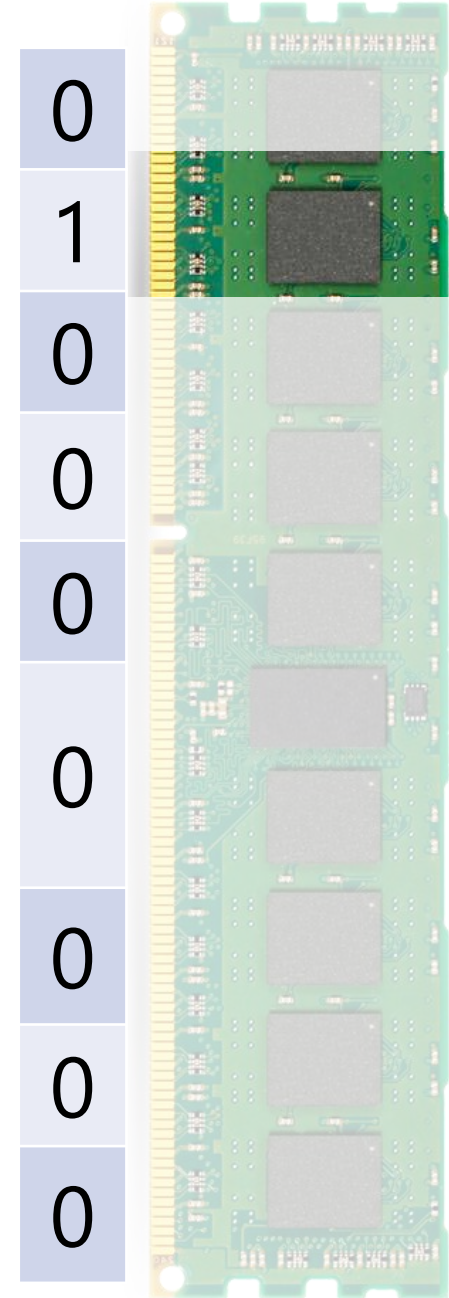
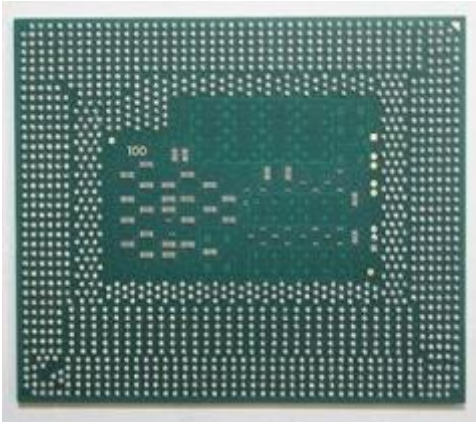
Encode Binary to 1-hot

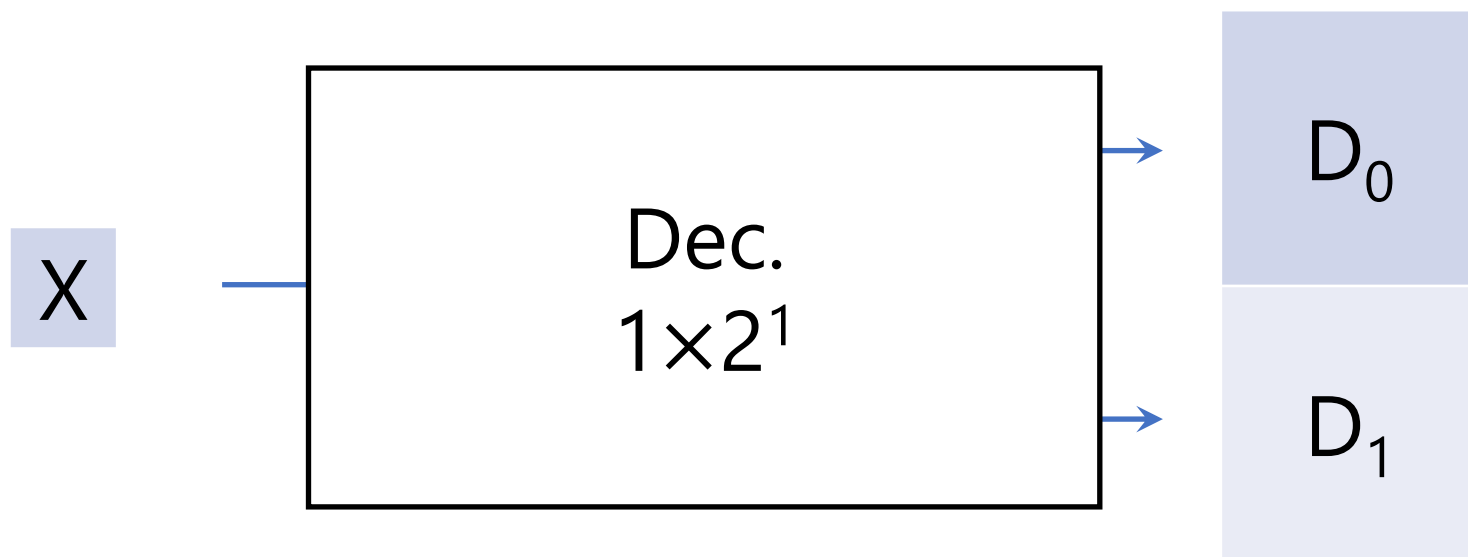
1-hot: a vector of bits with a single 1 and all the others 0

[0010000000]

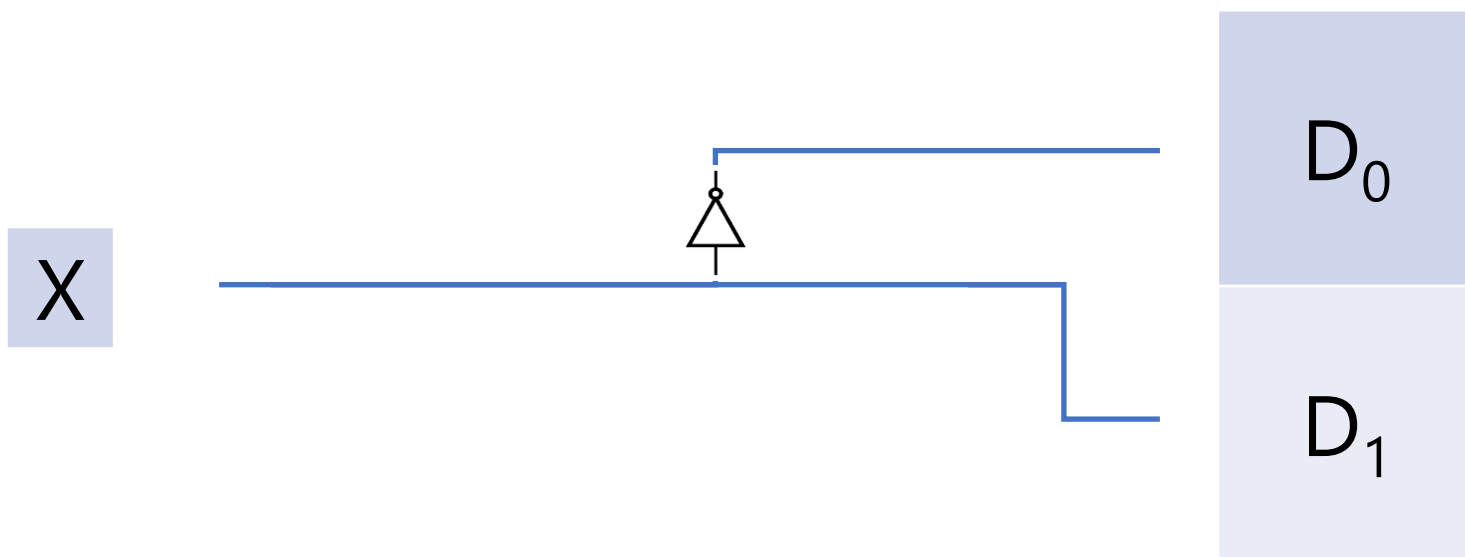
[0000000100]

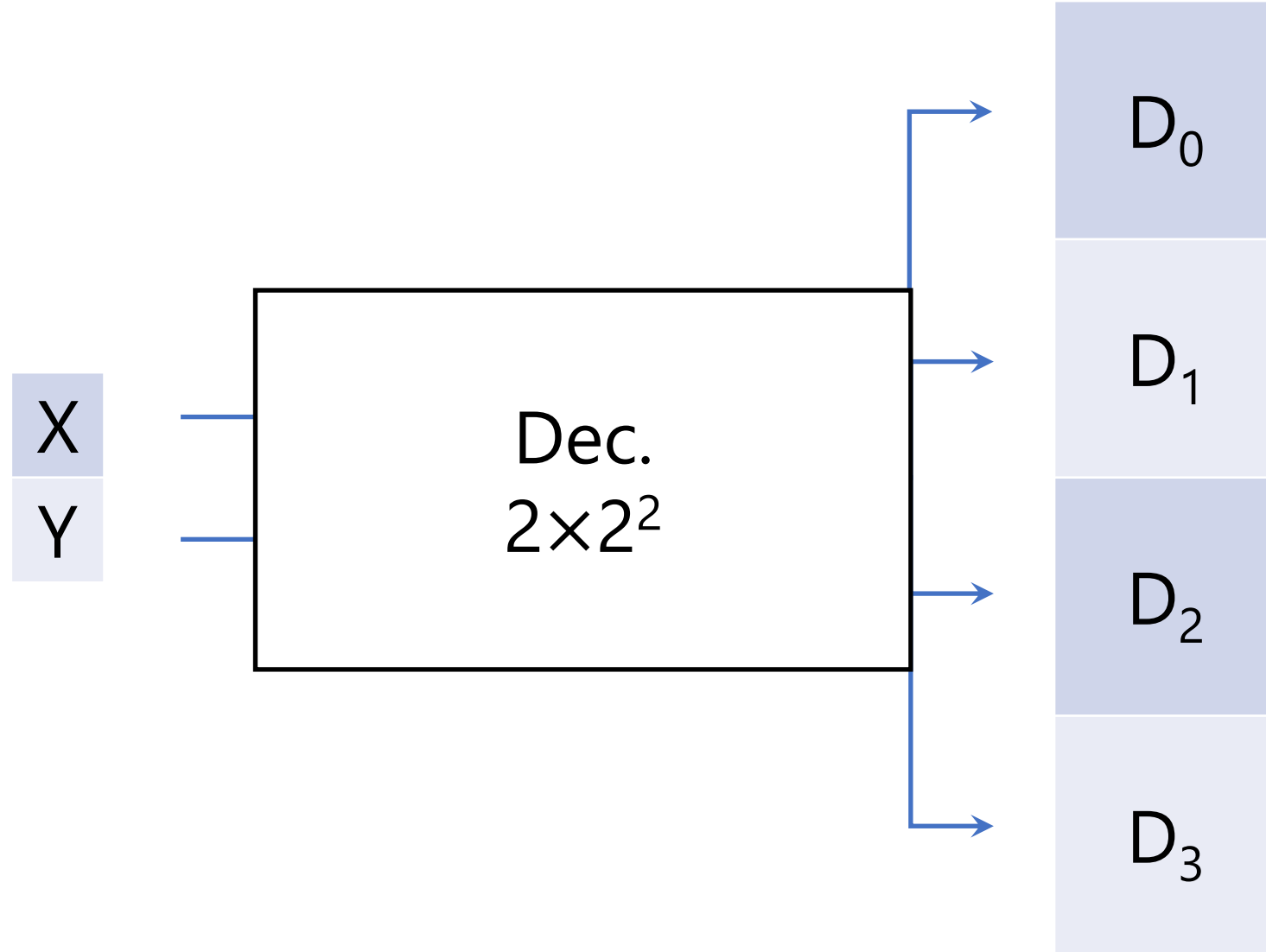
~~[0010010000]~~



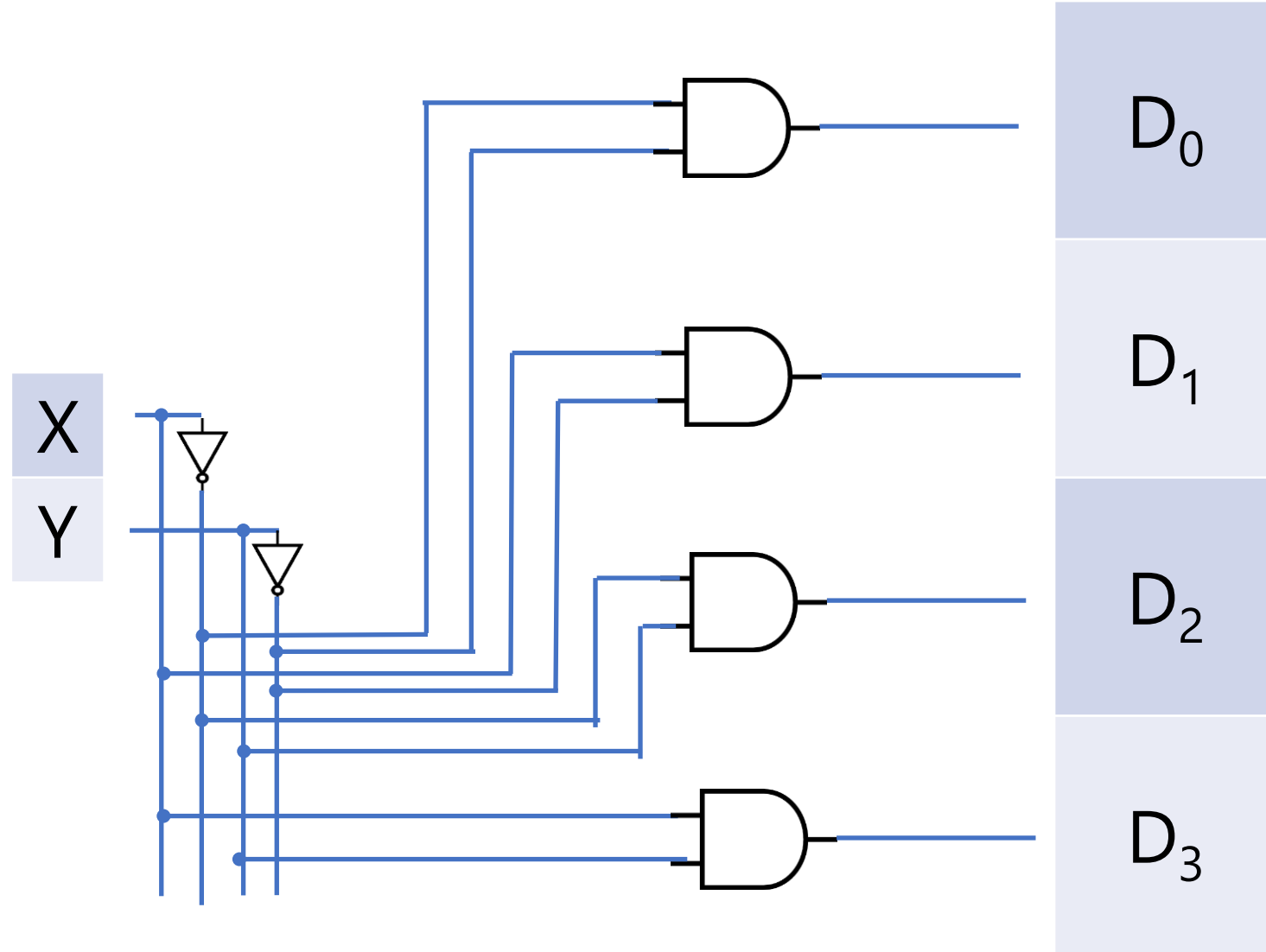


X	$D_0 = m_0$	$D_1 = m_1$
0	1	0
1	0	1





Y	X	$D_0 = m_0$	$D_1 = m_1$	$D_2 = m_2$	$D_3 = m_3$
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



Chapter 4 Combinational Logic

Table 4.6
Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
<i>x</i>	<i>y</i>	<i>z</i>	<i>D</i> ₀	<i>D</i> ₁	<i>D</i> ₂	<i>D</i> ₃	<i>D</i> ₄	<i>D</i> ₅	<i>D</i> ₆	<i>D</i> ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

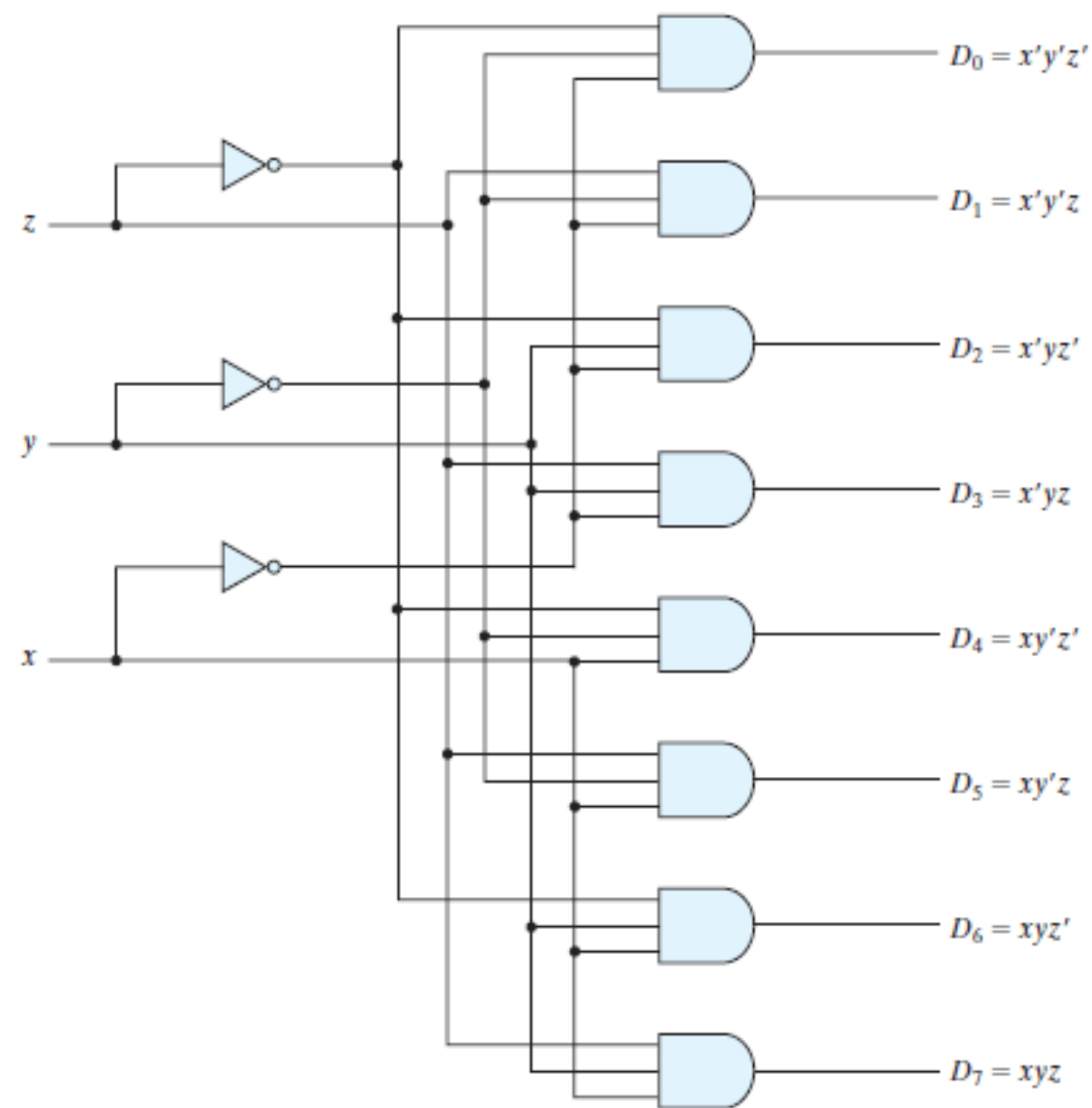


FIGURE 4.18
Three-to-eight-line decoder

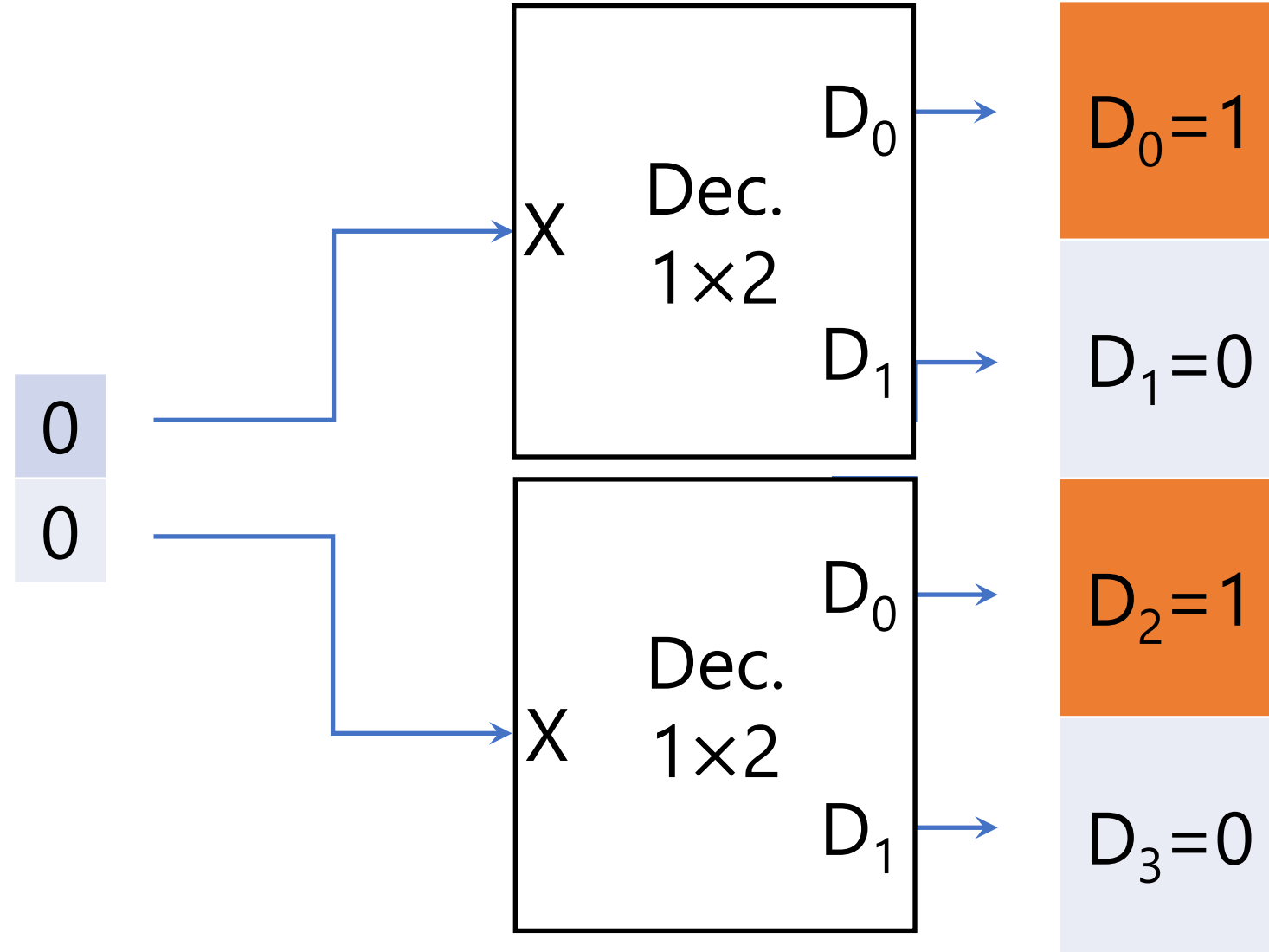
Decoder

Encode n -Bit Binary to 2^n One-hot

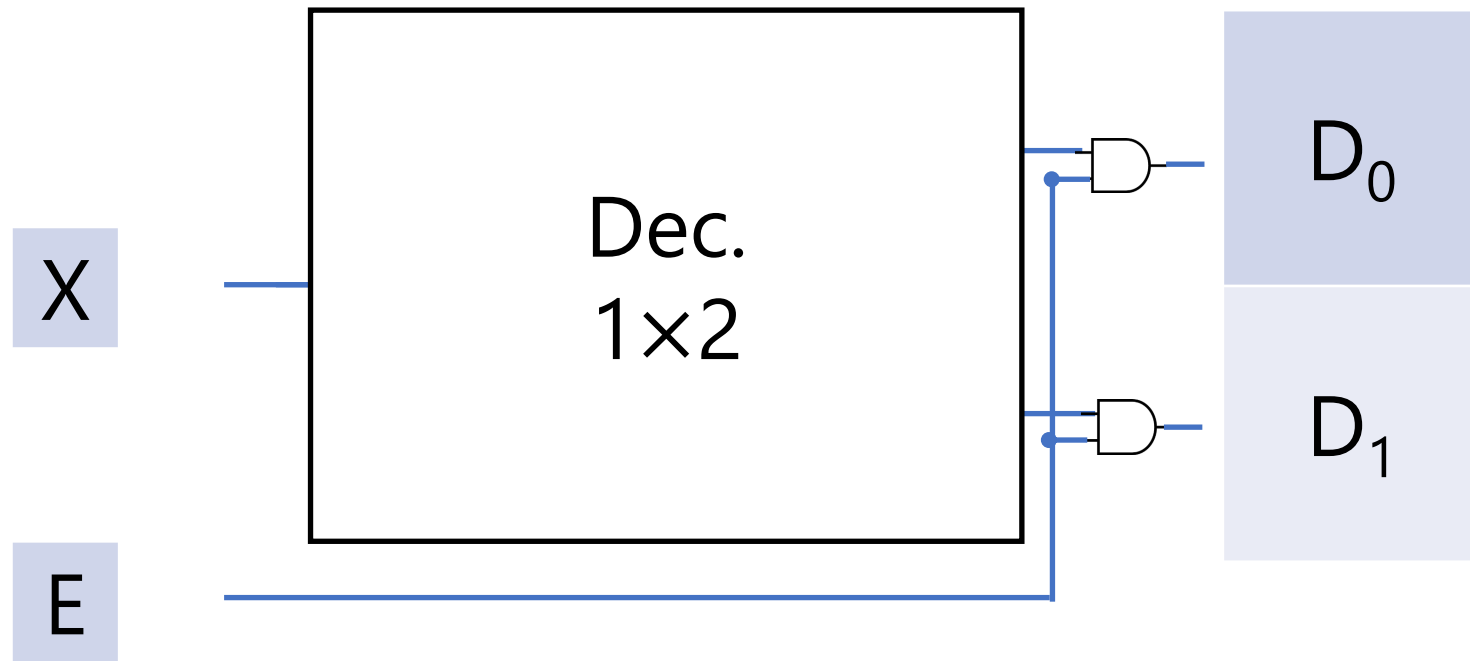
Decoder

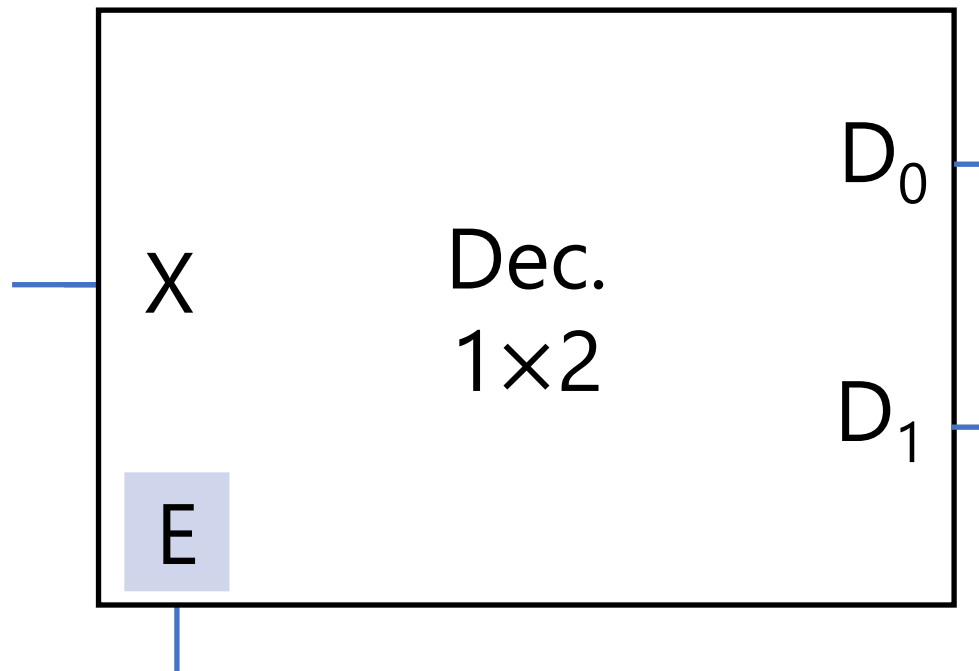
Encode 2-Bit Binary to 2^2 One-hot

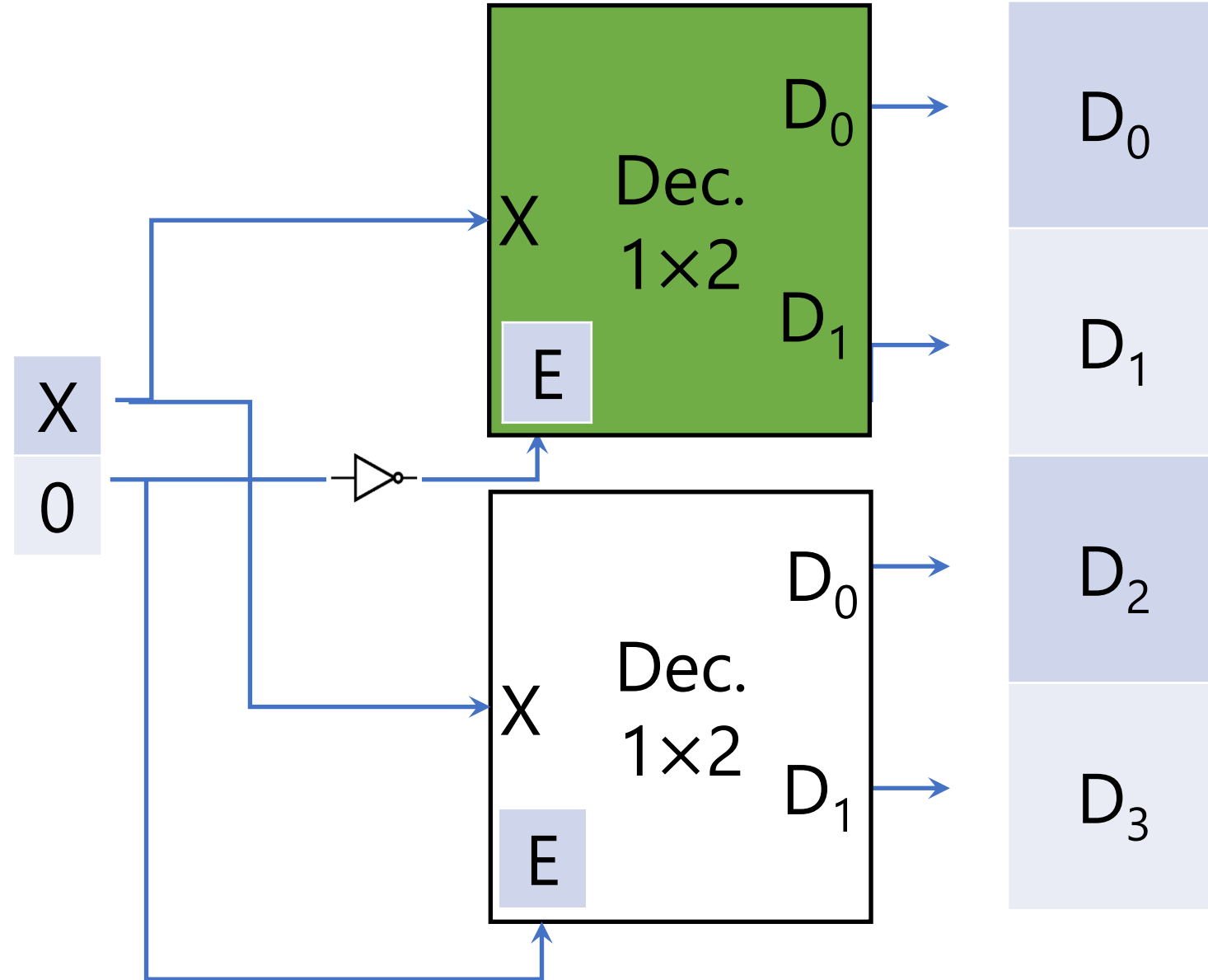
Re-Use 1×2^1 Decoder

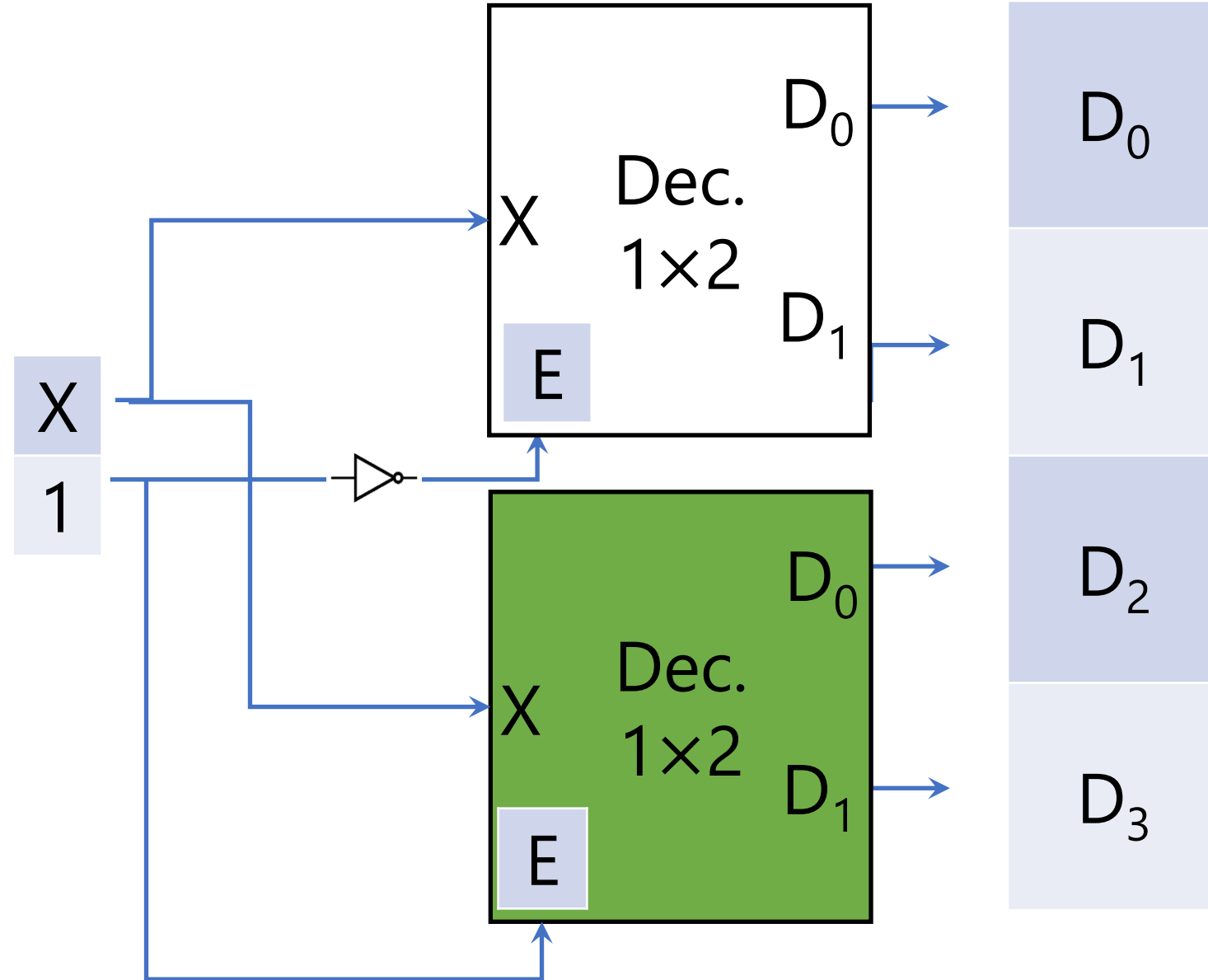


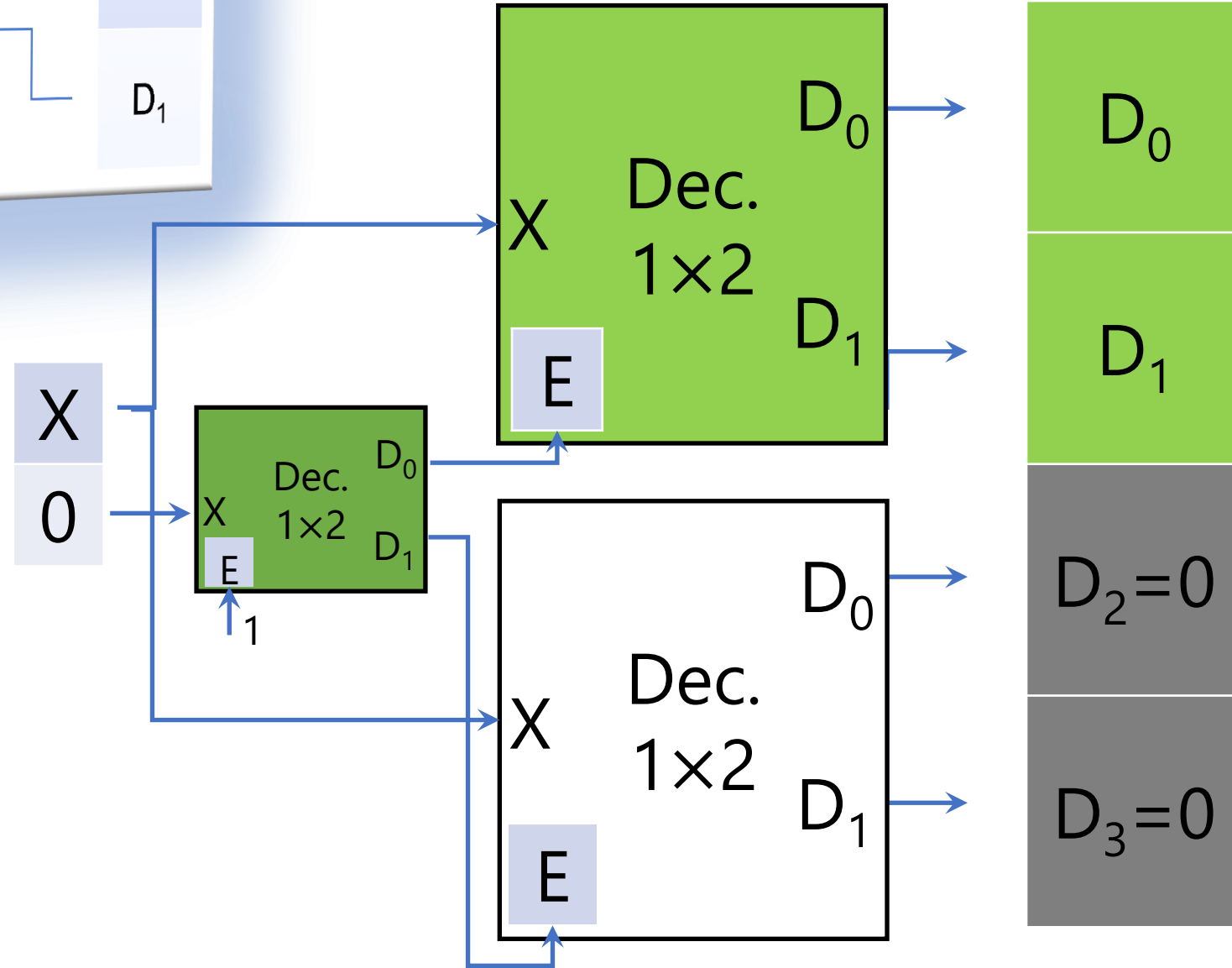
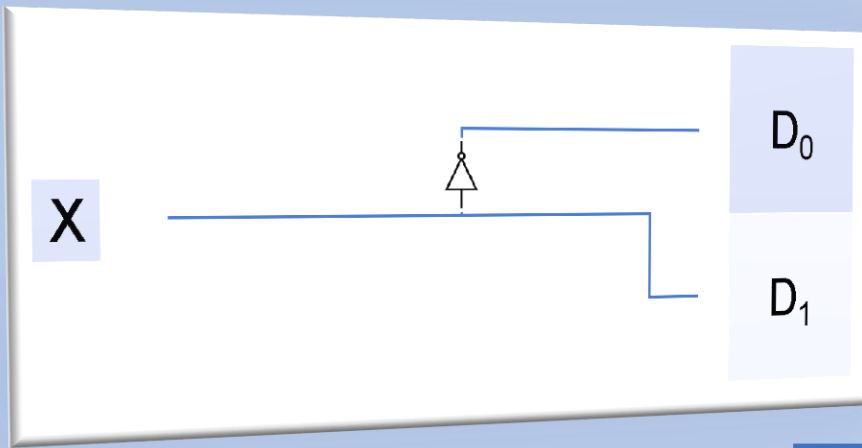
Decoder
Enable input

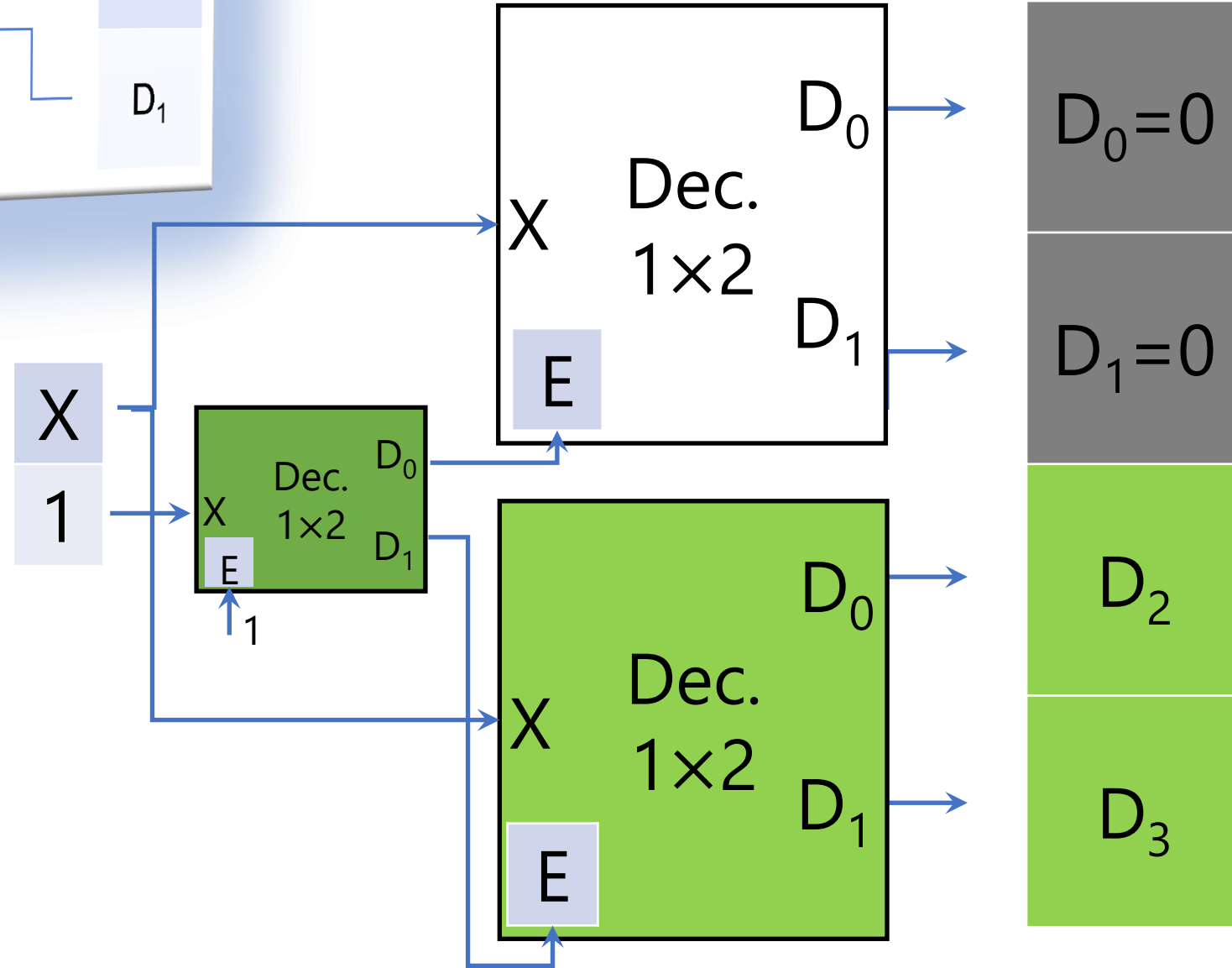
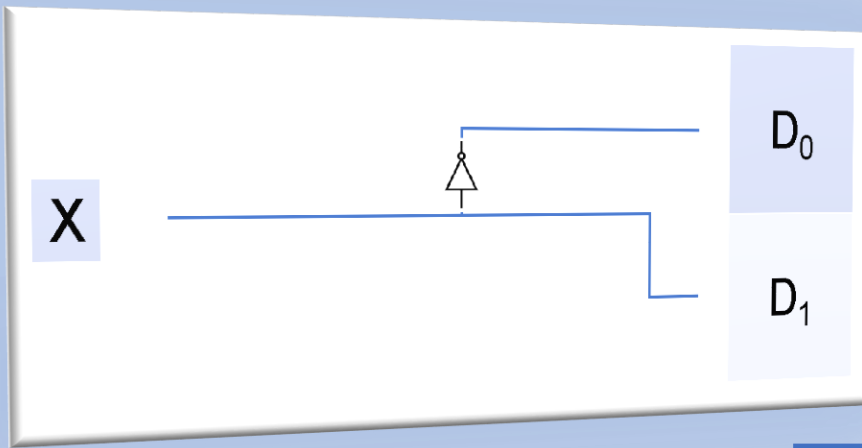








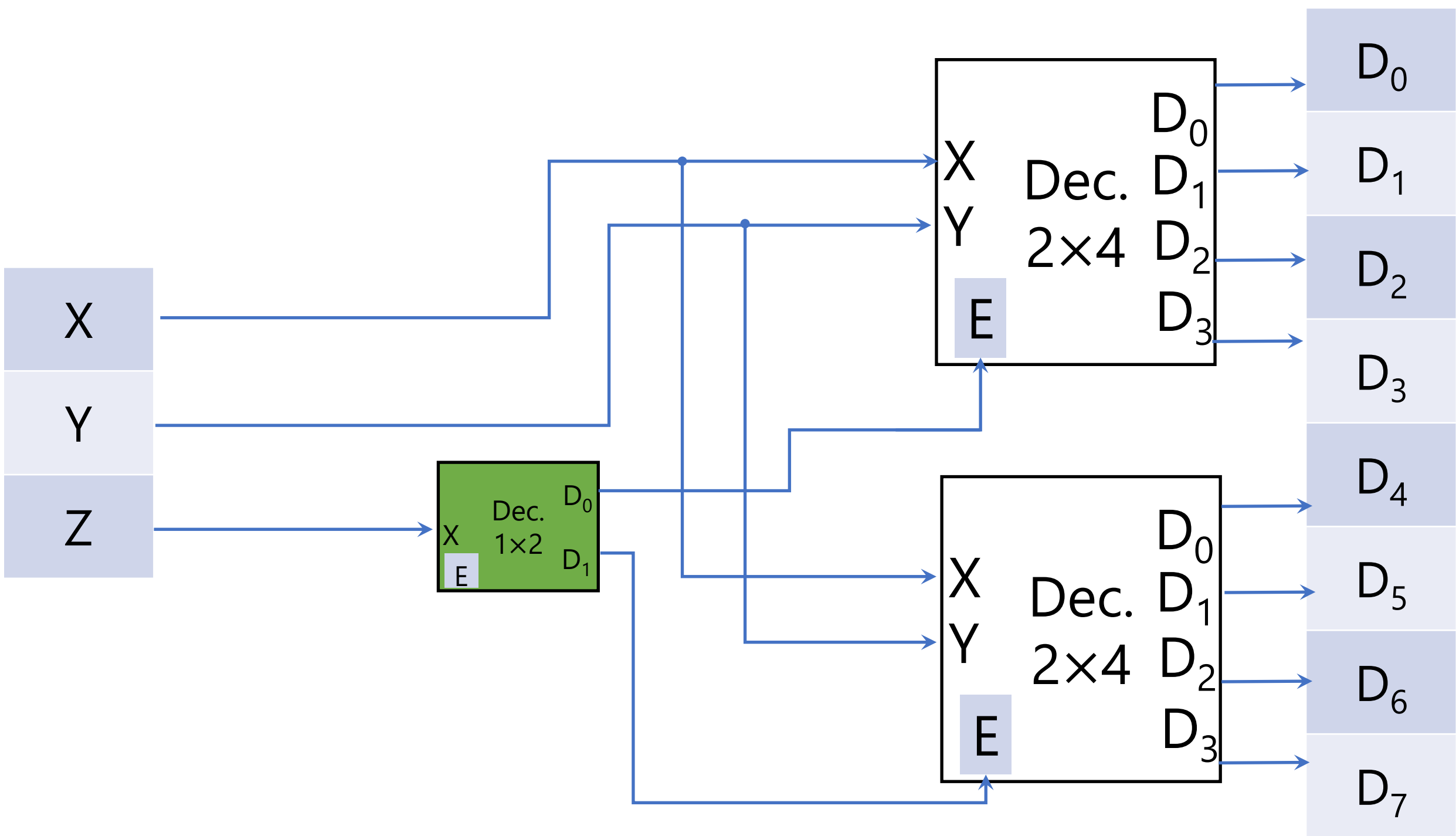


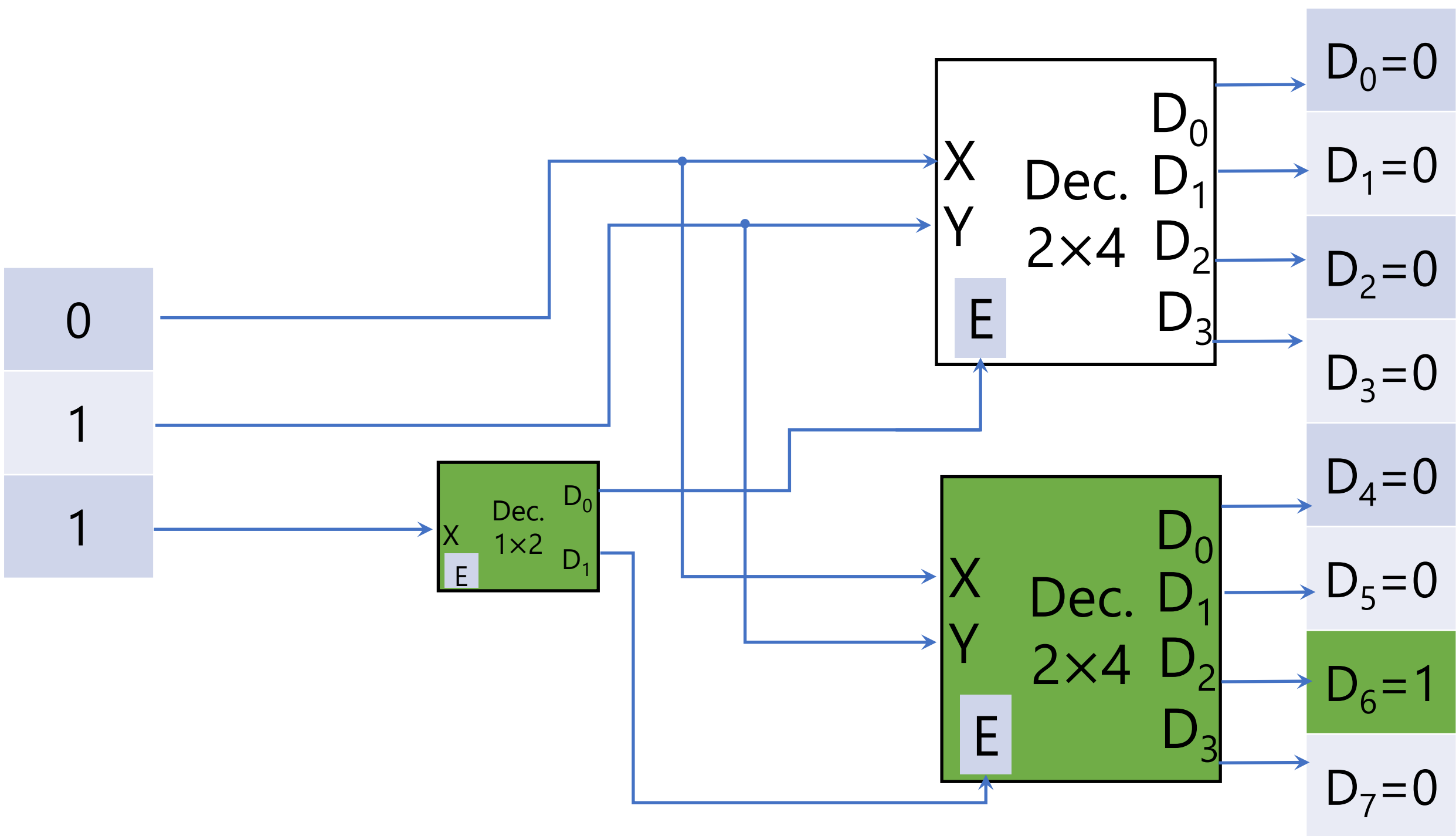


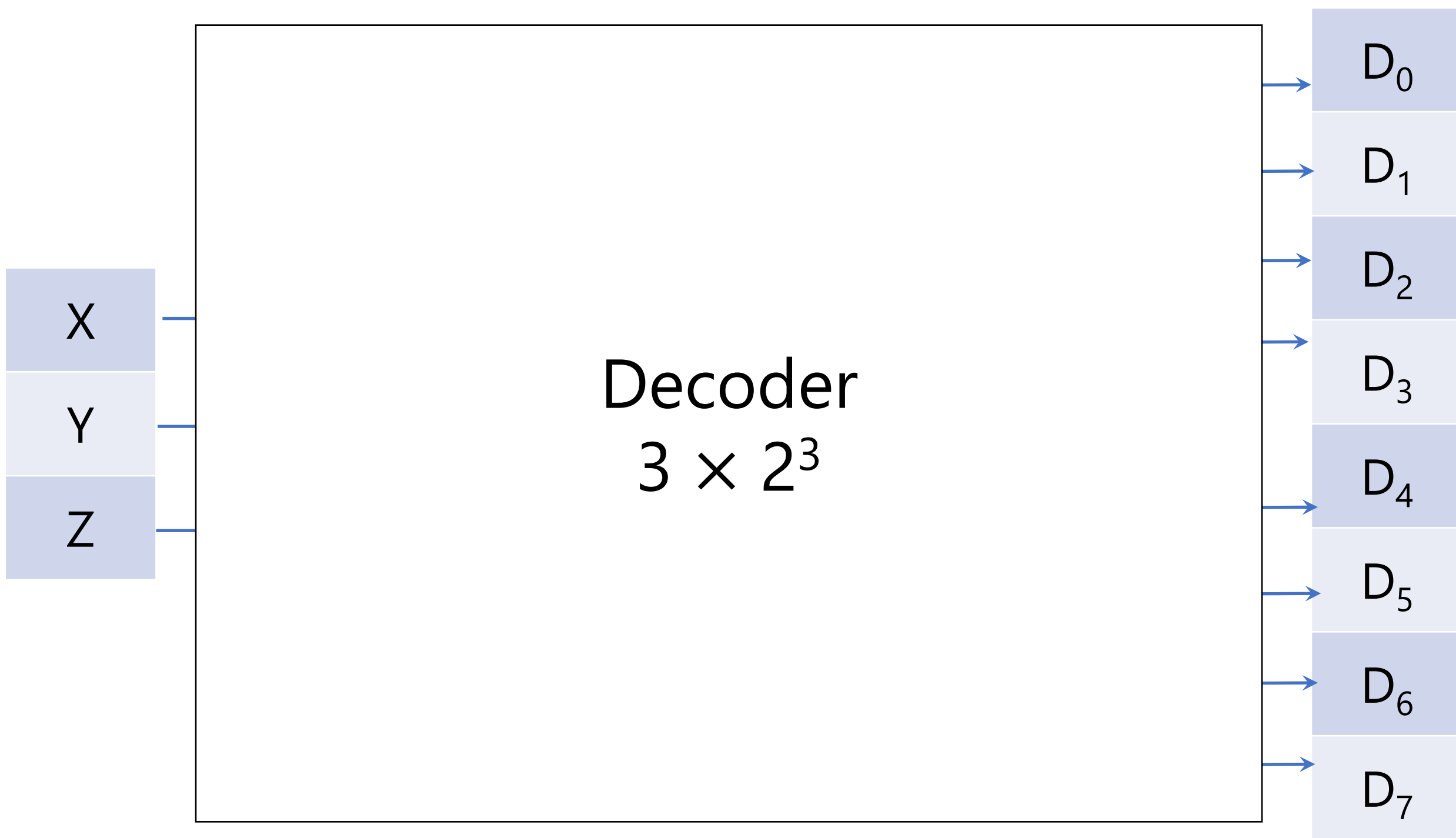
Decoder

Decode 3-Bit Binary to 2^3 One-hot

Re-Use 2×2^2 Decoder







Decoder

Decode 4-Bit Binary to 2^4 One-hot

Re-Use 1×2^1 Decoder

Re-Use 2×2^2 Decoder

Re-Use 3×2^3 Decoder



All Products ▾



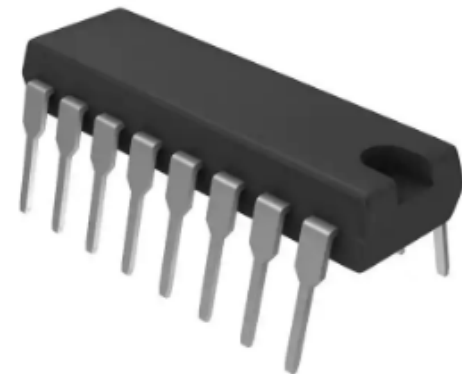
 Login or REGISTER

 0 ITEM(S)

Products Manufacturers Resources Tools

Product Index > Integrated Circuits (ICs) > Logic - Signal Switches, Multiplexers, Decoders > Texas Instruments SN74LS138N

Share 



SN74LS138N

[Datasheet](#)

Digi-Key Part Number	296-1639-5-ND
Manufacturer	Texas Instruments
Manufacturer Product Number	SN74LS138N
Supplier	Texas Instruments
Description	IC 3-8 LINE DECODER/DEMUX 16-DIP

Manufacturer Standard Lead Time 6 Weeks

Decoder/Demultiplexer 1 x 3:8
16-PDIP

[Customer Reference](#)

Price and Procurement

4,043 In Stock
Can ship immediately

QUANTITY

Quantity

Add to Cart

Add to BOM

Add to Favorites

Tube

QTY	UNIT PRICE	EXT PRICE
1	\$1.27000	\$1.27
10	\$1.12000	\$11.20
25	\$1.05280	\$26.32
100	\$0.85920	\$85.92

Media & Downloads

RESOURCE TYPE	LINK
Datasheets	SN54LS138, SN54S138, SN74LS138, SN74S138A
Featured Product	Logic Solutions Analog Solutions
PCN Design/Specification	Material Set 30/Mar/2017
EDA / CAD Models 	SN74LS138N by SnapEDA SN74LS138N by Ultra Librarian

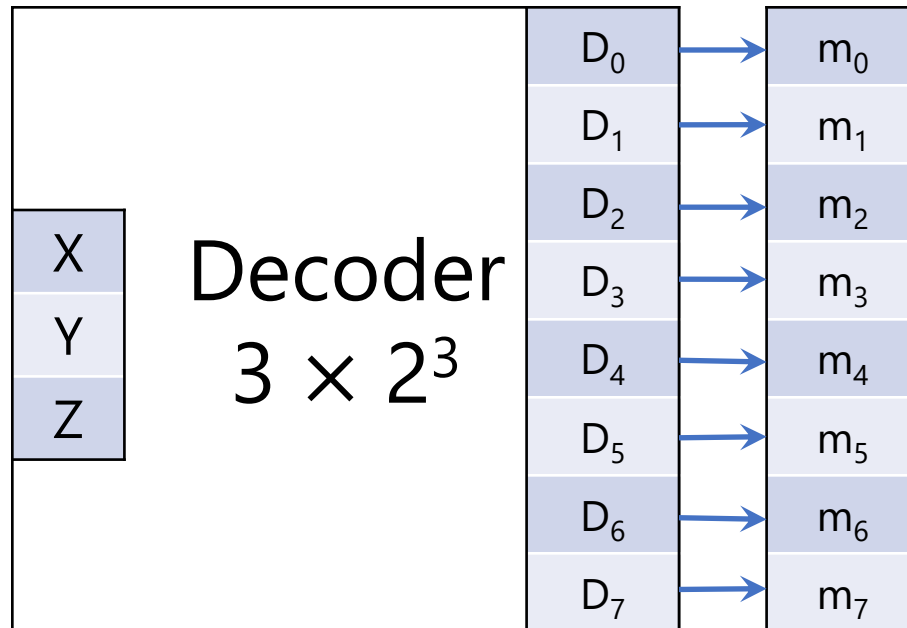
Decoder

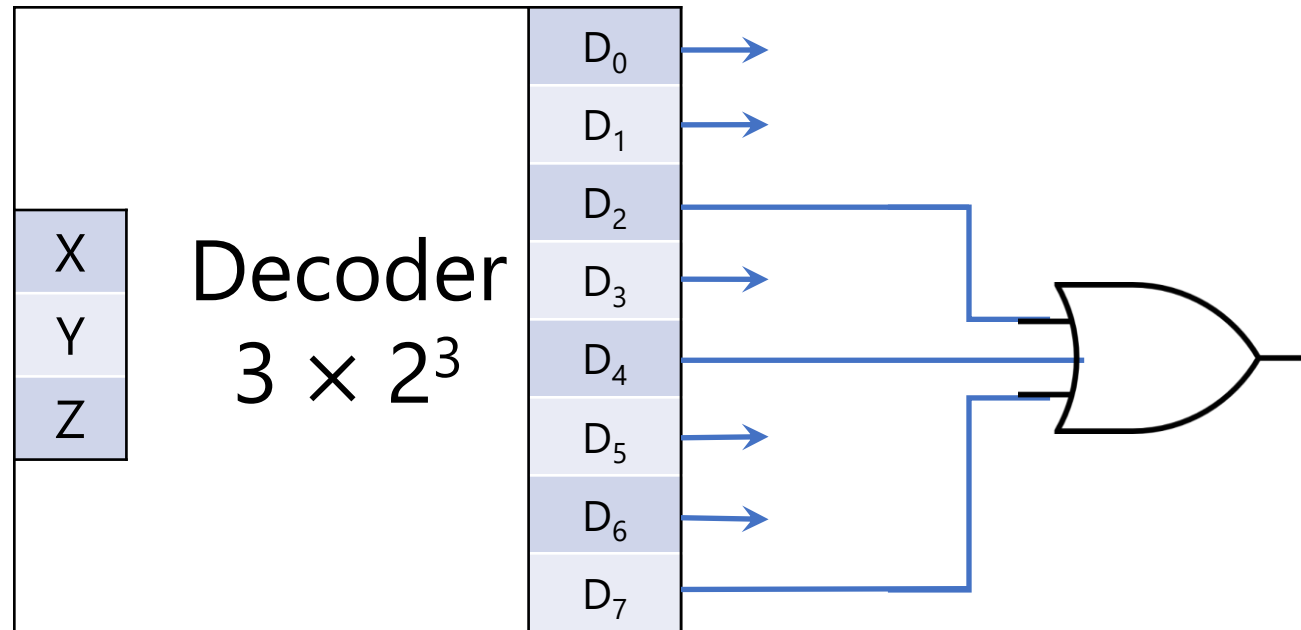
Boolean Function

(Off the road to our computer system)

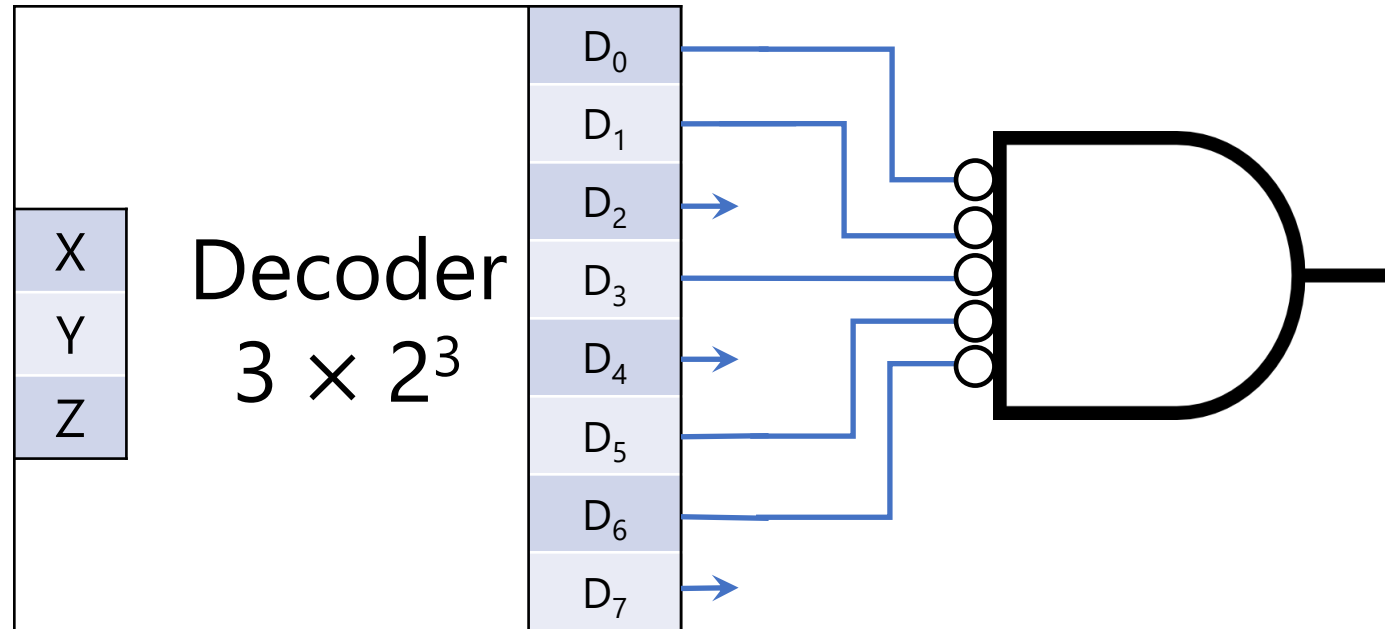
$$F_{\text{SoP}} = \sum m(\dots)$$

$$F_{\text{PoS}} = \prod M(\dots)$$





$$F_{\text{SoP}} = \sum m(2,4,7)$$



$$F_{\text{PoS}} = \prod M(0,1,3,5,6)$$

Decoder

Full Adder

$$S = \sum m(1, 2, 4, 7)$$

$$C = \sum m(3, 5, 6, 7)$$

C_p	Y	X	$C = \sum m(3,5,6,7)$	$S = \sum m(1,2,4,7)$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

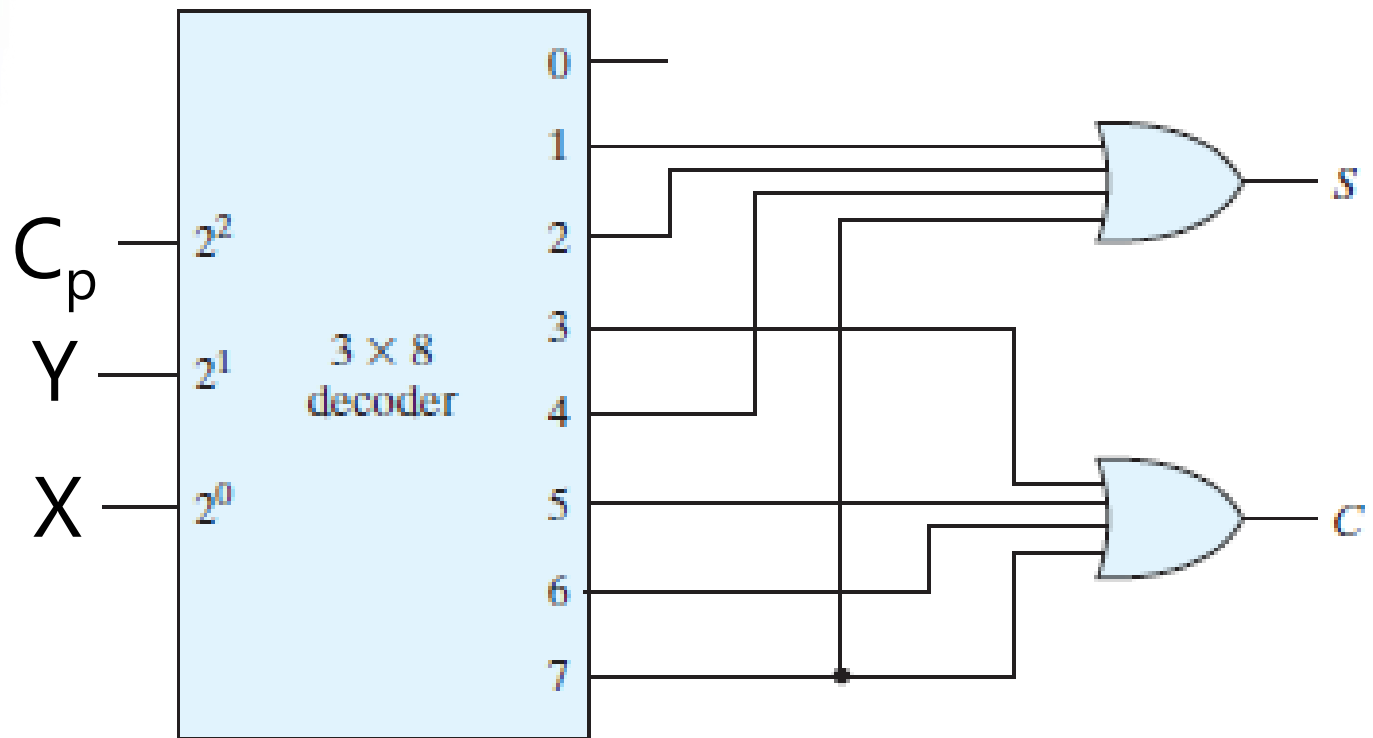
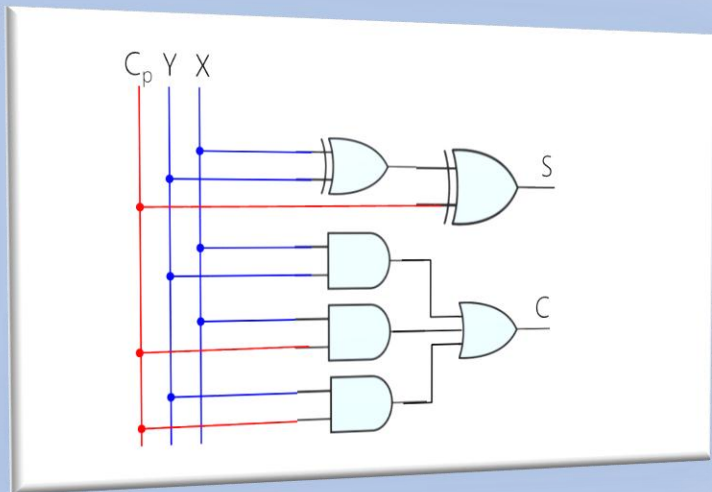
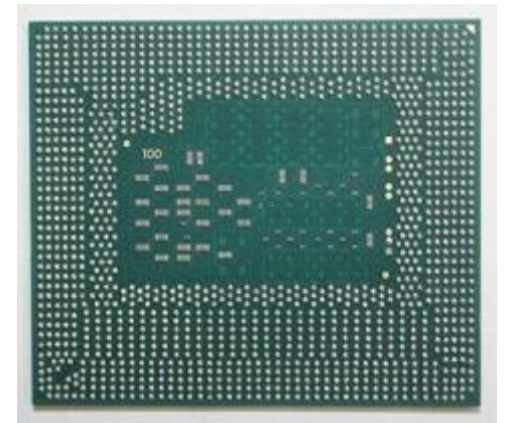


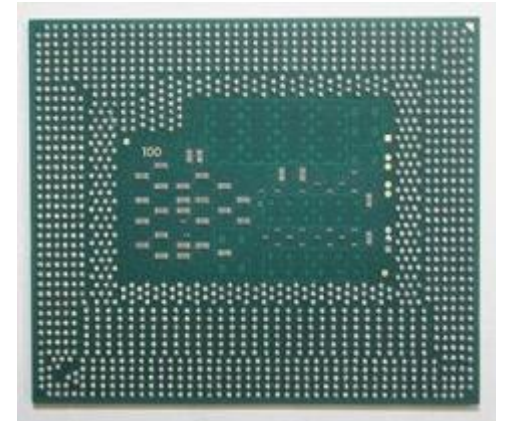
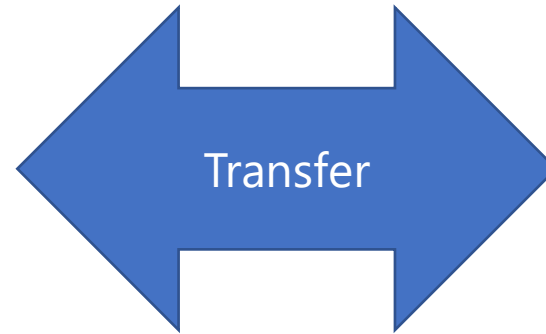
FIGURE 4.21
Implementation of a full adder with a decoder

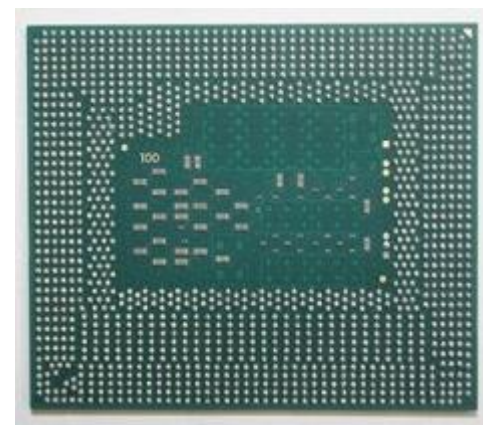
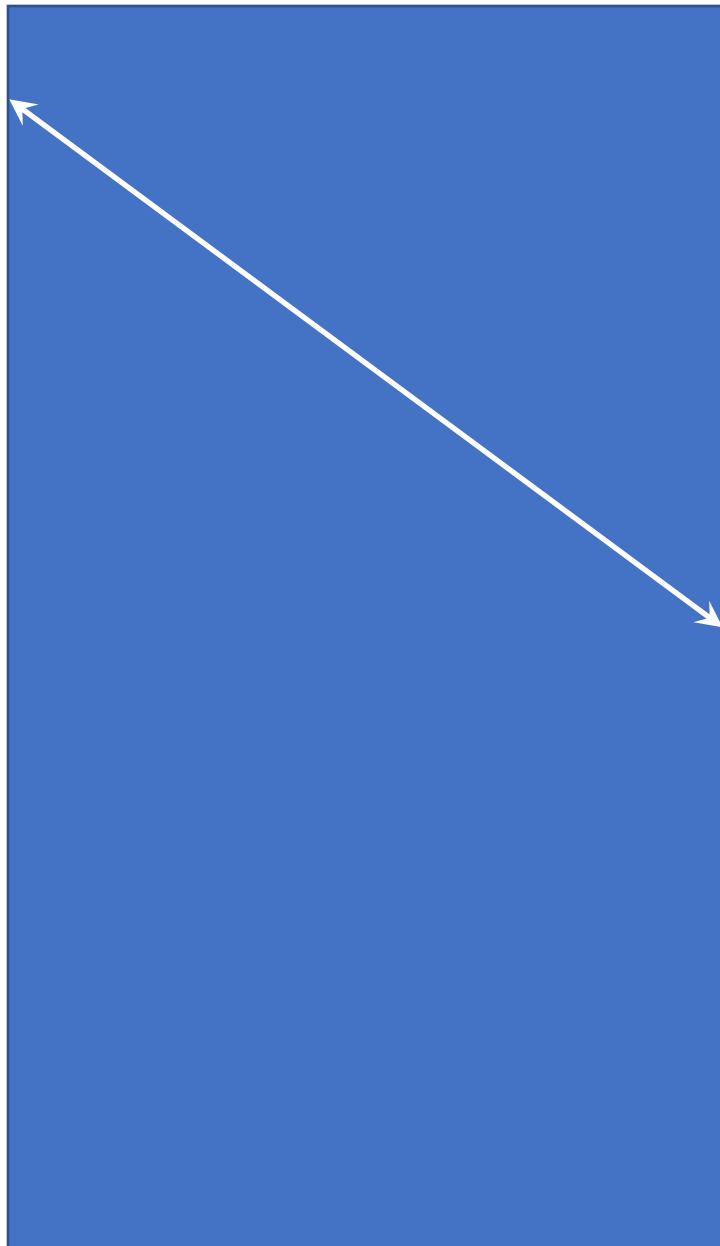
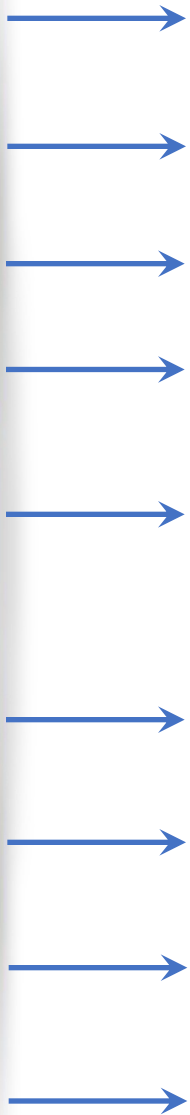


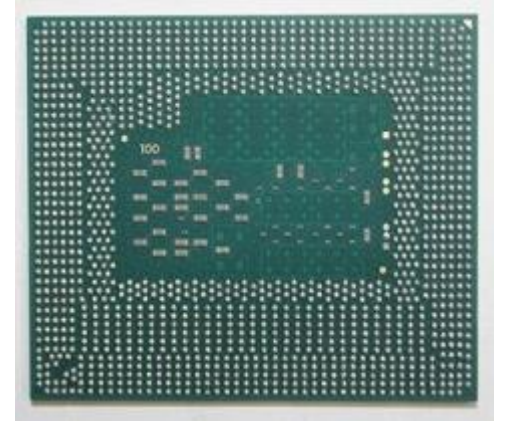
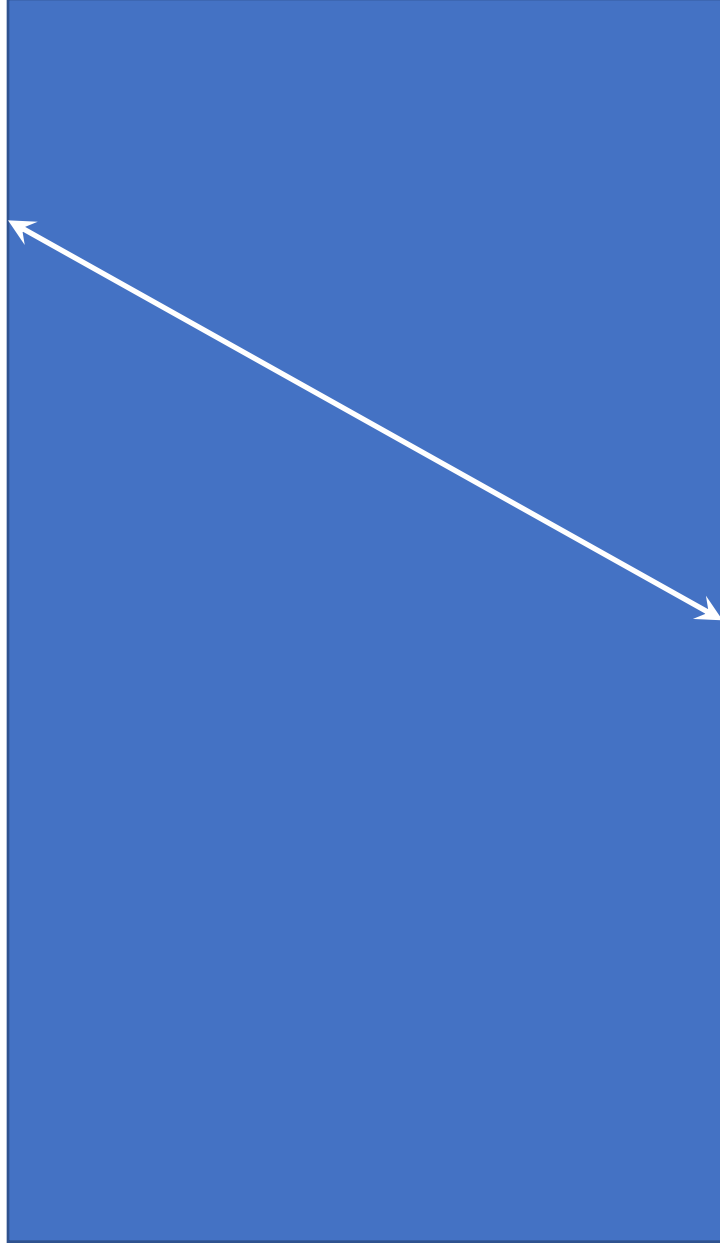
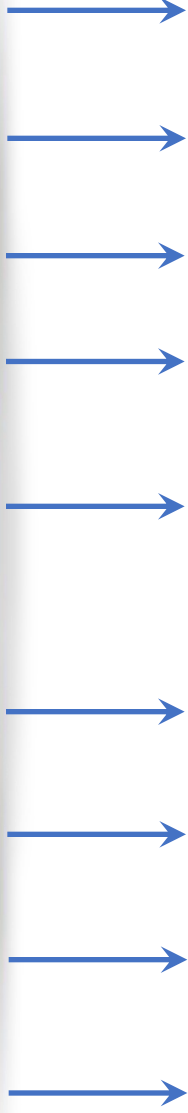
Multiplexer

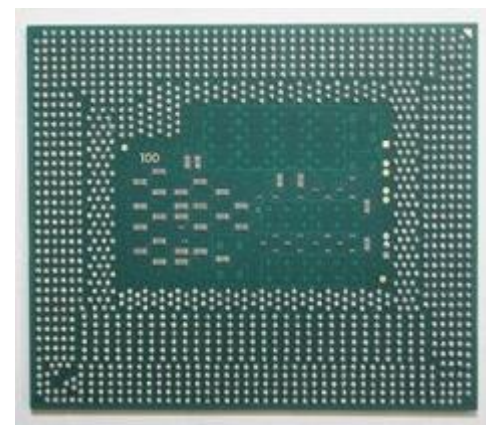
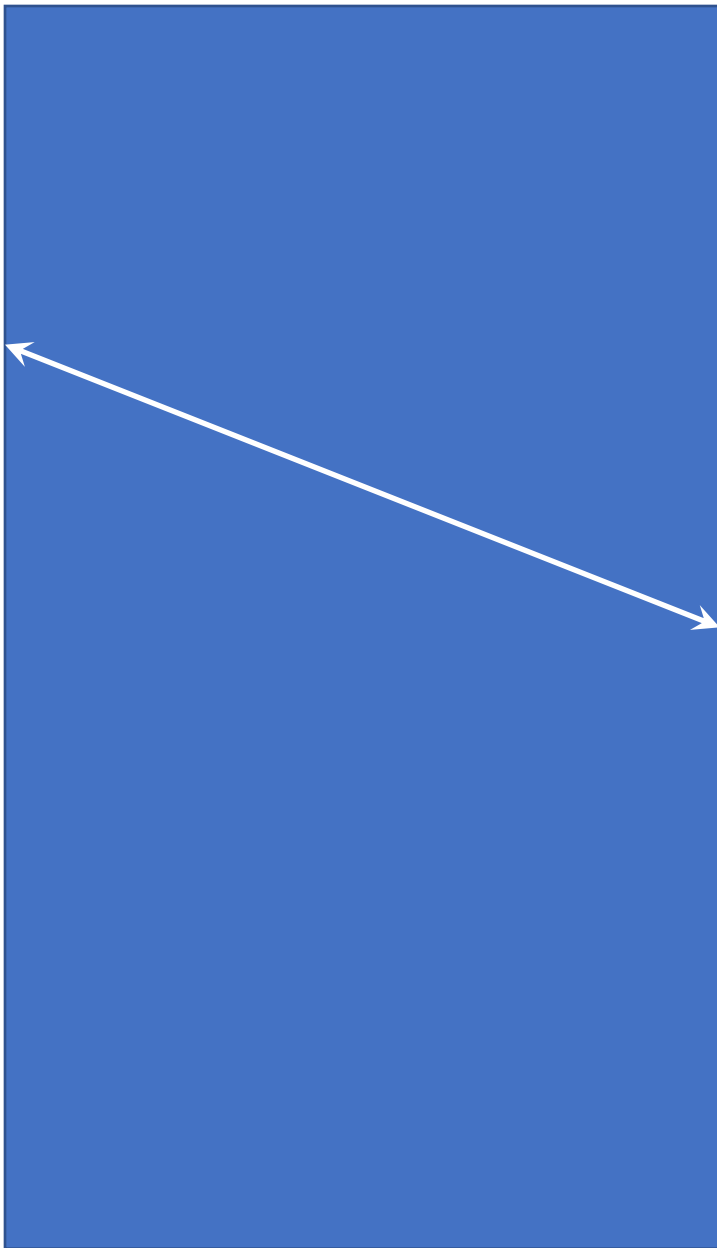
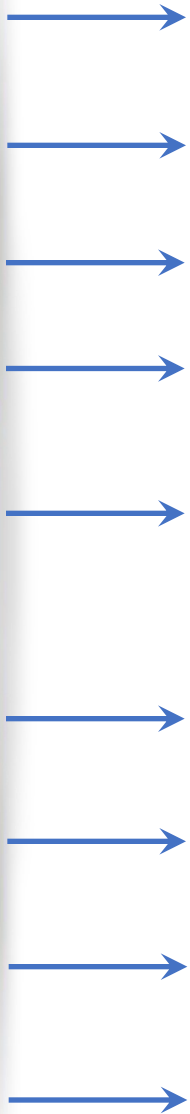
Shortened to MUX or MPX

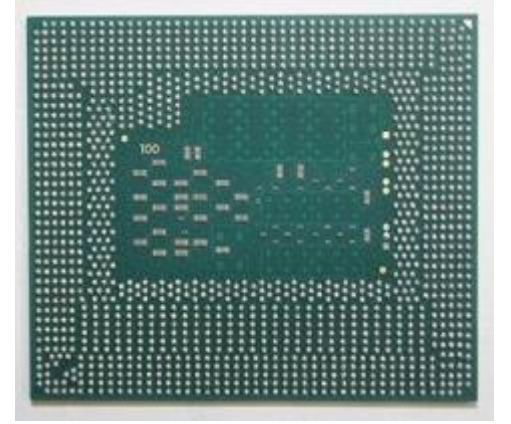
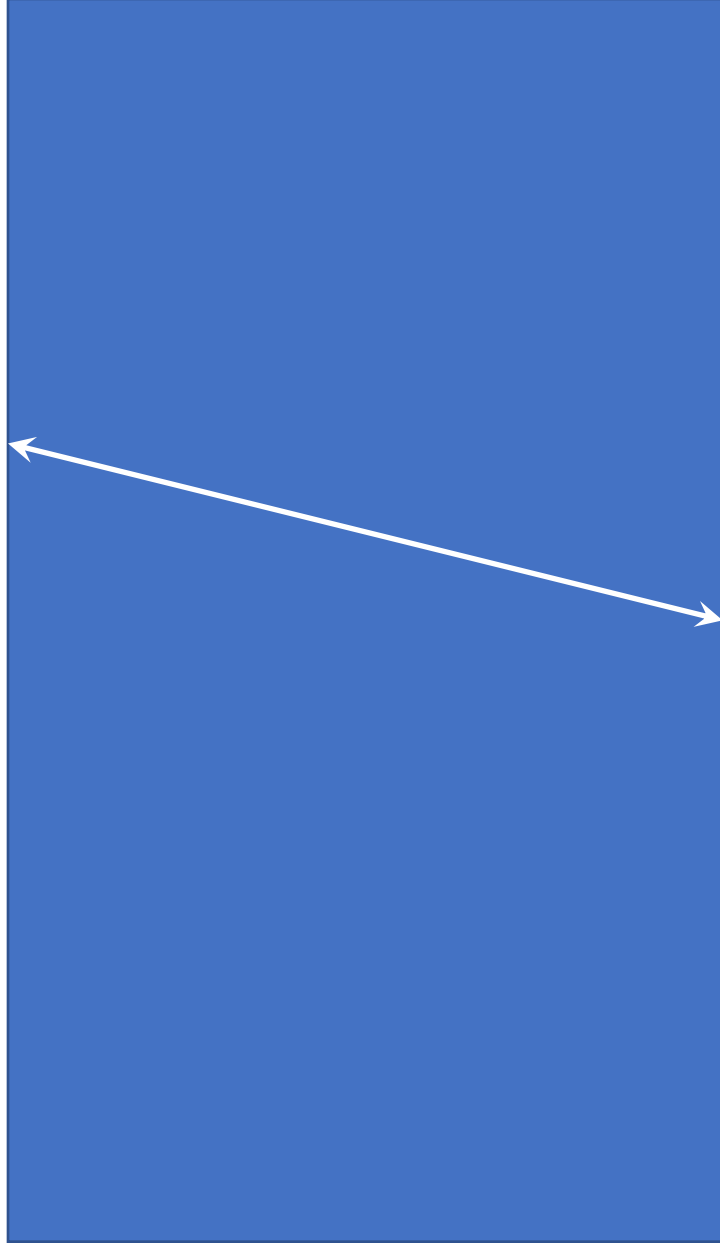
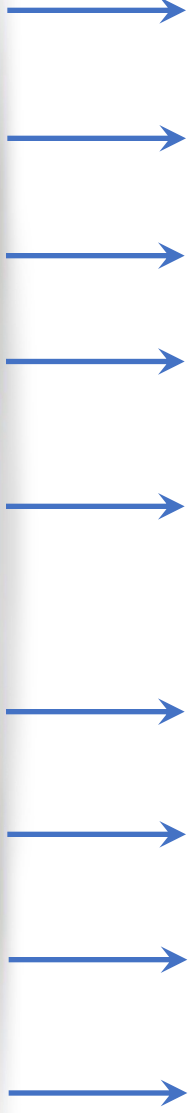


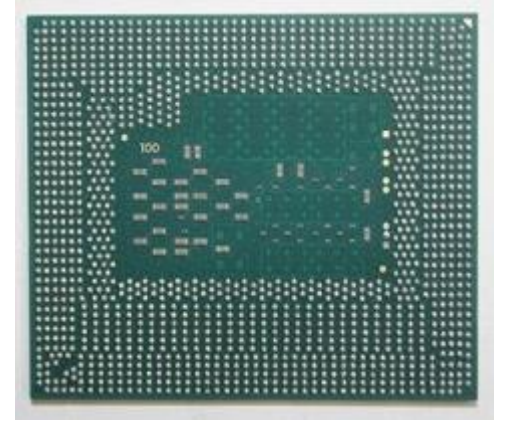
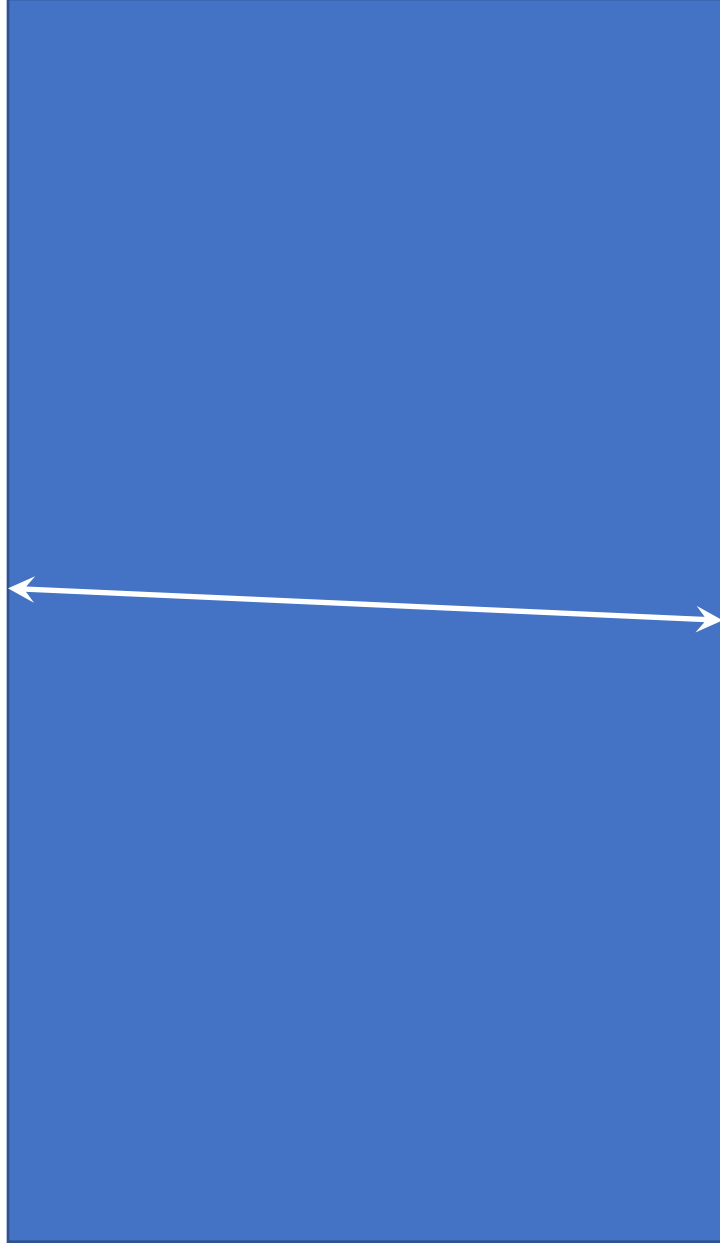
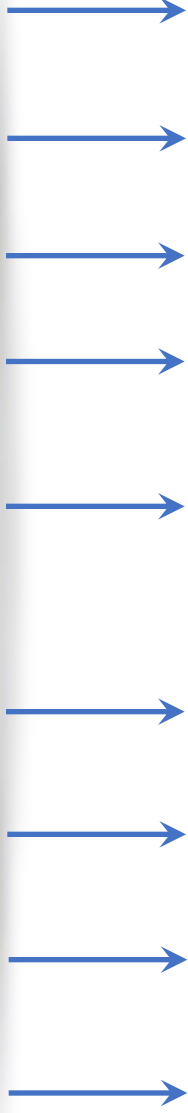


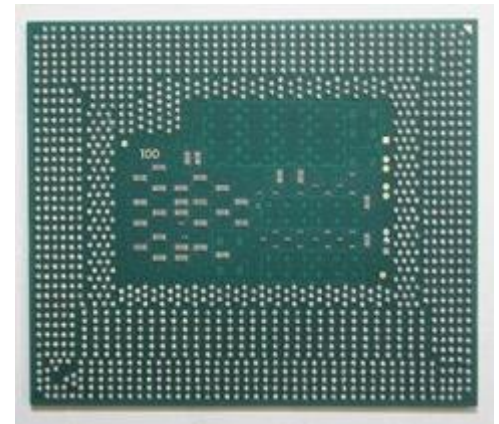
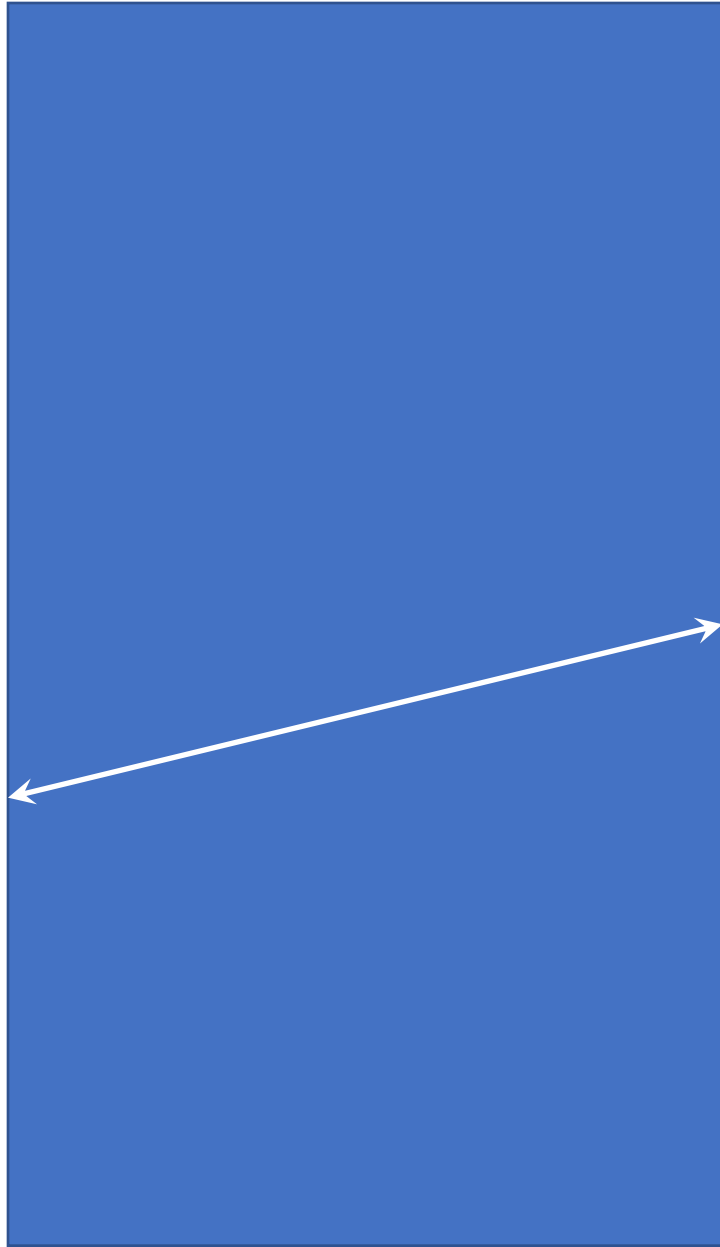
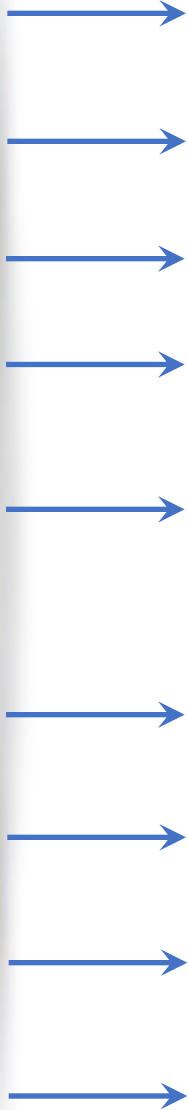


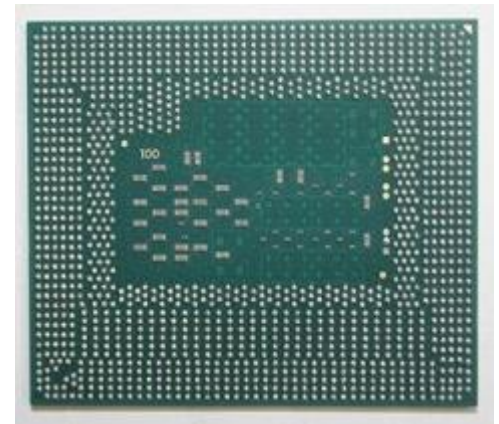
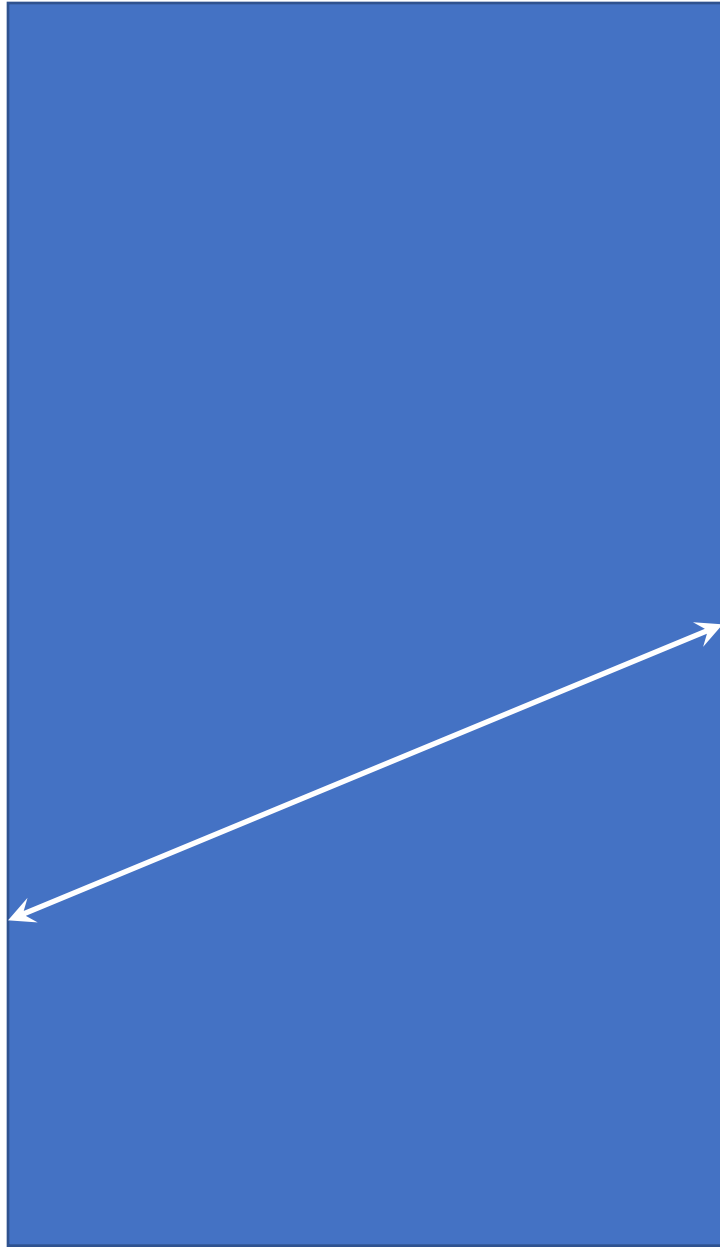
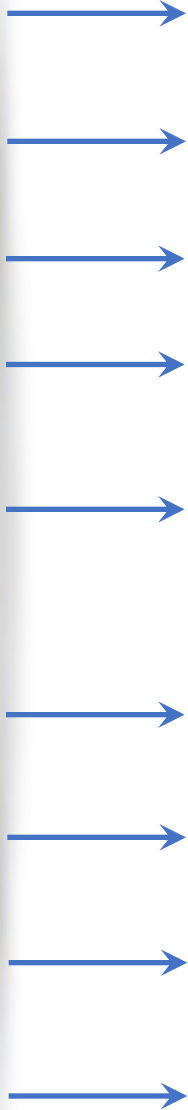


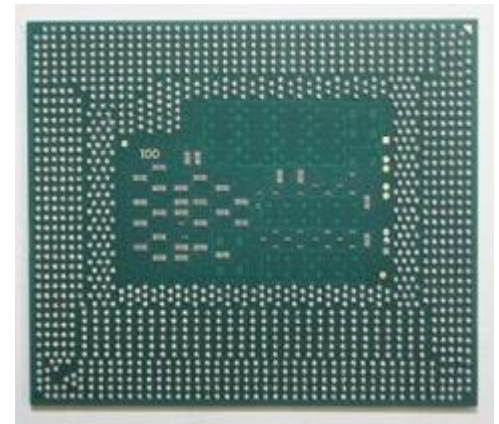
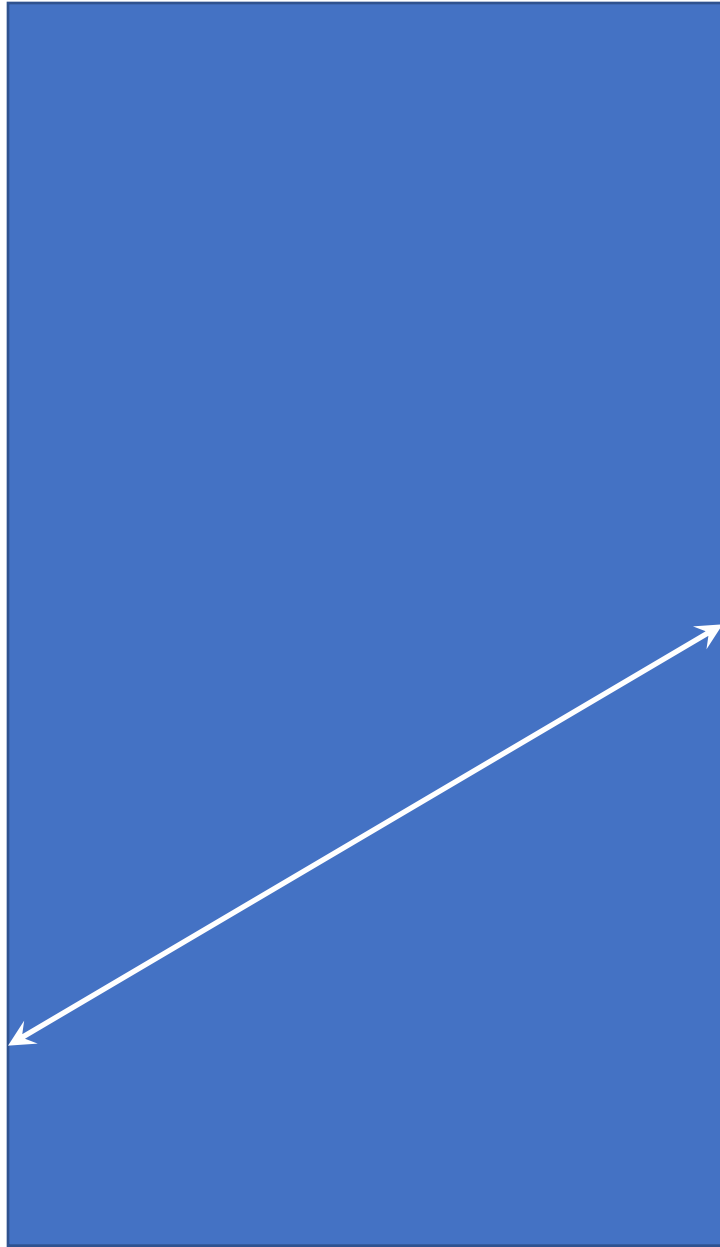
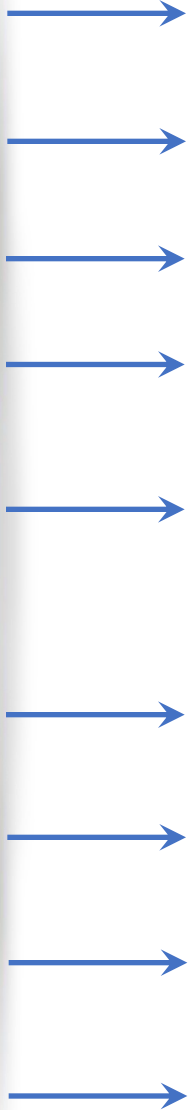


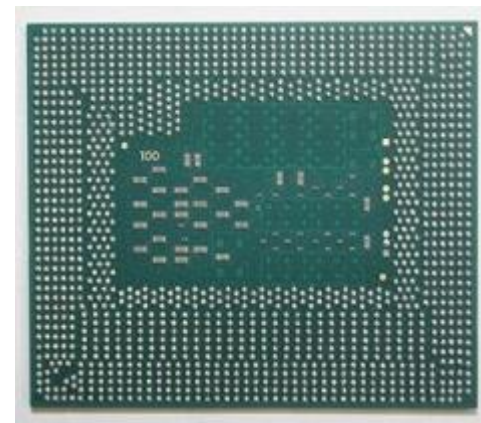
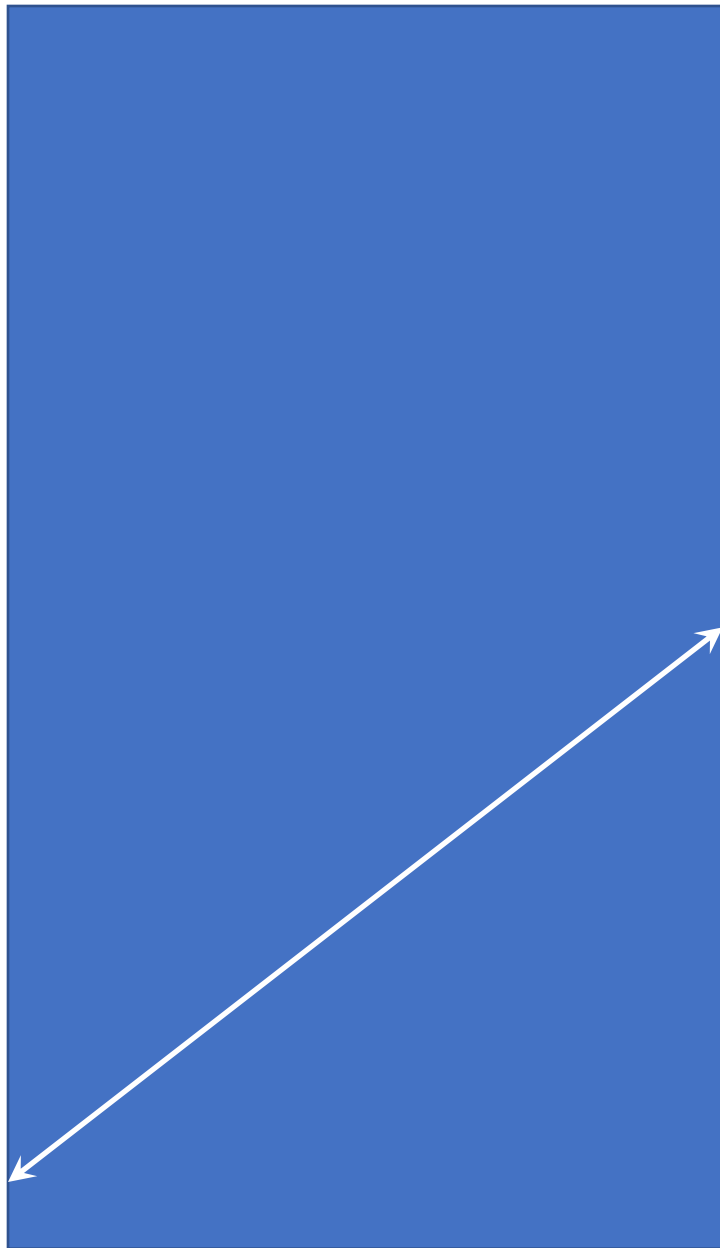
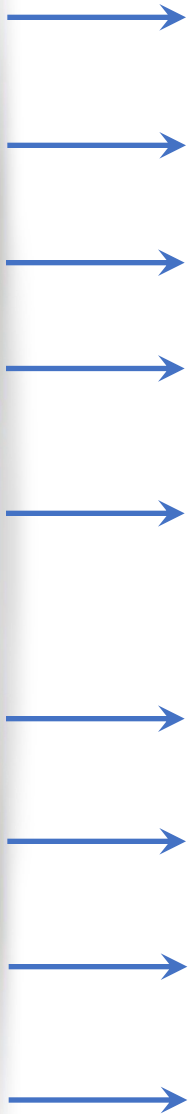






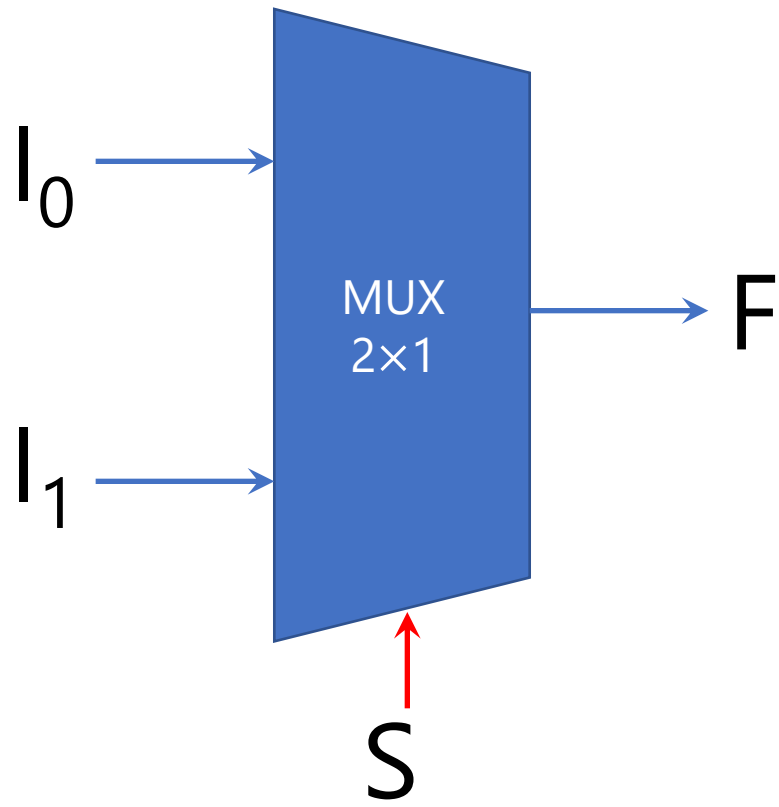


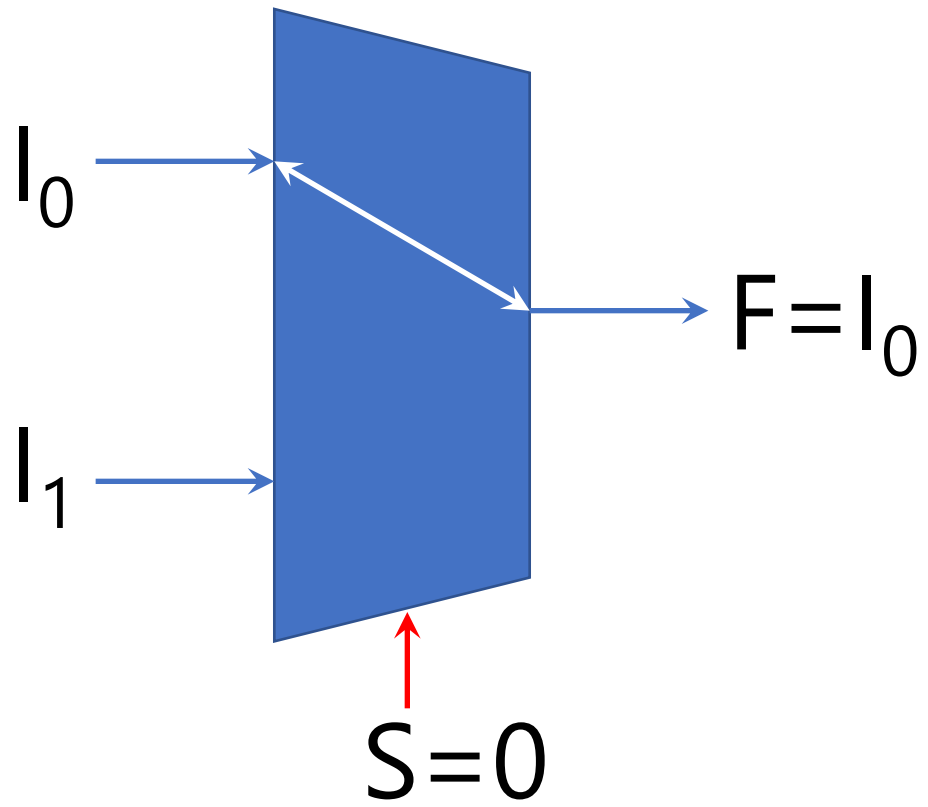


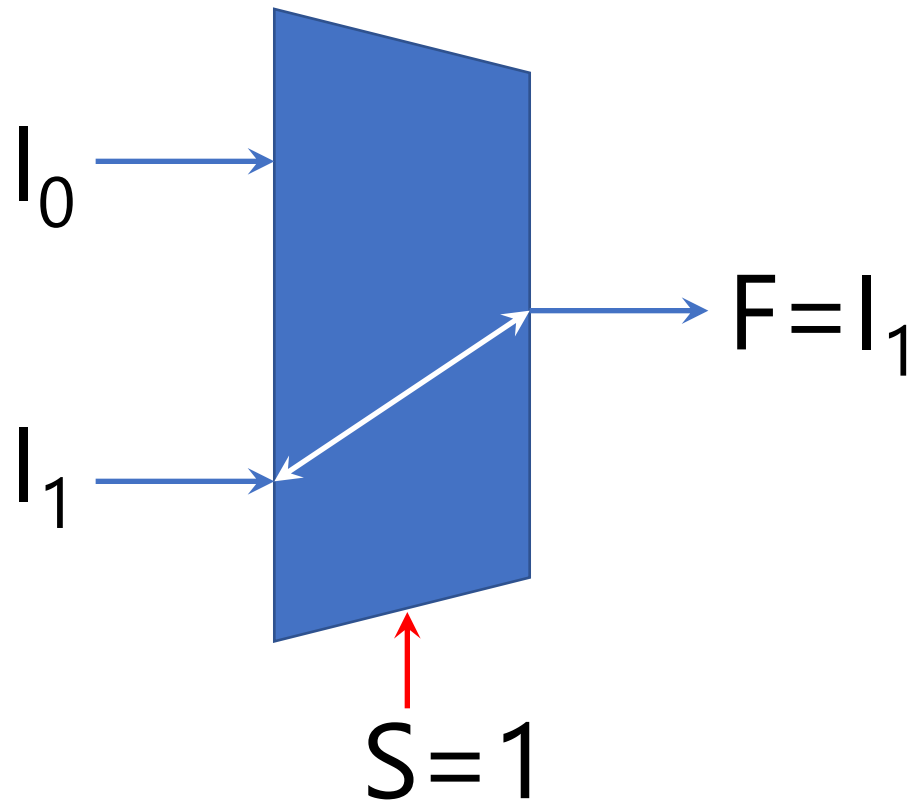


Multiplexer

$2^1 \times 1$





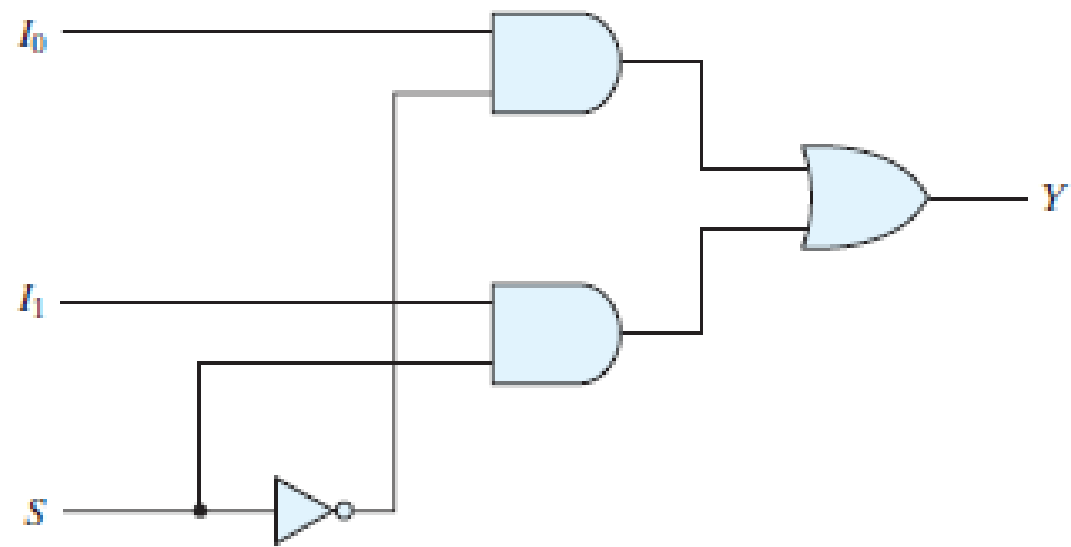


S	I ₁	I ₀	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

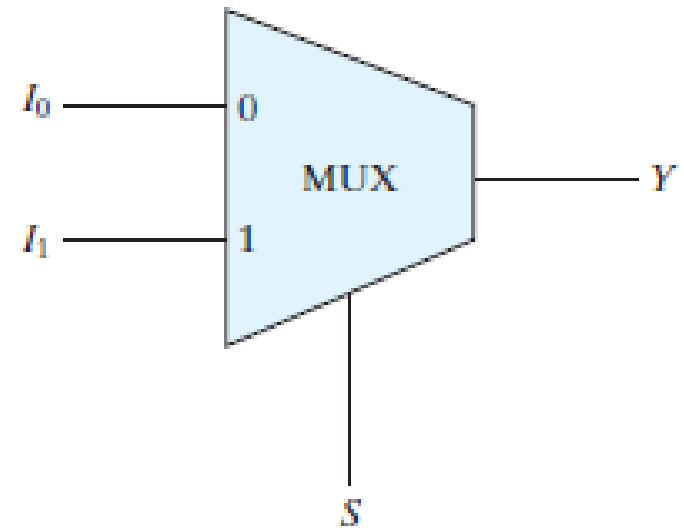
S	I ₁	I ₀	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

		$I_1 I_0$			
		00	01	11	10
S	0	0 m_0	1 m_1	1 m_3	0 m_2
	1	0 m_4	0 m_5	1 m_7	1 m_6

$$F = S'I_0 + SI_1$$



(a) Logic diagram



(b) Block diagram

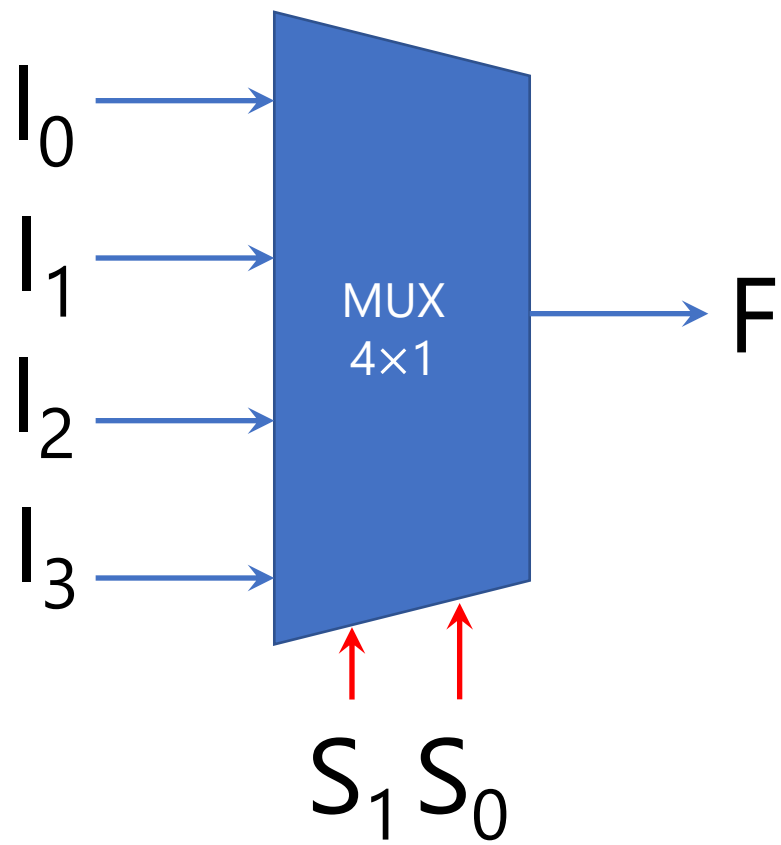
FIGURE 4.24
Two-to-one-line multiplexer

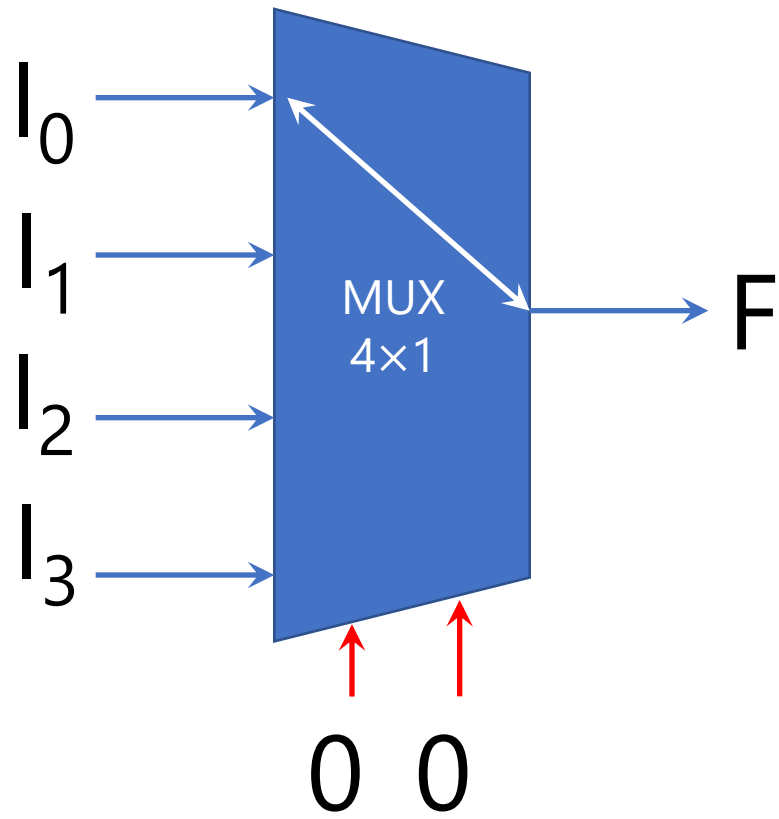
Not an official truth table, but condense version

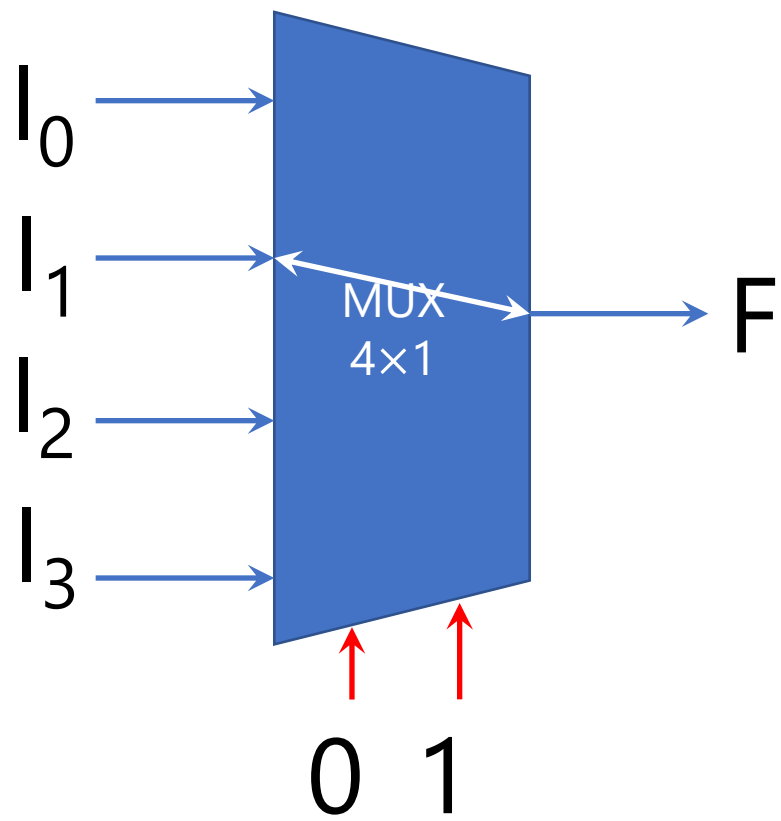
S	$F = S'I_0 + SI_1$
0	I_0
1	I_1

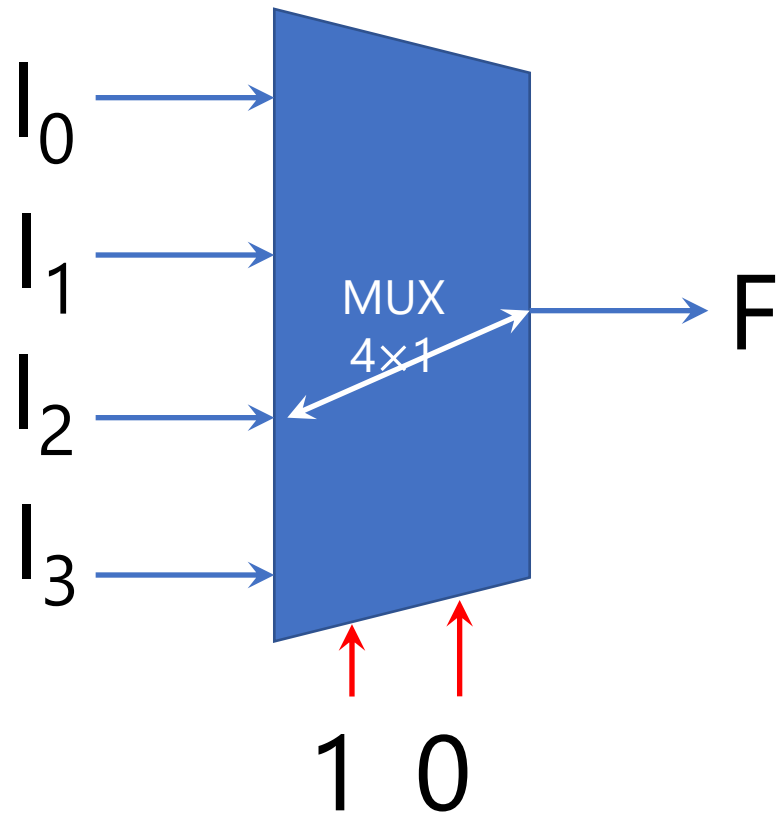
Multiplexer

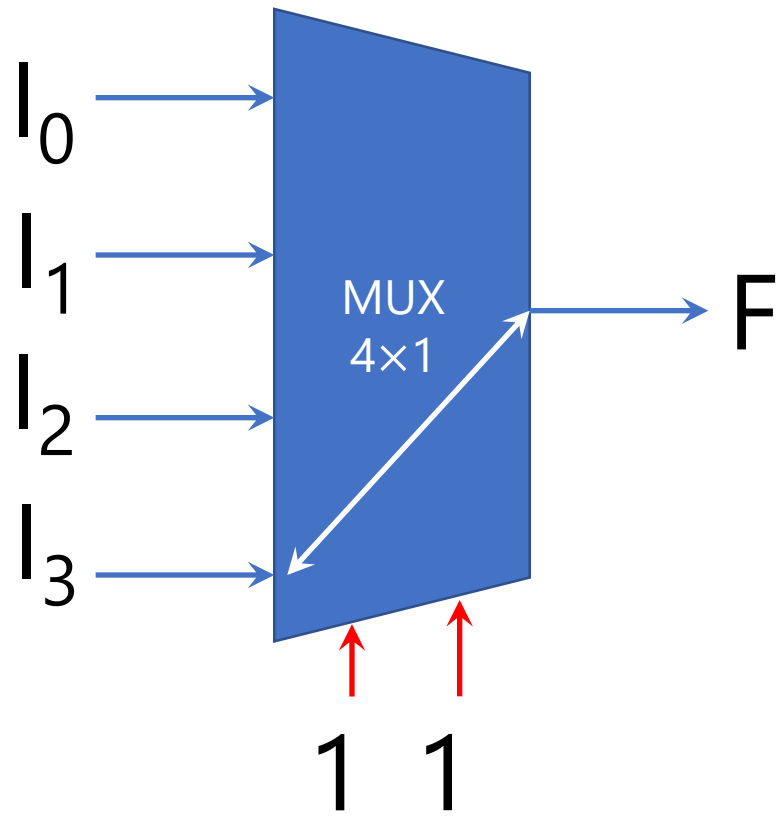
$2^2 \times 1$







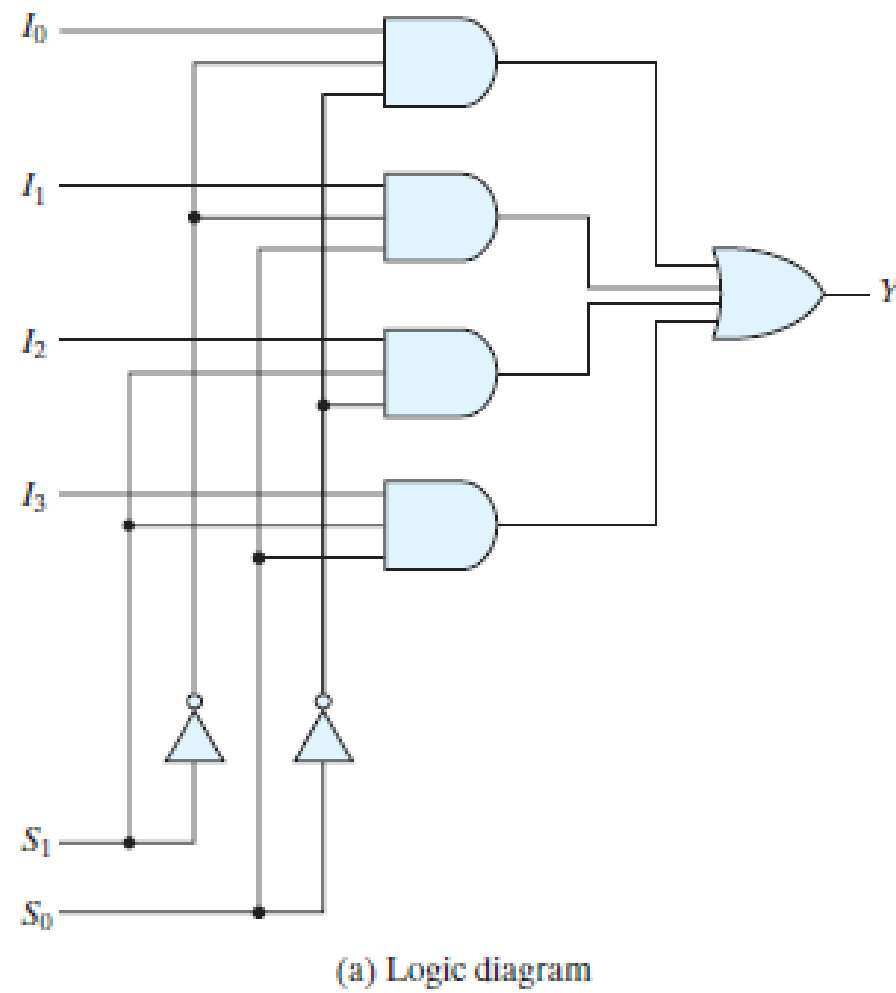




S_1	S_0	I_3	I_2	I_1	I_0	F
0	0	x	x	x	0	0
0	0	x	x	x	1	1
0	1	x	x	0	x	0
0	1	x	x	1	x	1
1	0	x	0	x	x	0
1	0	x	1	x	x	1
1	1	0	x	x	x	0
1	1	1	x	x	x	1

Indeed,
there are
 2^6 rows!

S_1	S_0	$F = S'_1 S'_0 I_0 + S'_1 S_0 I_1 + S_1 S'_0 I_2 + S_1 S_0 I_3$
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



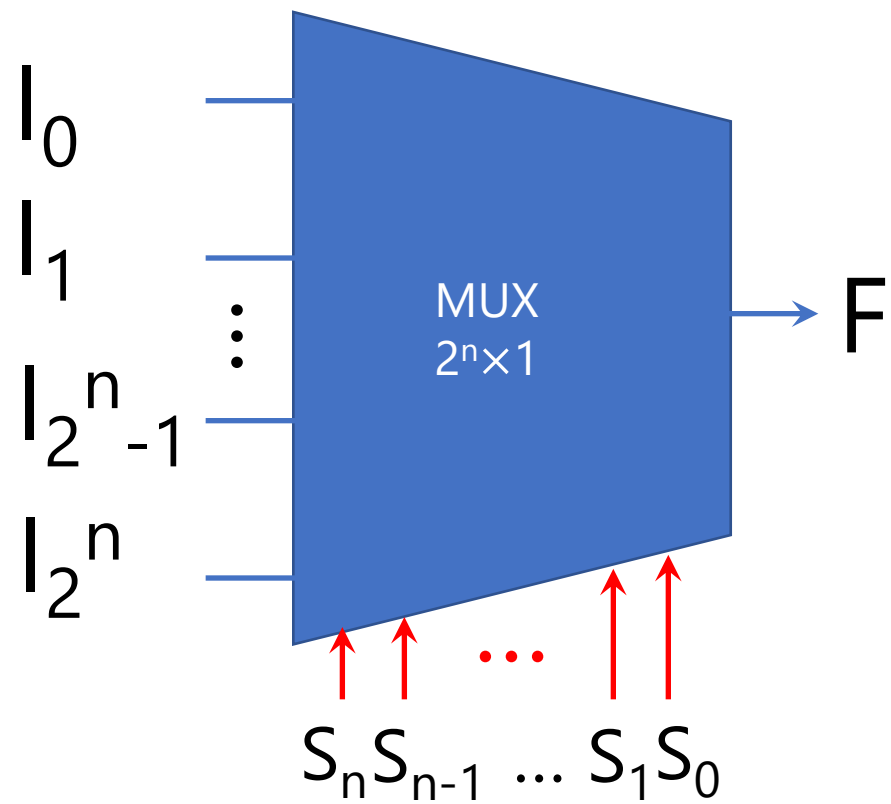
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

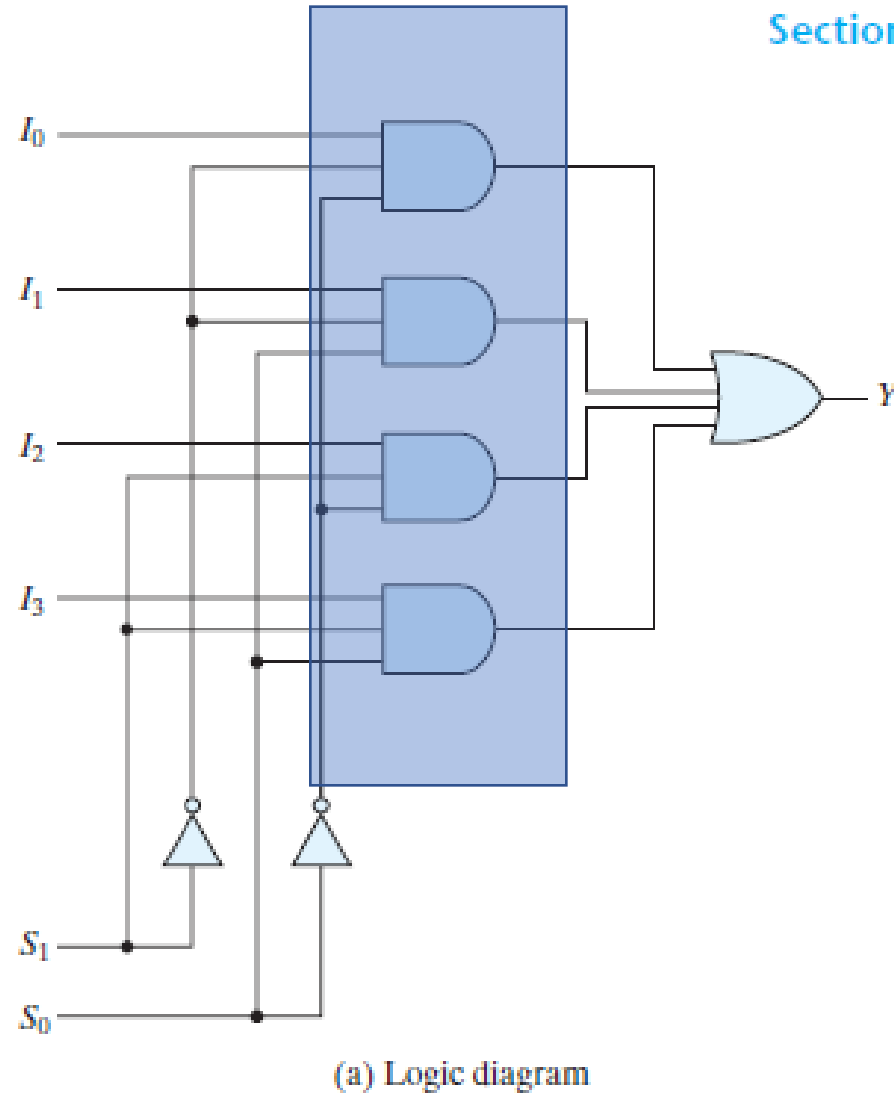
(b) Function table

FIGURE 4.25
Four-to-one-line multiplexer

Multiplexer

$2^n \times 1$



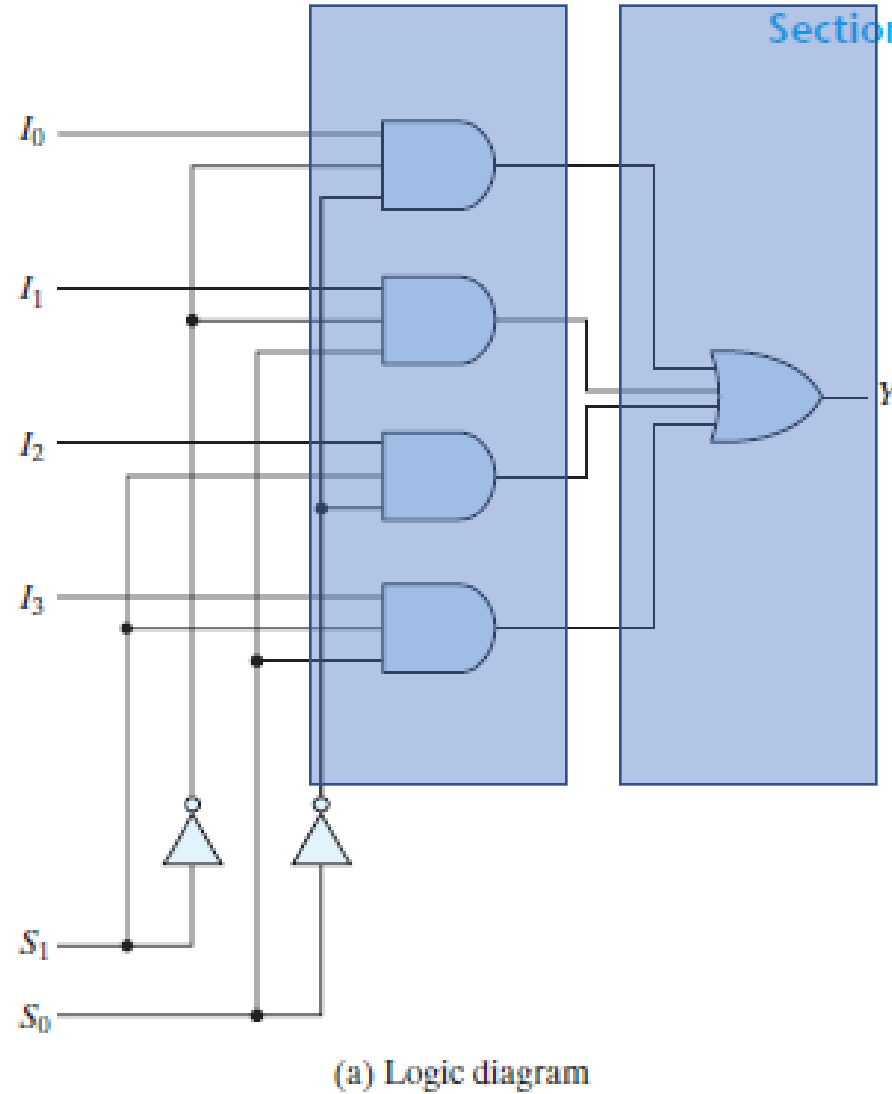


$\approx$ Decoder + OR
(Off the road to our computer system)

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

FIGURE 4.25
Four-to-one-line multiplexer



Sum of Products

2 Levels

ANDs-OR

(Off the road to our computer system)

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

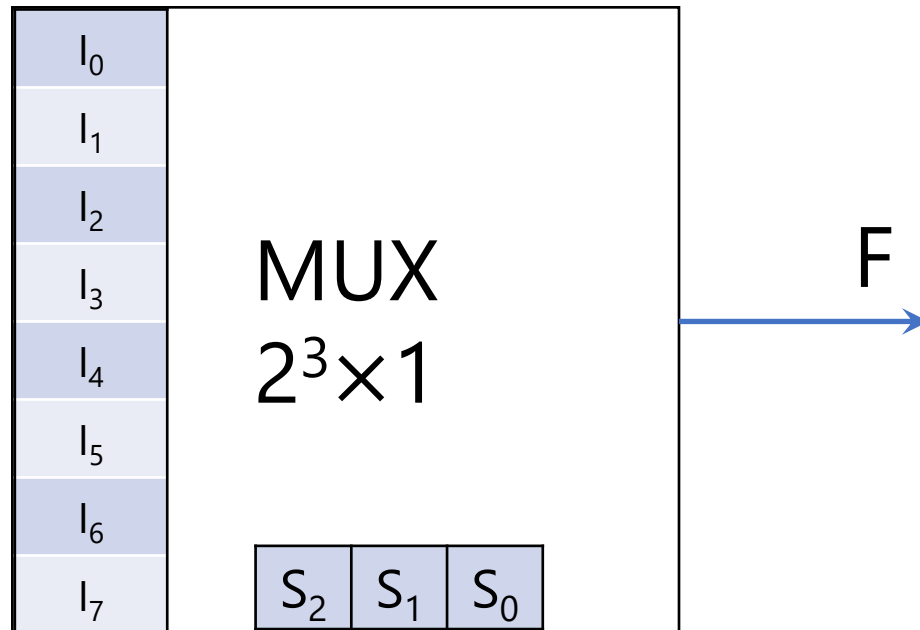
FIGURE 4.25
Four-to-one-line multiplexer

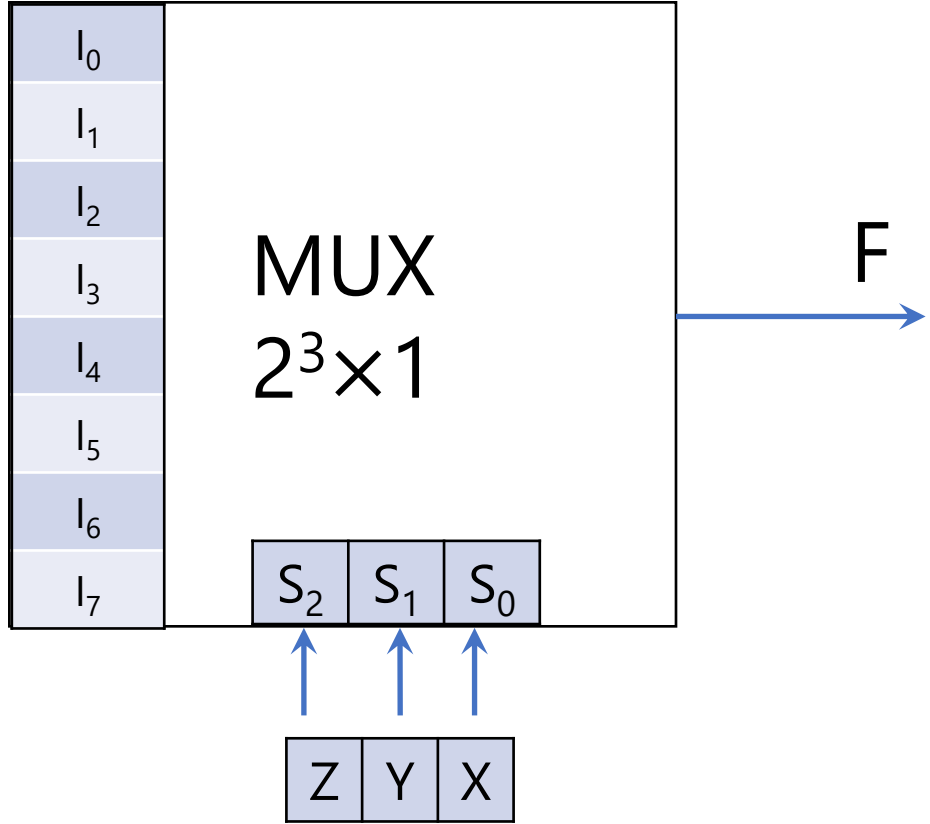
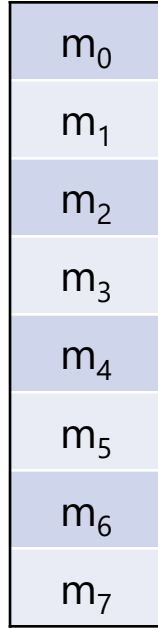
Multiplexer

Boolean Function

(Off the road to our computer system)

$$F_{\text{SoP}} = \sum m(\dots)$$





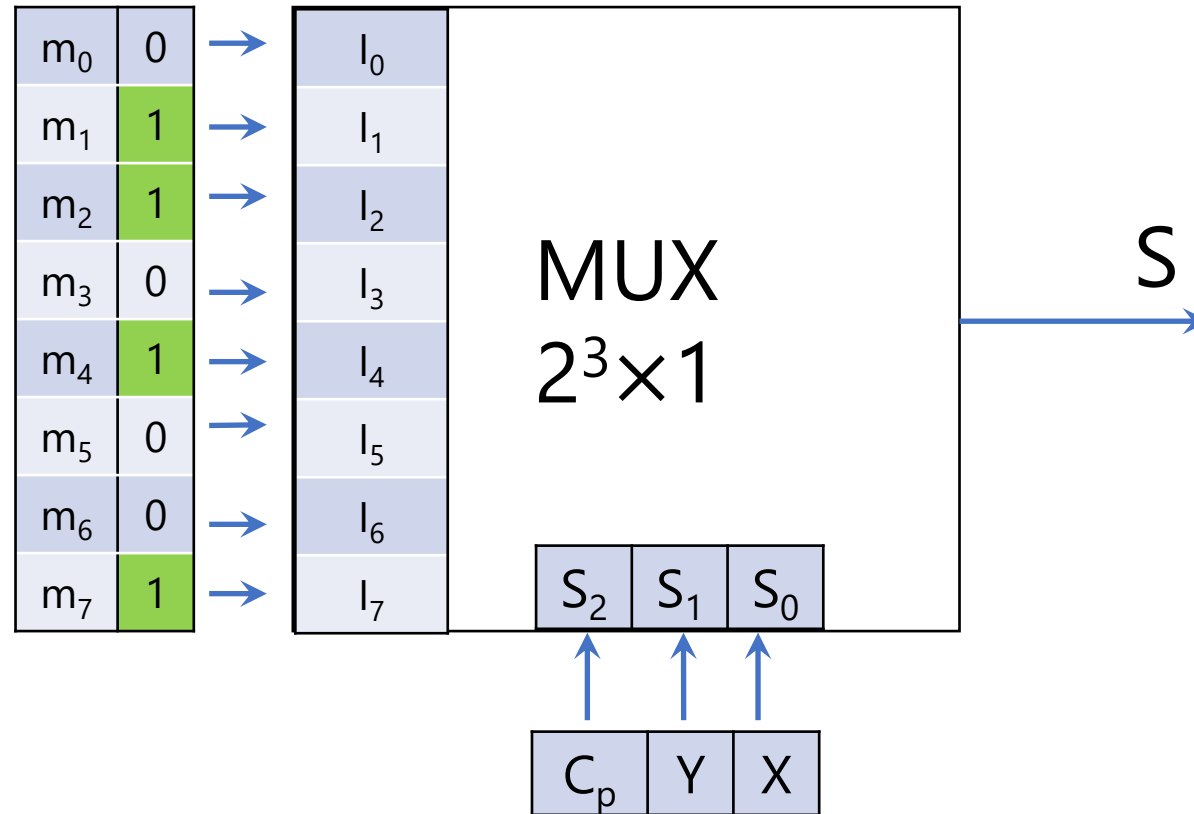
MUX

Full Adder

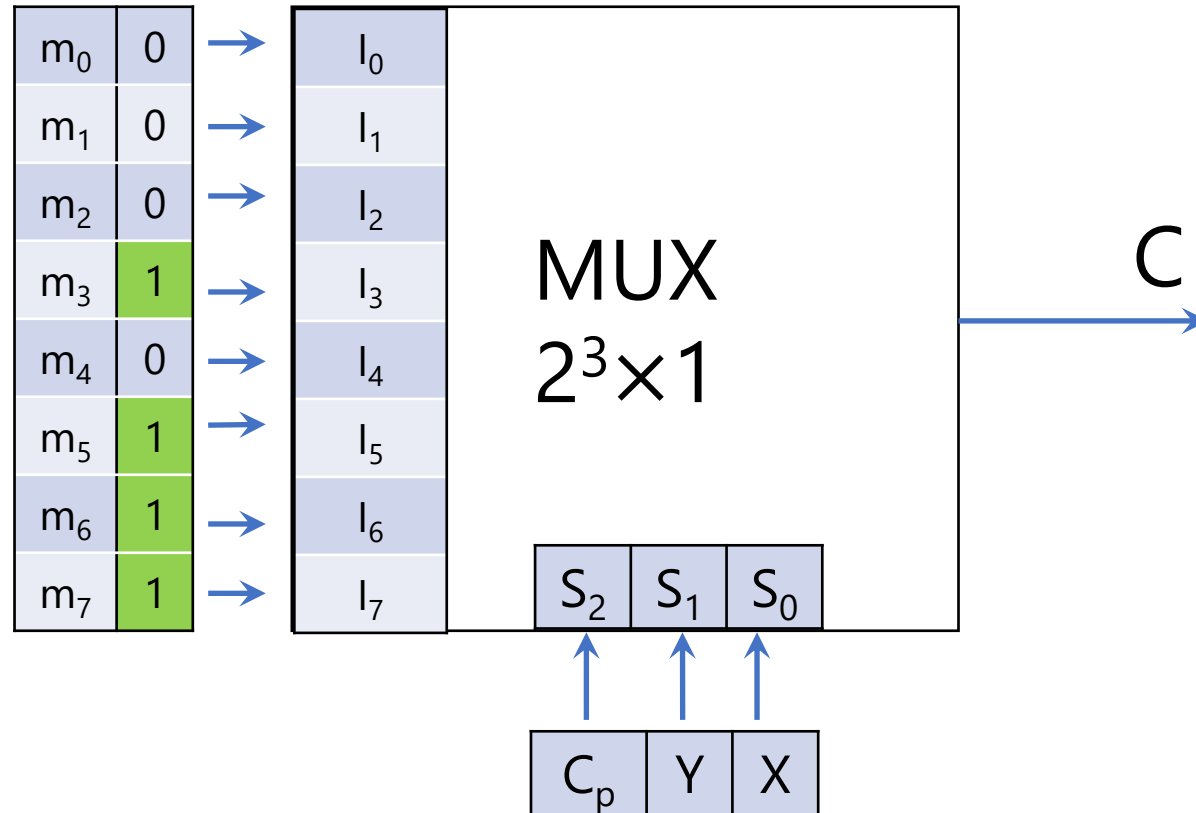
$$S = \sum m(1,2,4,7)$$

$$C = \sum m(3,5,6,7)$$

C_p	Y	X	$C = \sum m(3,5,6,7)$	$S = \sum m(1,2,4,7)$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



$$S = \sum m(1,2,4,7)$$

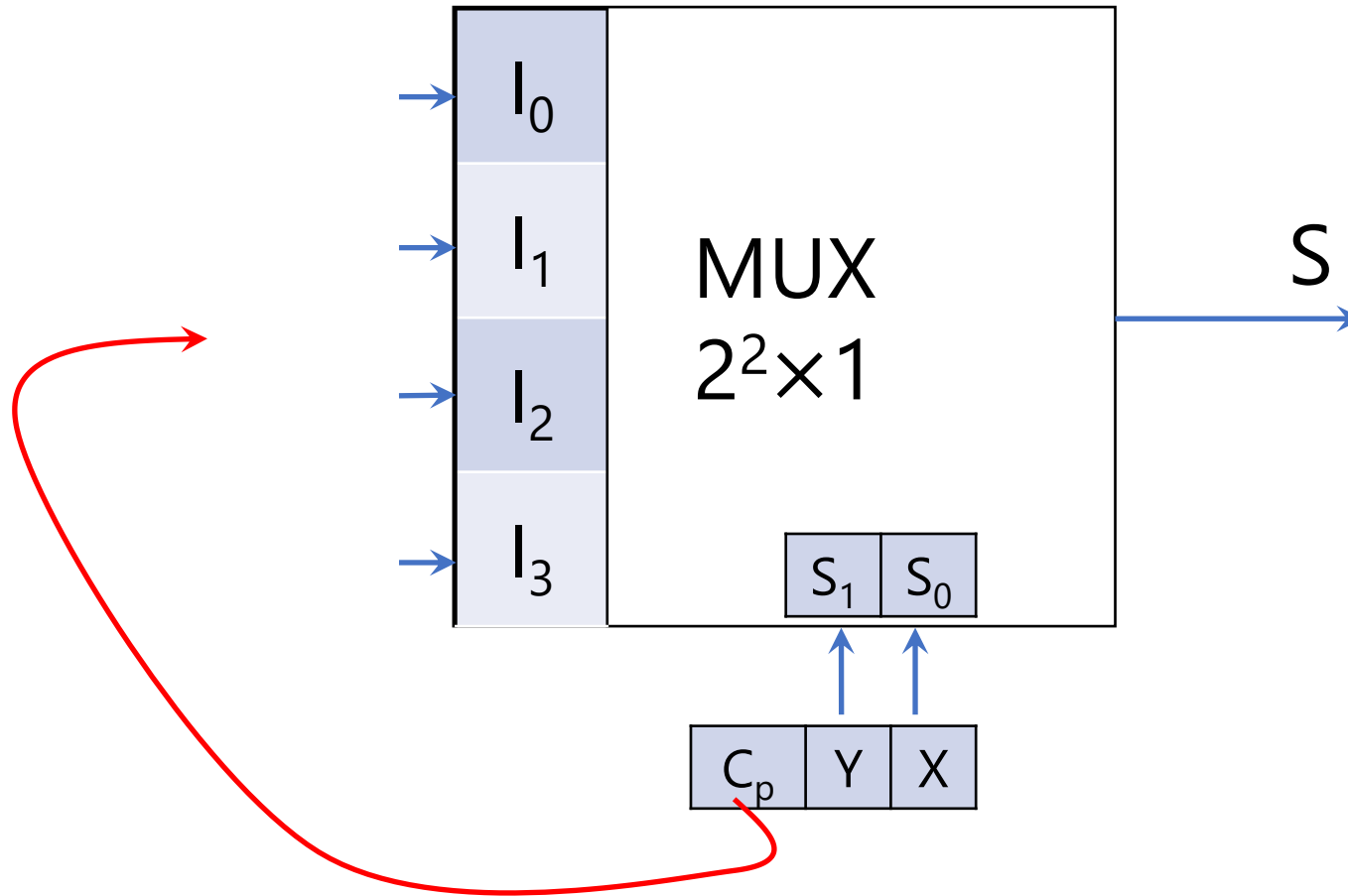


$$C = \sum m(3,5,6,7)$$

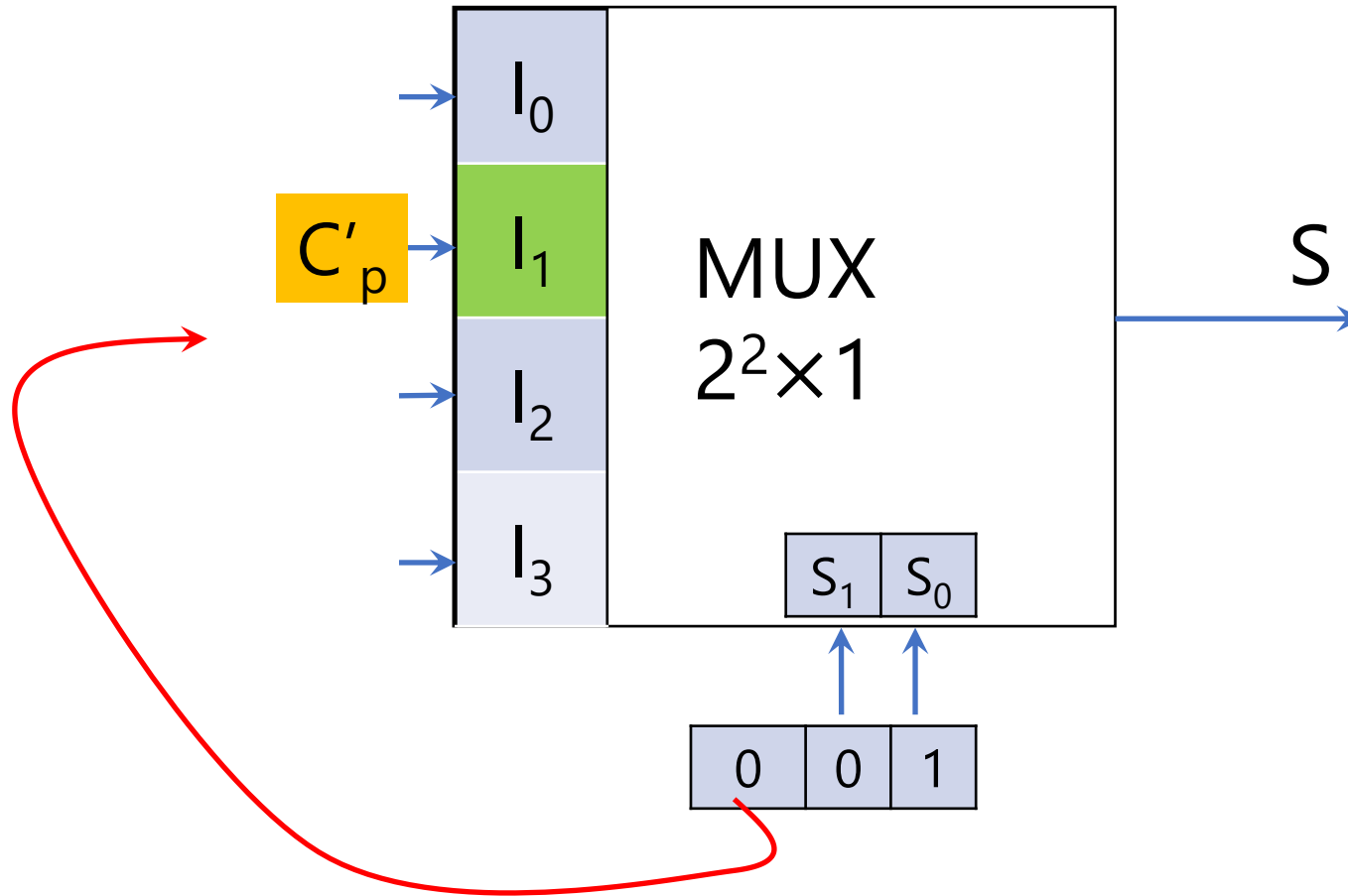
Multiplexer

Boolean Function II

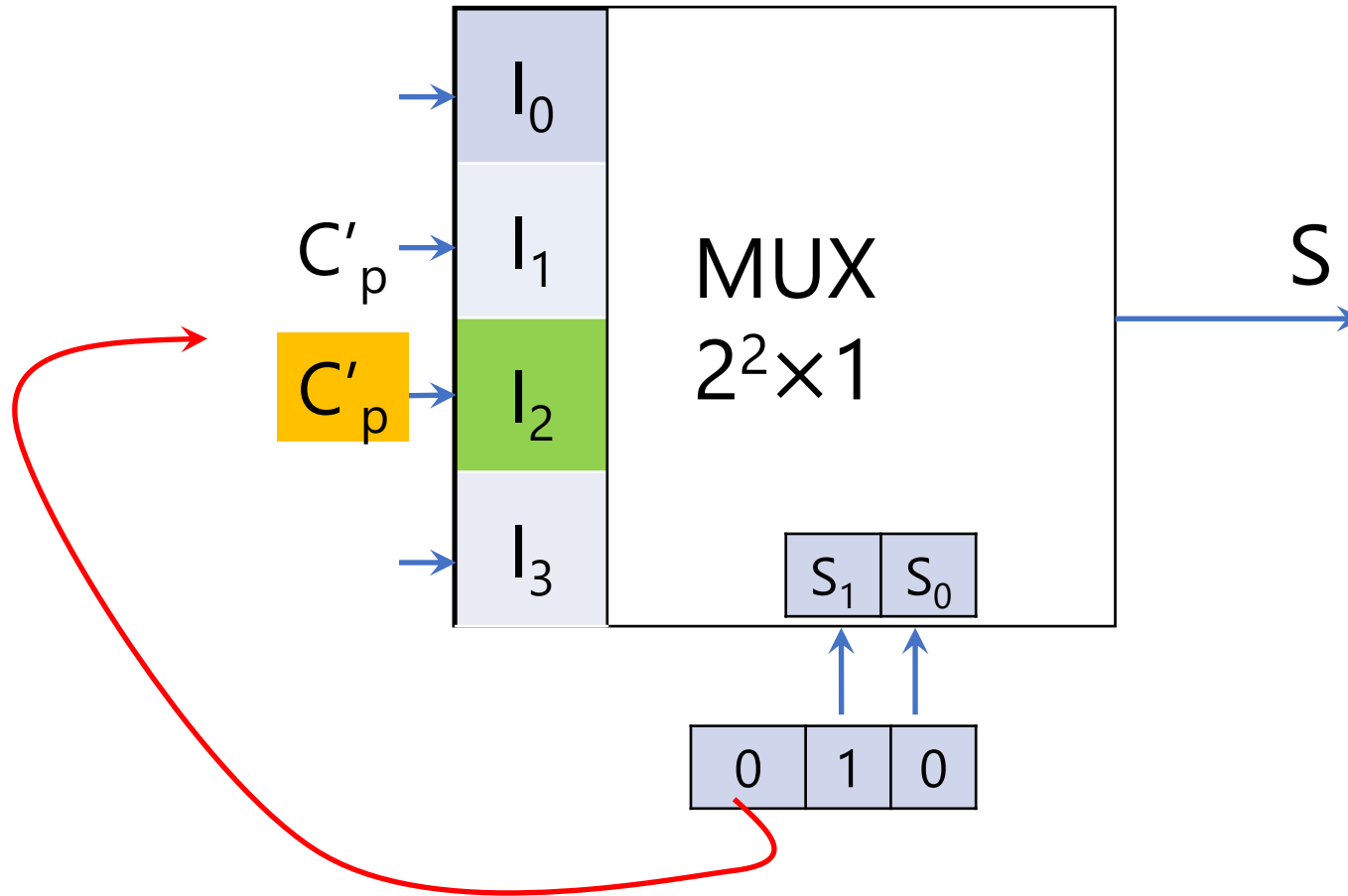
Book: Page 161



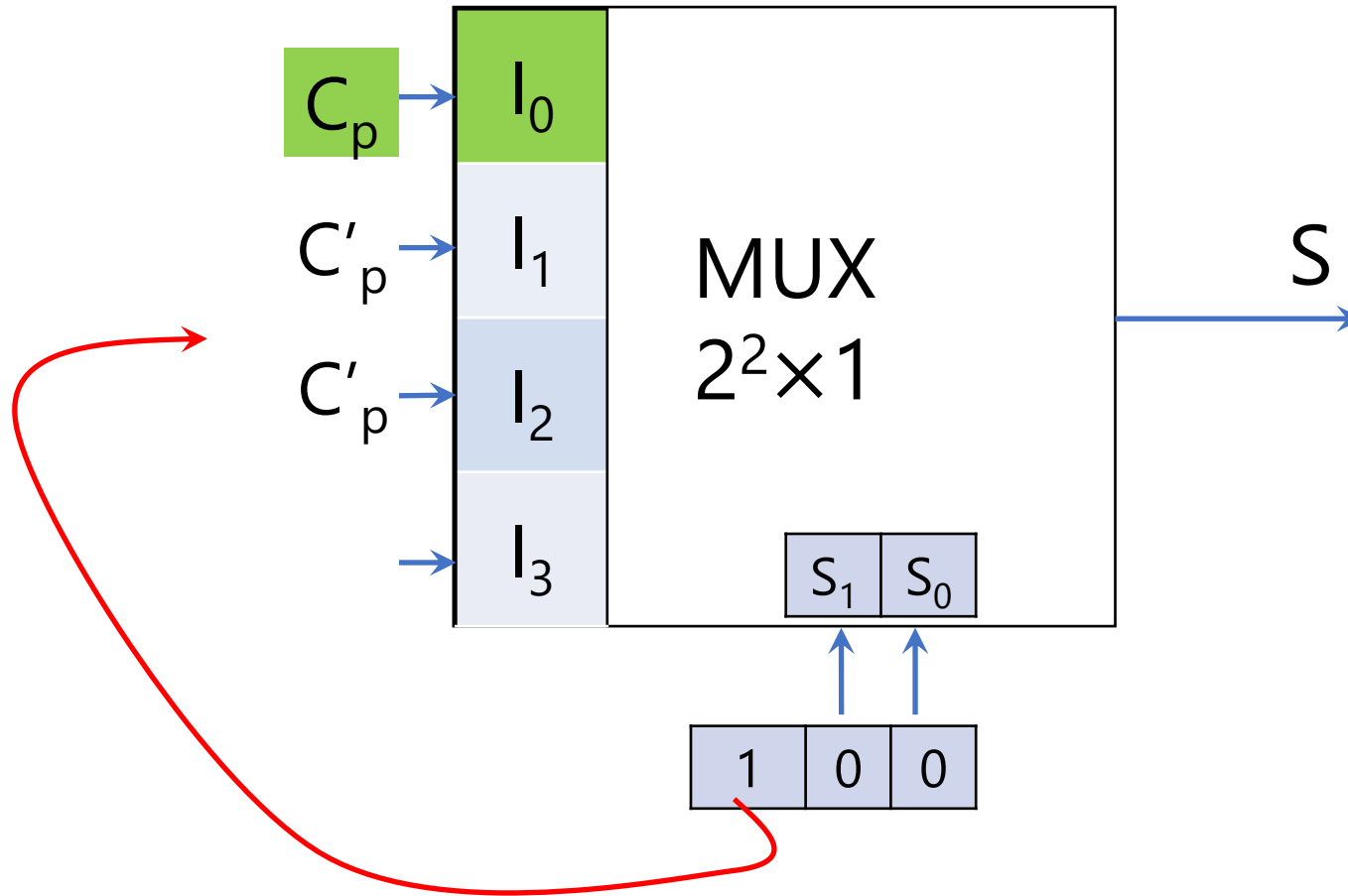
$$S = \sum m(1,2,4,7)$$



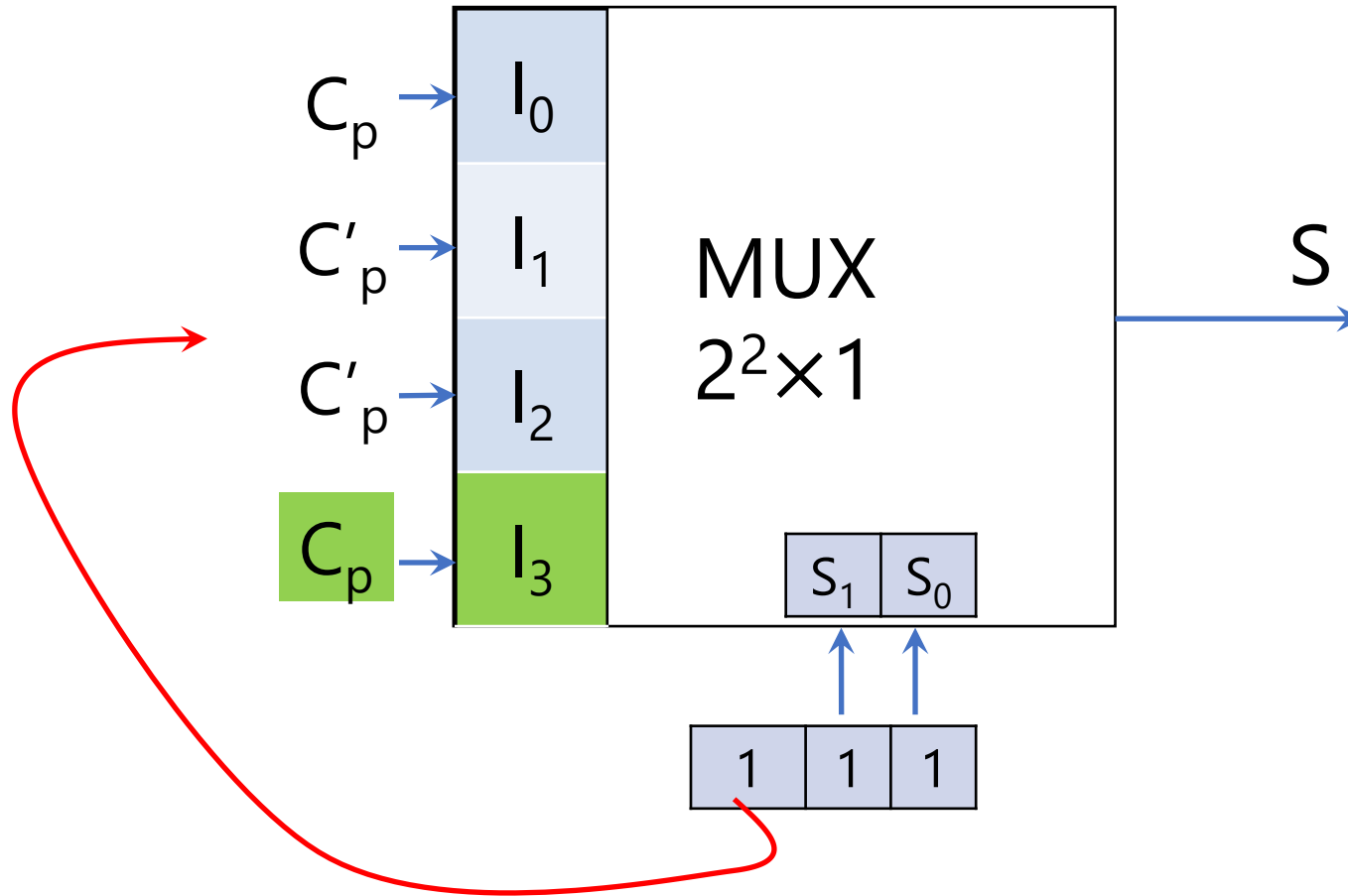
$$S = \sum m(\textcolor{red}{1}, 2, 4, 7)$$



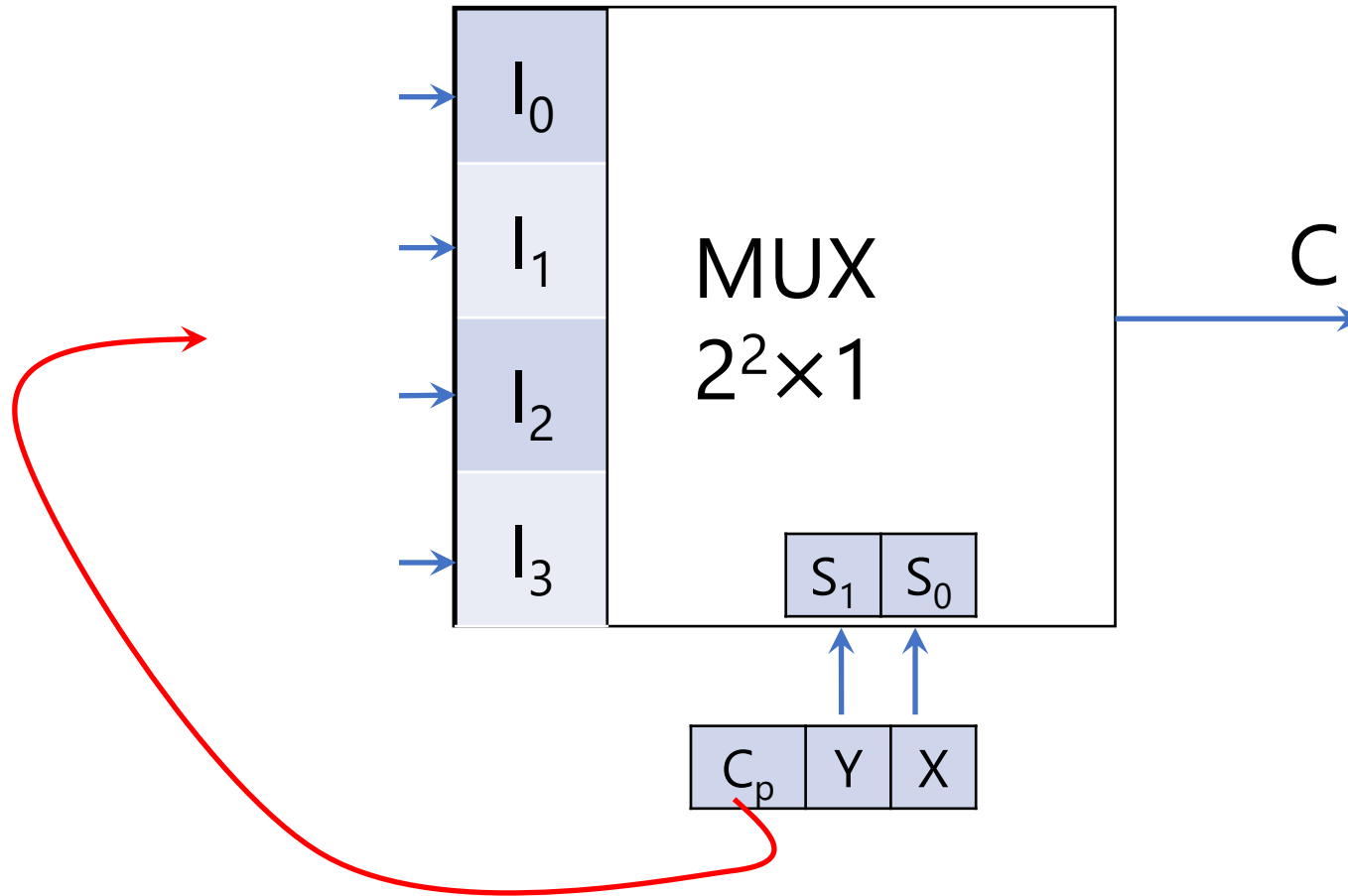
$$S = \sum m(1, \textcolor{red}{2}, 4, 7)$$



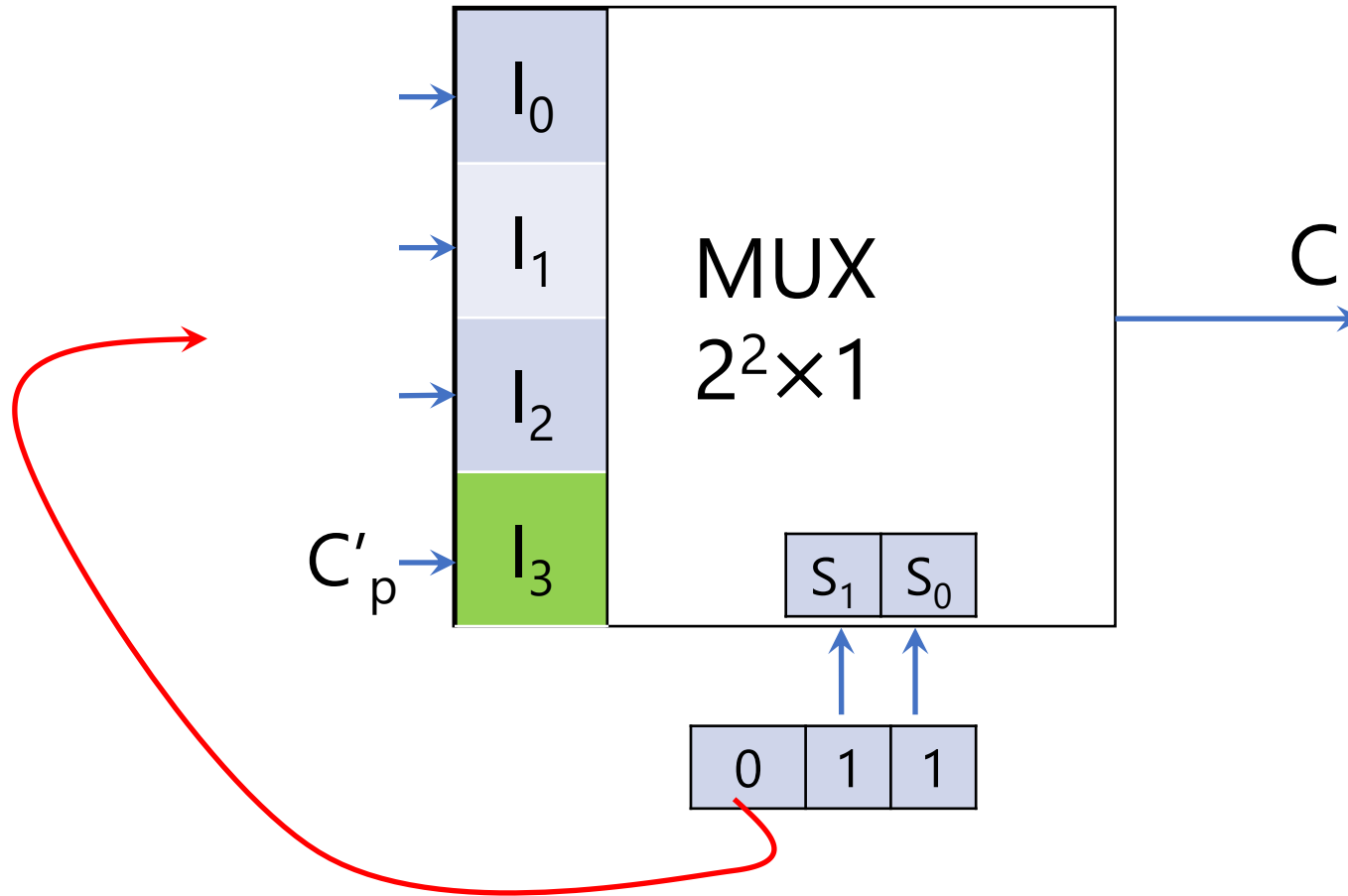
$$S = \sum m(1, 2, \textcolor{red}{4}, 7)$$



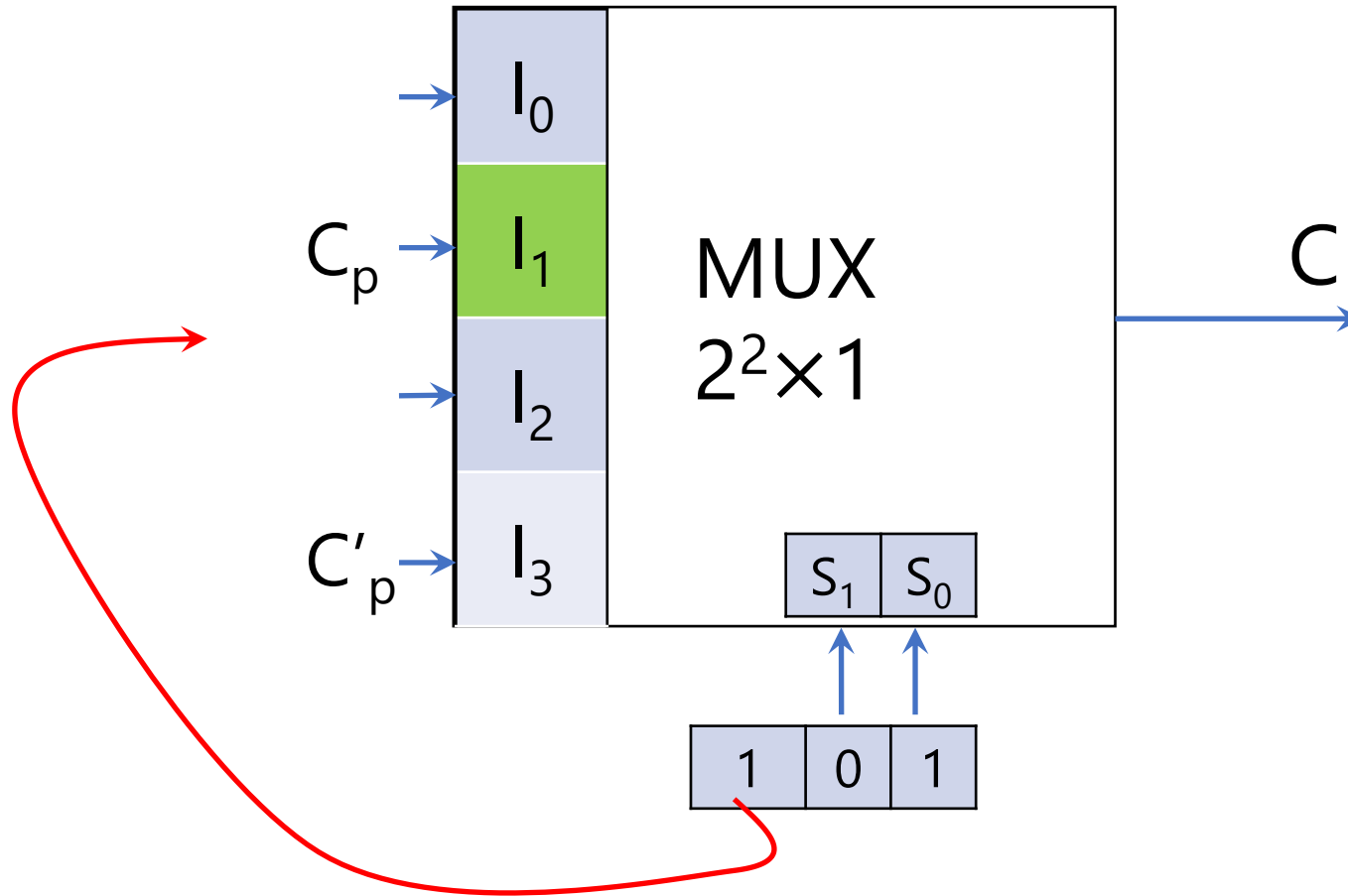
$$S = \sum m(1,2,4,7)$$



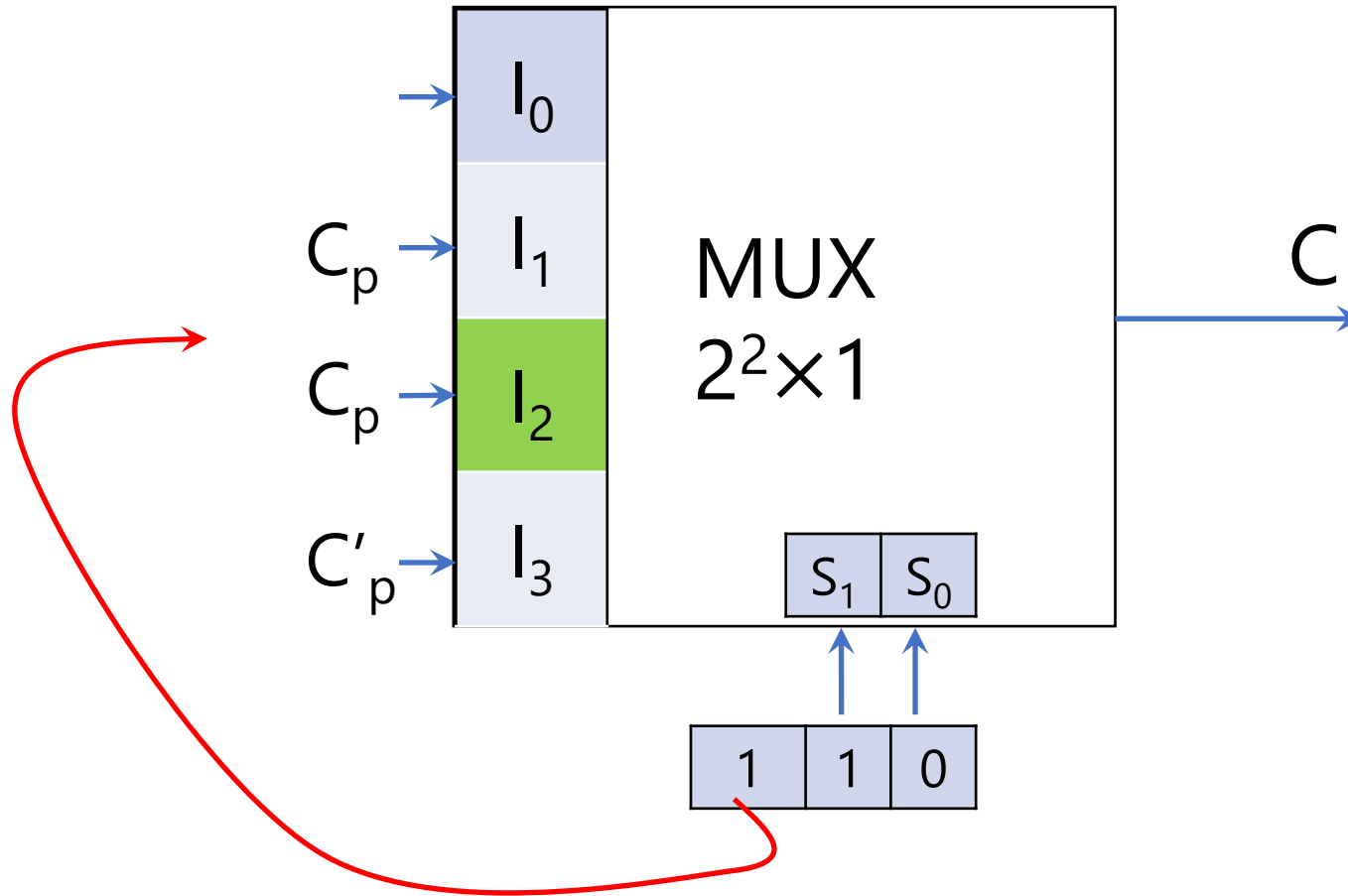
$$C = \sum m(3, 5, 6, 7)$$



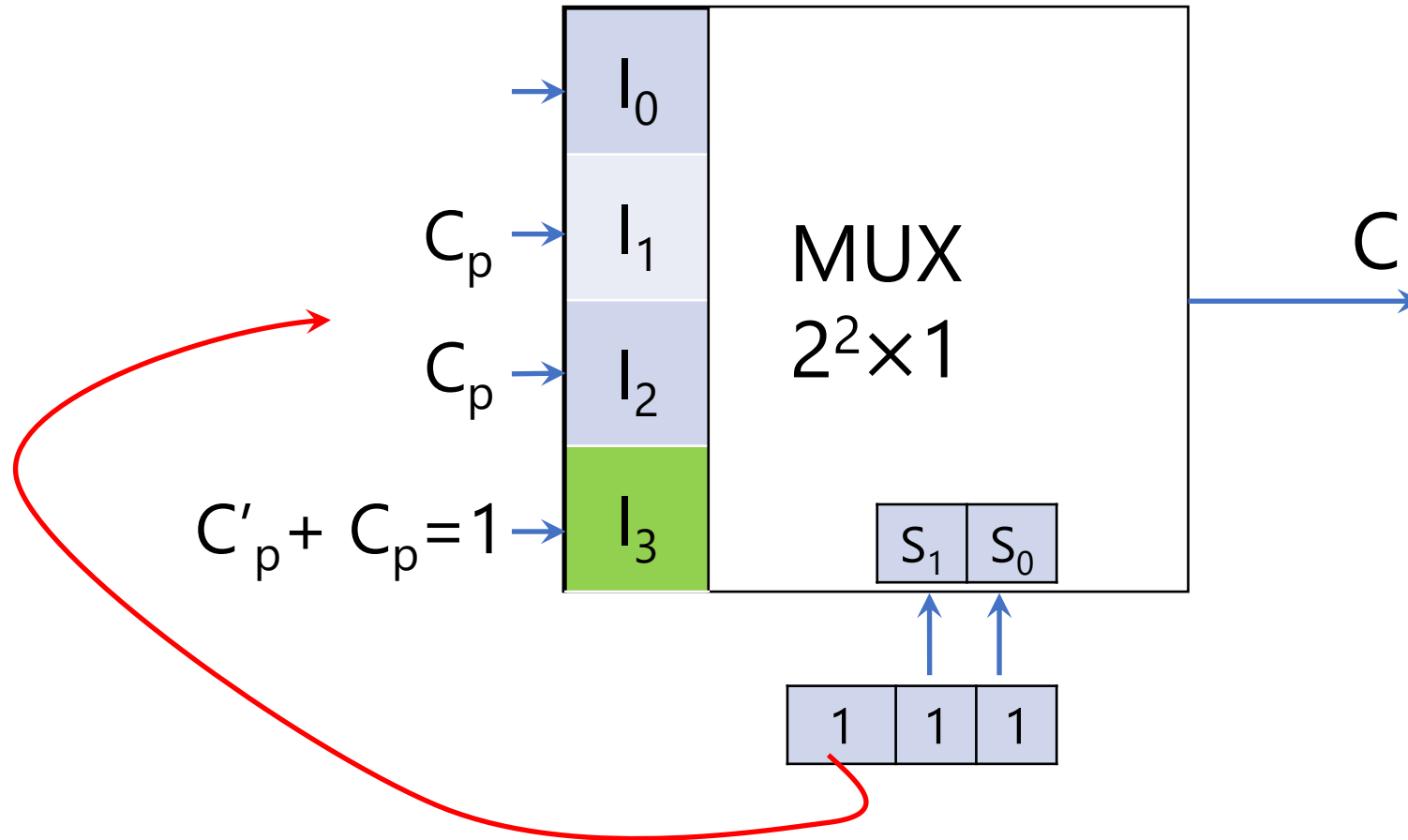
$$C = \sum m(\textcolor{red}{3}, 5, 6, 7)$$



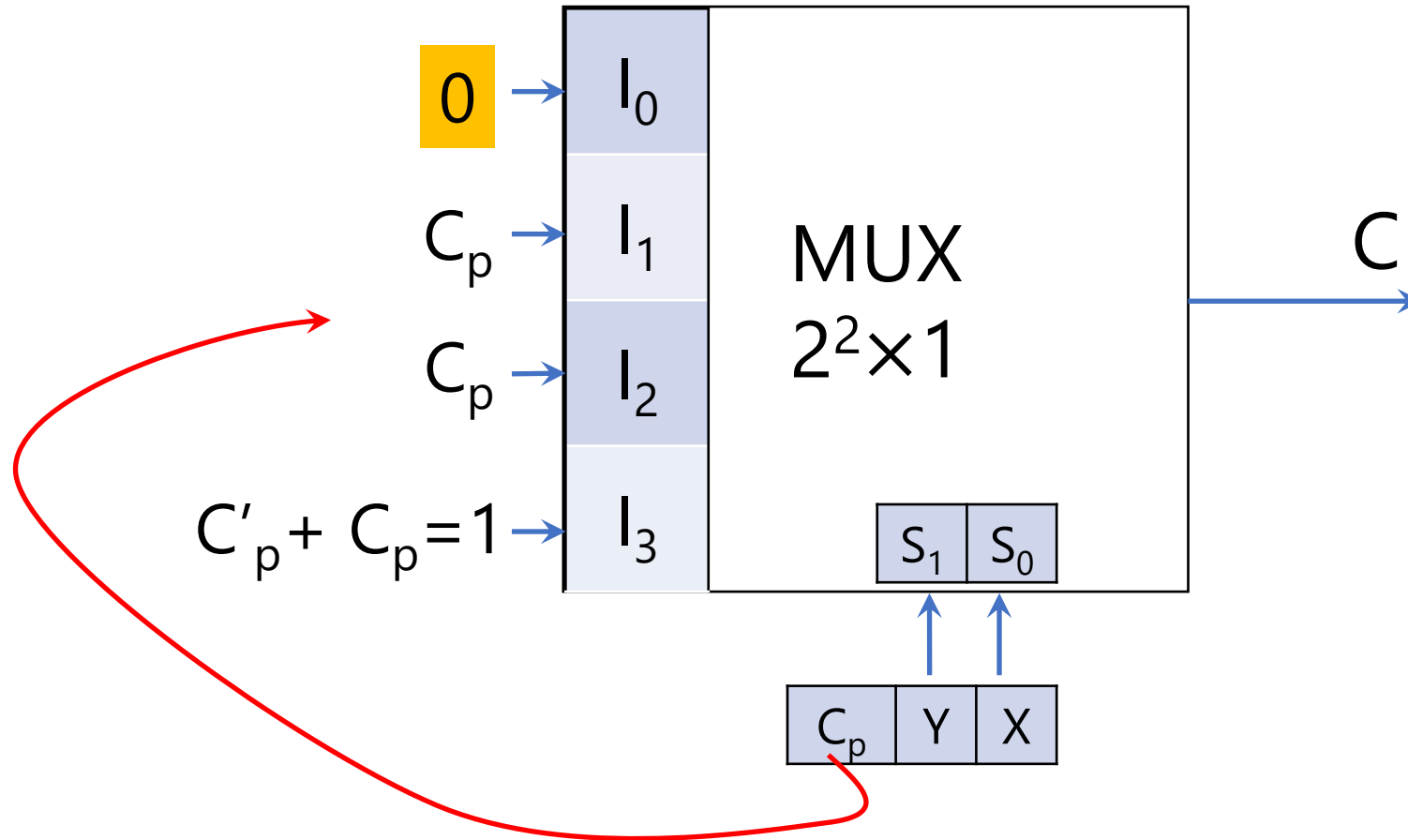
$$C = \sum m(3, \textcolor{red}{5}, 6, 7)$$



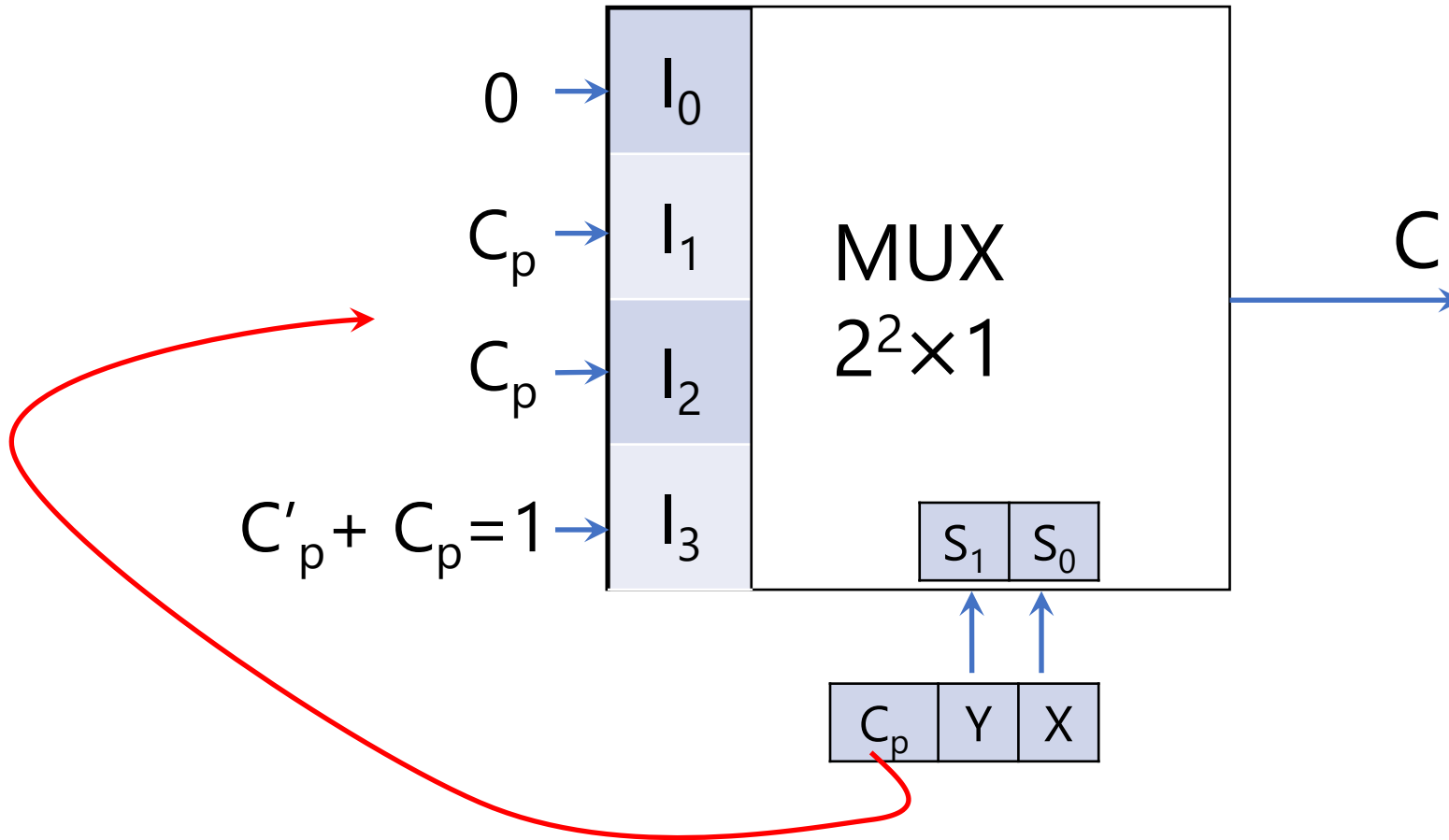
$$C = \sum m(3, 5, \textcolor{red}{6}, 7)$$



$$C = \sum m(3, 5, 6, 7)$$



$$C = \sum m(3, 5, 6, 7)$$



$$C = \sum m(3, 5, 6, 7)$$

